7115056039 張嘉晏

1. 資料集來源與研究脈絡

(1) 資料集來源

本研究使用 Kaggle 平台提供的公開資料集 **Student Performance Factors**，由 lainguyn123 發布。資料集包含 **6,607 筆學生資料，共 20 個欄位**，其中 19 個為輸入特徵（Features），1 個為預測目標（Target）。

資料內容涵蓋學生的讀書時間、出席率、過往成績、睡眠時間、家庭背景及學校環境等因素，預測目標為 **Exam_Score（考試成績）**。

本資料集屬於合成資料（Synthetic Data），主要用於資料分析與機器學習研究。

資料集資訊：

- 資料集名稱：Student Performance Factors
- 資料來源：Kaggle
- 發布者：lainguyn123
- 資料筆數：6,607 筆
- 輸入特徵：19 個（6 個數值特徵、13 個類別特徵）
- 預測目標：Exam_Score
- 資料集連結：https://www.kaggle.com/datasets/lainguyn123/student-performance-factors

(2) 研究背景與動機

學生的考試成績可能受到多種因素影響，例如讀書時間、出席率、過往成績及家庭環境等。透過機器學習技術，可以分析這些特徵與考試成績之間的關係，並建立預測模型。

本研究選擇**多元線性回歸（Multiple Linear Regression）**作為主要預測方法，因為線性回歸具有模型結構簡單、容易理解及可解釋性較高等優點。

此外，當模型使用較多特徵時，並非所有特徵都具有相同的預測貢獻。因此，本研究加入**特徵選擇（Feature Selection）**，探討是否能減少模型使用的特徵數量，同時維持良好的預測準確度。

(3) 研究目的

本研究依循 CRISP-DM 資料探勘流程，使用學生相關特徵預測考試成績，並比較以下三種模型：

- Model A：Simple Linear Regression — 僅使用讀書時間預測成績，作為基準模型。
- Model B：Multiple Linear Regression — 使用全部 19 個原始輸入特徵建立預測模型。
- Model C：Multiple Linear Regression + RFE — 使用遞迴特徵消除法（Recursive Feature Elimination）選擇特徵，評估模型簡化後的預測效果。

最後透過 MAE、RMSE、R^2 及 95% 預測區間評估模型表現，分析特徵選擇對預測能力的影響，並探討模型的應用限制。

2. CRISP-DM 流程完整且邏輯清楚

(1) Business Understanding

本研究旨在利用學生的學習習慣、家庭背景與學校環境等特徵，建立考試成績預測模型，並探討不同特徵組合對預測準確度的影響。

研究以線性回歸為主要方法，比較單純線性回歸、多元線性回歸，以及結合 RFE 特徵選擇的多元線性回歸，評估增加或減少特徵是否影響預測能力。

(2) Data Understanding

本研究使用 Kaggle 的 Student Performance Factors 資料集，共包含 **6,607 筆資料、19 個輸入特徵**，其中 6 個為數值特徵、13 個為類別特徵。

透過探索式資料分析（EDA）檢查資料分布、缺失值及相關係數。結果顯示：

- 平均考試成績約為 **67.24 分**。
- Attendance（出席率）與考試成績的相關係數約為 **0.58**。
- Hours_Studied（讀書時間）與考試成績的相關係數約為 **0.45**。
- Teacher_Quality、Parental_Education_Level 及 Distance_from_Home 分別存在 78、90、67 筆缺失值。
- 資料中有一筆考試成績為 101 分，超出本研究設定的 0～100 分範圍。

上述分析顯示，出席率與讀書時間具有較明顯的正向線性關聯，可作為後續建模分析的參考。

(3) Data Preparation

首先，排除一筆考試成績為 101 分的紀錄，資料由 6,607 筆減少至 **6,606 筆**。

接著將 Exam_Score 設定為預測目標，其餘 19 個欄位作為輸入特徵，並按照 80% 與 20% 的比例切分資料：

- 訓練集：5,284 筆
- 測試集：1,322 筆

在資料前處理方面，使用 SimpleImputer 填補缺失值，數值特徵透過 StandardScaler 進行標準化，類別特徵則使用 One-Hot Encoding 轉換為數值。

經過前處理後，19 個原始輸入特徵轉換為 **27 個編碼後特徵**。所有前處理步驟皆整合於 Pipeline 中，並使用訓練資料進行擬合，以避免資料洩漏。

(4) Modeling

本研究建立三個線性回歸模型進行比較：

- Model A：Simple Linear Regression

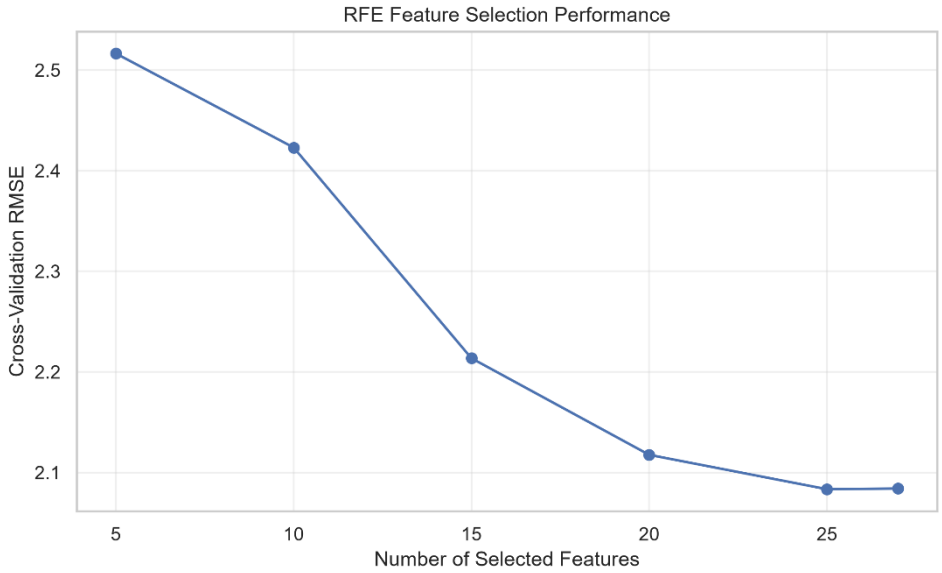
  僅使用 Hours_Studied 作為輸入特徵，建立單純線性回歸模型，作為後續模型比較的基準。

- Model B：Multiple Linear Regression

  使用全部 19 個原始輸入特徵，經過資料前處理後共包含 27 個編碼欄位，建立完整多元線性回歸模型。

- Model C：Multiple Linear Regression + RFE

  使用遞迴特徵消除法（Recursive Feature Elimination, RFE）執行特徵選擇，並搭配五折交叉驗證（5-Fold Cross-Validation），比較不同特徵數量的模型表現。

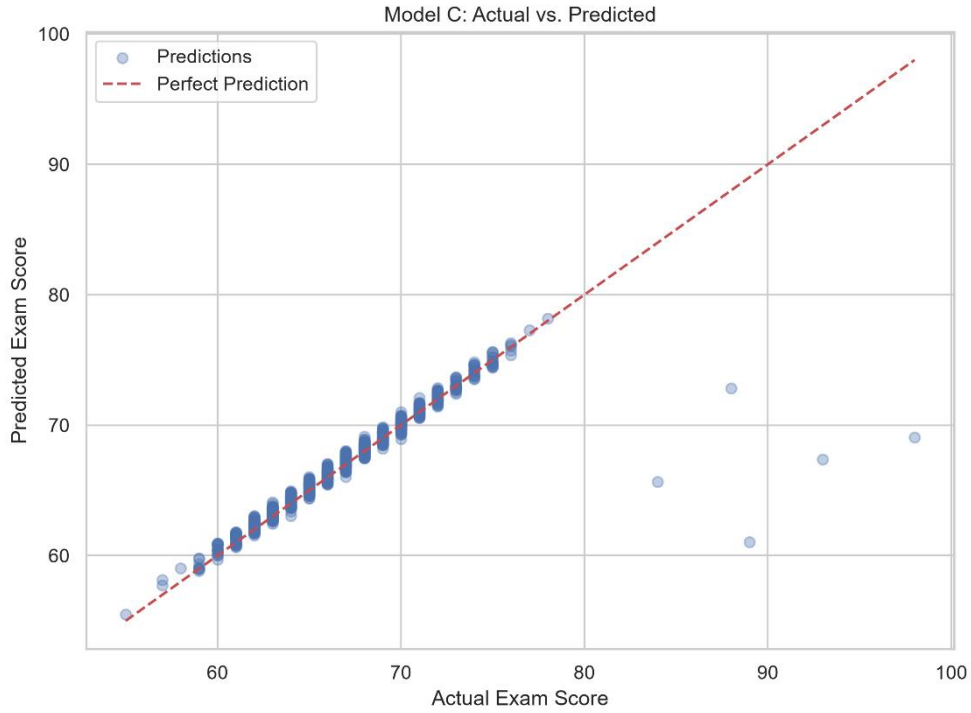


實驗結果顯示，保留 **25 個編碼後特徵**時，交叉驗證 RMSE 最低，約為 **2.0833**。因此，Model C 最終使用 25 個特徵建立預測模型，較 Model B 減少 2 個編碼欄位。

(5) Evaluation

本研究使用 MAE、RMSE 及 R^2 評估三個模型的預測能力。

模型	MAE	RMSE	R^2
Model A	2.4190	3.1559	0.2469
Model B	0.4160	1.5213	0.8250
Model C	0.4157	1.5210	0.8251



結果顯示，Model B 與 Model C 的預測能力明顯優於 Model A，說明使用多個特徵有助於改善學生成績預測效果。

Model C 的評估指標略優於 Model B，但差異非常小，主要優勢在於使用較少特徵維持接近完整模型的預測表現。

此外，本研究透過殘差分析發現，模型對大部分學生的預測誤差較小，但仍有少數高分學生被明顯低估。

最後，使用 Split Conformal Prediction 建立 95% 預測區間，測試集涵蓋率（Coverage）達到 95.08%，平均預測區間寬度為 1.6089 分，顯示模型對整體測試資料具有良好的邊際區間涵蓋表現，但不保證對極端高分學生同樣可靠。

(6) Deployment

本研究使用 joblib 儲存完成訓練的 Model C，以及用於建立預測區間的校準模型，並透過重新載入模型測試單筆學生成績預測功能。

實際測試結果如下：

項目	結果
Model C 預測成績	66.01
區間模型預測成績	66.06
實際成績	66
95% 預測區間	[65.25, 66.86]

測試結果顯示，模型能成功重新載入並完成預測，且該筆學生的實際成績落在預測區間內。

由於本研究使用合成資料，模型主要適用於教學及方法驗證，不宜直接應用於真實教育決策。

3. 結果合理、美觀且具有說服力

(1) 模型比較結果

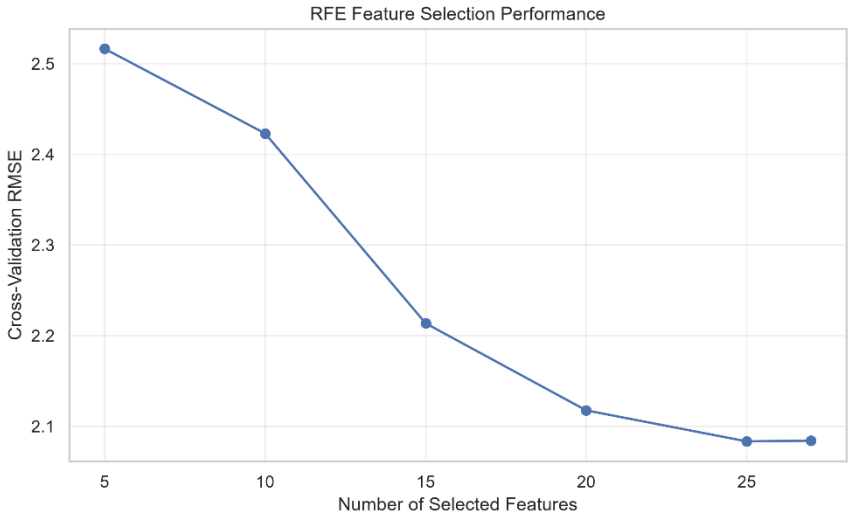
本研究比較單純線性回歸（Model A）、多元線性回歸（Model B）及結合 RFE 特徵選擇的多元線性回歸（Model C）。

實驗結果顯示，Model A 僅使用讀書時間進行預測，R^2 為 0.2469，表示單一特徵的預測能力有限。當 Model B 加入全部 19 個原始特徵後，R^2 提升至 0.8250，顯示使用多個特徵能有效改善預測表現。

Model C 透過 RFE 特徵選擇，將編碼後特徵由 27 個減少至 25 個，R^2 達到 0.8251，與 Model B 幾乎相同。這表示適當移除部分特徵後，仍可維持良好的預測準確度。

(2) 特徵選擇結果

本研究使用 RFE 搭配五折交叉驗證，比較保留 5、10、15、20、25 及 27 個特徵時的模型表現。

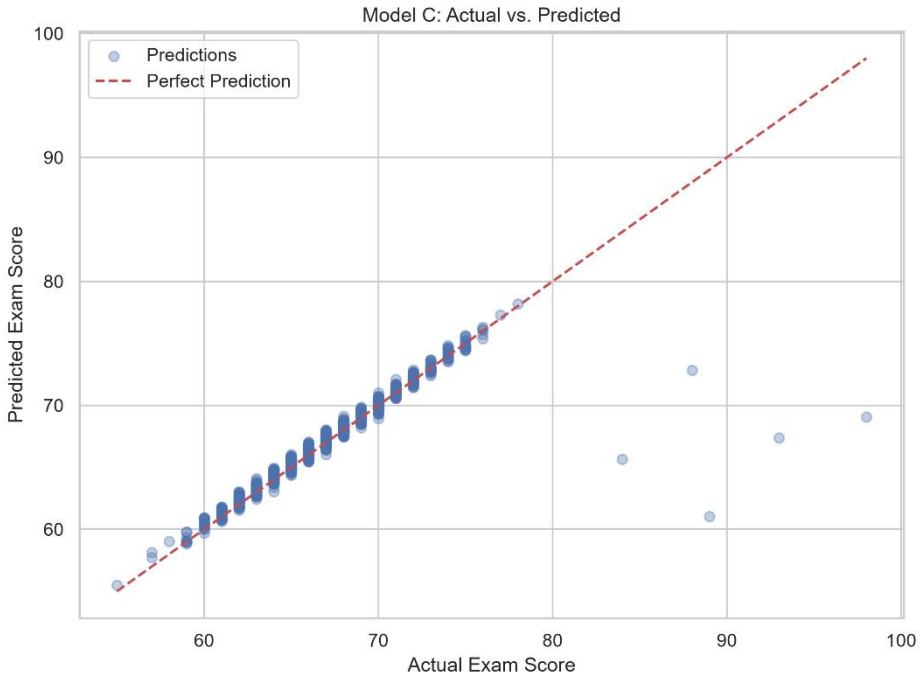


實驗結果顯示，保留 25 個特徵時，交叉驗證 RMSE 最低，為 2.0833。RFE 最終移除了 Sleep_Hours（睡眠時間）及 School_Type_Public（學校類型編碼欄位），保留其餘 25 個特徵。

此結果與先前的相關係數分析部分一致，例如 Sleep_Hours 與考試成績的相關係數約為 -0.02，顯示其線性相關程度較低。

不過，特徵選擇結果僅反映目前資料與模型設定下的預測貢獻，不能直接解讀為這些因素在真實教育環境中不重要。

(3) 預測結果分析



從 Model C 的 Actual vs. Predicted 散佈圖可以觀察到，大部分預測結果接近理想預測線，尤其在主要成績區間內，預測值與實際值相當接近。

然而，少數實際成績較高的學生仍存在明顯低估情況。例如部分學生實際成績為 84～98 分，但模型僅預測約 61～73 分，顯示模型對極端高分資料的預測能力仍有限。

(4) 研究結果討論

綜合三個模型的實驗結果，可以得到以下發現：

- 多元線性回歸優於單純線性回歸： 加入出席率、過往成績及其他特徵後，模型預測能力明顯提升。
- 特徵選擇能略微降低模型複雜度： RFE 減少了約 7.4% 的編碼後特徵數量，仍維持與完整模型接近的預測表現。
- 模型對極端高分仍有限制： 大部分樣本的預測誤差較小，但少數高分紀錄存在明顯低估，顯示整體評估指標無法完整反映每筆資料的預測情況。

整體而言，本研究證實多元線性回歸能有效預測此合成資料集中的學生成績，而 RFE 可以在不明顯影響預測效能的情況下簡化模型。不過，由於研究使用合成資料，結果不宜直接推廣至真實教育情境。

4. 預測結果評估:

(1) 模型評估指標

本研究使用 MAE（平均絕對誤差）、RMSE（均方根誤差）及 R^2（決定係數）作為模型評估指標，並以獨立測試集比較三種線性回歸模型的預測能力。

實驗結果如下：

模型	MAE ↓	RMSE ↓	R^2 ↑
Model A：Simple Linear Regression	2.4190	3.1559	0.2469
Model B：Multiple Linear Regression	0.4160	1.5213	0.8250

Model C：Multiple Linear Regression + RFE	0.4157	1.5210	0.8251

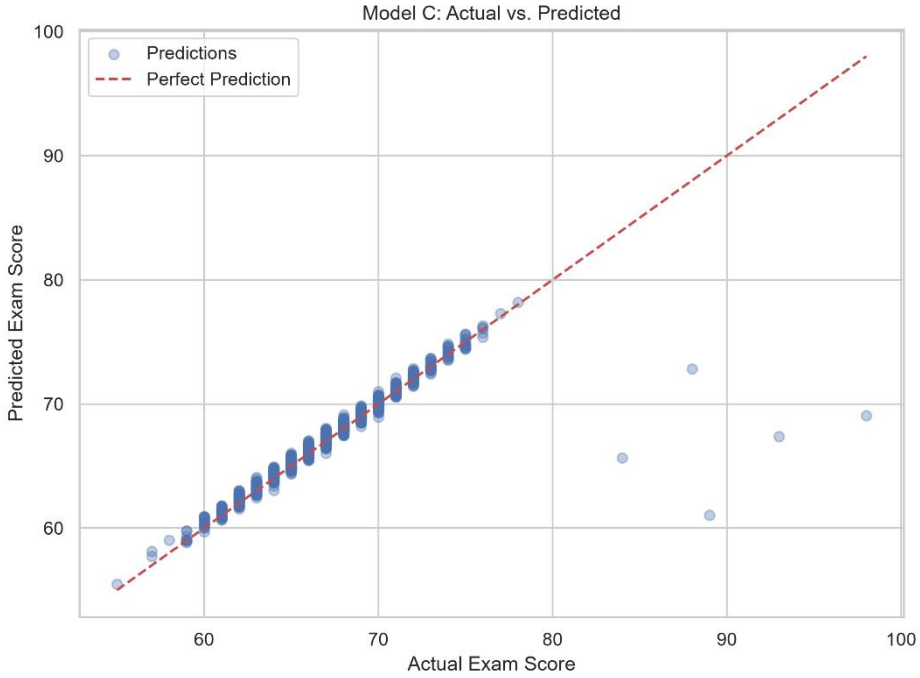
由結果可知，Model B 與 Model C 的預測表現明顯優於 Model A，顯示加入多個特徵有助於提升考試成績的預測能力。

其中 Model C 的 MAE 為 0.4157，代表測試資料的平均絕對預測誤差約為 0.42 分；R^2 為 0.8251，顯示模型能解釋測試資料約 82.51% 的成績變異。

雖然 Model C 的三項指標略優於 Model B，但差異非常小。因此，Model C 的主要優勢是減少兩個編碼後特徵，並維持與完整模型相近的預測效能，而非顯著提升準確度。

(2) 實際值與預測值比較

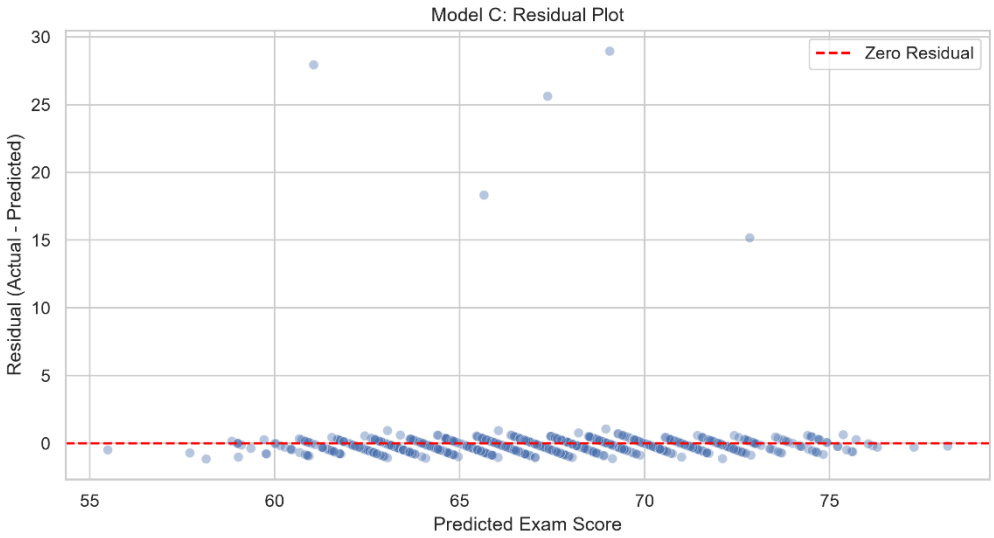
本研究使用 Actual vs. Predicted 散佈圖，比較 Model C 對測試資料的預測結果。



從圖中可以觀察到，大部分預測點接近理想預測線，表示模型對主要成績區間具有良好的預測能力。

然而，部分實際成績較高的學生仍出現明顯低估情況。例如實際成績為 98 分的學生，模型僅預測約 69.05 分，顯示模型對少數極端高分紀錄的預測仍存在限制。

(3) 殘差分析

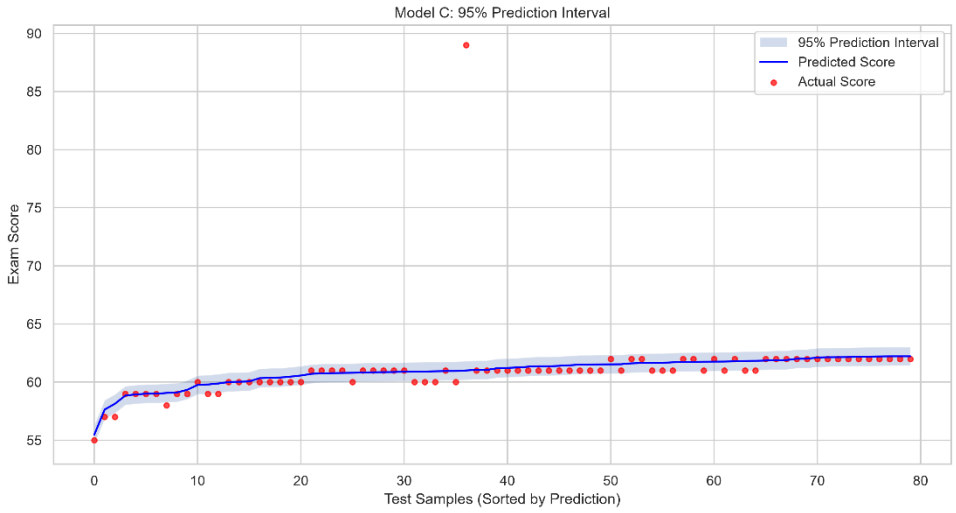


為進一步了解模型的預測誤差，本研究使用殘差圖分析實際成績與預測成績之間的差異。分析結果顯示，大部分殘差集中在 0 附近，表示模型對多數學生的預測誤差較小。

不過，仍有五筆資料的正殘差約介於 15.17～28.95 分之間，表示模型對這些高分學生存在明顯低估現象。

由於 RMSE 對較大的誤差更加敏感，因此少數極端誤差對整體 RMSE 造成顯著影響。這也說明模型即使具有良好的整體預測指標，仍可能對部分學生產生較大的預測偏差。

(4) 95%預測區間



除了預測單一考試分數外，本研究使用 Split Conformal Prediction 建立 95% 預測區間，以評估模型預測結果的不確定性。

實驗使用 1,057 筆校準資料，取得校準誤差分位數為 0.8044，最終結果如下：

評估項目	實驗結果
預測區間信心水準	95%
測試集涵蓋率（Coverage）	95.08%
平均預測區間寬度	1.6089 分
校準集筆數	1,057

結果顯示，約 95.08% 的測試資料實際成績落在預測區間內，整體涵蓋率接近預先設定的 95% 目標。

然而，部分極端高分學生仍落在預測區間之外，表示此方法提供的是整體邊際涵蓋率，而非對每個學生或每個分數區間皆保證 95% 涵蓋率。

(5) 綜合評估

綜合模型評估結果，Model C 在保留 25 個編碼後特徵的情況下，取得 MAE 0.4157、RMSE 1.5210 及 R^2 0.8251，為本研究三個模型中測試指標數值最好的模型，但與 Model B 的差距極小。

此外，95% 預測區間的整體涵蓋率達到 95.08%，顯示模型能對多數測試資料提供有效的預測範圍。

不過，殘差分析顯示模型對少數極端高分學生仍有明顯低估情況。未來可進一步研究這些紀錄的特性，並比較非線性模型是否能改善高分區間的預測效果。

整體而言，本研究完成多元線性回歸的預測能力評估，並透過特徵選擇、殘差分析與預測區間，呈現模型的優點與限制。

5. NotebookLM 摘要

網路主流解法與先進技術研究摘要

針對學生學業表現預測以及 Kaggle《Student Performance Factors》資料集，網路競賽與最新文獻展現了超越傳統基礎樹模型的先進解法：

- **CatBoost 與原生類別特徵處理（Native Categorical Encoding）：**由於教育資料集包含大量類別型特徵（如家長參與度、學校類型等），CatBoost 的原生處理機制表現最為突出。近年針對同一 Kaggle 資料集的研究（如 Jurnal FASILKOM, 2026）驗證，CatBoost 能有效避免傳統序數或單熱編碼導致的資訊損失，將預測決定係數 R^2 大幅提升至 0.851（RMSE 降低至 1.414），顯著優於 LightGBM（$R^2 = 0.809$）與傳統隨機森林。
- **多模型異質堆疊集成（Stacked Ensemble / Blending）：**高績效競賽解法普遍採用模型堆疊技術，結合 CatBoost、LightGBM、XGBoost 與表格神經網路（Tabular NN），並以邏輯回歸或 Ridge 作為 Meta-Learner，可有效消除單一模型在極端高分或低分區間的預測偏差。
- **自動化超參數調優與特徵工程（Optuna & Feature Engineering）：**搭配 Optuna 進行貝葉斯超參數搜尋，並建構特徵交互作用項（如學習時數與出席率之乘積），再結合 SHAP 可解釋 AI，能同時達成極高的預測精確度與教育決策所需的透明度。

6. GPT 輔助內容：(下一頁開始)

推薦Kaggle資料集

10/16/2026 16:30:33
https://chatgpt.com/c/6ad1abaf-d5f8-83ee-9e74-e93096c5ae20

一、作業目標：\ 本次作業延伸自 HW1，目標是讓同學能夠以實際資料集進...

我先幫你挑幾個符合「10～20 個特徵、適合多元線性回歸、能做特徵選擇與預測...

我想選學生成績預測，請繼續幫我評估這份資料集的優缺點與分析方向。

我會進一步確認這份 Kaggle 資料的欄位、資料品質與已知限制，特別留意是否有...

我選進階平衡型，請幫我規劃學生學習成績預測的 HW2 完整研究架構與分析流程...

我會把規劃整理成可以直接當作 HW2 報告大綱的版本，包含研究問題、資料處理...

好 請一步一步教我

Thinking ● 已搜尋 6 個網站 ✓ 處理時間為 24s 好！我們就按照前面規劃好的 ...

我不要用colab

沒問題！那我們就改成 在自己電腦上使用 VS Code + Python + Jupyter...

我使用Windows，VS Code 和 Python 的安裝情況是：都已安裝。請從適合我的...

好！既然你使用的是 Windows，而且已經安裝 VS Code 和 Python，我們就直...

我已經弄好環境了 請開始教我

很好！既然你已經完成 Windows + VS Code + Python 的環境設定，我們就正...

我完成了 HW2 Data Understanding 第一部分，請繼續教我解讀資料統計結果...

很好！接下來我們正式進入 Data Understanding 的第二部分：探索式資料分析...

[Image: image(20261016-050653).png] [Image: image(20261016-...

很好！你這三張 EDA 圖表都成功了，而且已經可以從中發現一些值得討論的現象...

[Image: image(20261016-051041).png] 最低成績： 55\ 最高成績： 101\ ...

很好！現在我們已經掌握這份資料集的主要品質問題，可以正式從 Data...

[Image: image(20261016-051413).png] [Image: image(20261016-...

很好！你提供的三張執行結果都正確，代表我們已經成功完成 資料清理、特徵與...

數值特徵： ['Hours_Studied', 'Attendance', 'Sleep_Hours',...

很好！你的前處理結果完全符合預期，代表 Data Preparation 已經成功完成，現...

[Image: image(20261016-051954).png] === Model A Evaluation ===...

很好！你的 Model A（Simple Linear Regression）已經成功完成，而且這張預...

[Image: image(20261016-052203).png] === Model B Evaluation ===...

Model B 的改善很明顯；我會先幫你解讀指標與圖中少數高分離群點，再接著教...

[Image: image(20261016-052435).png] [Image: image(20261016-...

很好！你已經成功完成 Model C 的 RFE 特徵選擇與 5-Fold Cross-Validation，...

[Image: image(20261016-052800).png] [Image: image(20261016-...

Thinking ● 規劃預測區間 我會繼續處理殘差與區間預測，避免直接使用訓練內殘...

[Image: image(20261016-053217).png] [Image: image(20261016-...

你已經完成 HW2 非常重要的 Evaluation（模型評估） 階段了！這次的結果有一...

Model C 預測成績：66.01 區間模型預測成績：66.06 95% 預測區間：[65.25,...

Thinking ● 已搜尋 2 個網站 ✓ 處理時間為 26s 很好！你的 Deployment（模...

研究目的與資料集特性：本研究旨在建構透明的可解釋機器學習流程[1]。採...

Thinking ● 正在搜尋網路 ✓ 處理時間為 21s 很好！你這份 NotebookLM 摘要...

呈現出Kaggle名次(若有)/預測結果評估(預測圖、評估指標)（10%） 還有這個沒...

你提醒得對，這項還需要補上。我先確認我們使用的 Kaggle 頁面是一般 Datase...

那算了 你開始教我撰寫報告

好！我們先不處理 Kaggle 上傳，直接開始撰寫 HW2 正式報告。你目前已經完成...

太麻煩了 簡化

可以，我們把報告簡化成 8～10 頁左右，不追求篇幅，只求符合老師的評分標準...

CRISP-DM有哪些步驟

CRISP-DM（Cross-Industry Standard Process for Data Mining）是資料探勘...

1. 資料集來源與研究脈絡 2. CRISP-DM 流程完整且邏輯清楚 (1) Business...

可以！我們就按照你列出的 6 個大項來製作 HW2 報告，不需要再增加其他章節...

先給我1. 資料集來源與研究脈絡

1. 資料集來源與研究脈絡 1.1 資料集來源 本研究使用 Kaggle 平台提供的公開資...

1. CRISP-DM 流程完整且邏輯清楚 (1) Business Understanding (2) Data...

以下是根據你實際完成的 HW2 實驗結果整理的 CRISP-DM 六大階段，內容精簡...

3. 結果合理、美觀且具有說服力

以下是第 3 章「結果合理、美觀且具有說服力」的報告內容，根據你實際執行的...

4. 預測結果評估

以下是第 4 章「預測結果評估」的正式報告內容，根據你實際執行的模型數據撰...

5. NotebookLM 摘要：網路上主流或更優解法之比較與說明

以下是第 5 章「NotebookLM 摘要：網路上主流或更優解法之比較與說明」，根...

一、作業目標：\
本次作業延伸自 HW1，目標是讓同學能夠以實際資料集進行「多元線性回歸 (Multiple Linear Regression)」的完整分析，並遵循 CRISP-DM 流程完成從資料理解、建模到評估的全過程。\

二、作業內容：
1\. 資料來源
 至 Kaggle 選擇一個具有 10 至 20 個特徵 (features) 的公開資料集。\
 類型不限（可為房價預測、醫療、車輛效能等主題）。\
 請明確標示資料集來源與連結。
2\. 分析任務\
 使用線性回歸 (Linear Regression) 模型進行預測。\
 可嘗試單純線性回歸、多元線性回歸或 Auto Regression。\
 必須執行** 特徵選擇 (Feature Selection) **與 **模型評估 (Model Evaluation)**。\
 結果部分需包含請提供預測圖(加上信賴區間或預測區間)
3\. CRISP-DM 流程說明\
  Business Understanding\
  Data Understanding\
  Data Preparation\
  Modeling\
  Evaluation\
  Deployment
\
4\. AI協助要求
 所有與 ChatGPT 的對話請以 pdfCrowd 或其他方式須**匯出為 PDF**\
 請**使用 NotebookLM** 對網路上同主題的解法進行研究，並撰寫一份 100 字以上的摘要，放入報告中。\
 請在報告中明確標示「GPT 輔助內容」與「NotebookLM 摘要」
5\. 繳交內容\
 主程式：7114056XXX_hw2.py/.ipynb\
 報告檔： PDF，需包含以下內容：\
  按照 CRISP-DM 說明的分析流程\
  GPT 對話過程（pdfCrowd 匯出）\

NotebookLM 研究摘要\
網路上主流或更優解法之比較與說明\
以上檔案與資料夾**請壓縮為學號命名的一個zip**（例如7114056XXX_hw2.zip）上傳。\
\
(optional) 若上傳至 GitHub，或是以colab撰寫，需提供連結，並在 README.md 中整理流程與成果。\
\
三、評分標準\
** 文件說明（50%）**\
CRISP-DM 流程完整且邏輯清楚（25%）\
包含 GPT 對話與 NotebookLM 摘要（15%）\
有明確說明資料集來源與研究脈絡（10%）\
** 結果呈現（50%）**\
模型正確可執行，具特徵選擇與評估（25%）\
結果合理、美觀且具有說服力（15%）\
呈現出Kaggle名次(若有)/預測結果評估(預測圖、評估指標)（10%）

這是作業的完整規範 你有推薦的kaggle的資料集嗎? 先與我討論

2026/10/16 下午12:46:14

我先幫你挑幾個符合「10～20 個特徵、適合多元線性回歸、能做特徵選擇與預測區間」的 Kaggle 資料集，也會考慮哪個最容易寫出完整的 CRISP-DM 報告。

Thinking

- **已搜尋 10 個網站**

✓ 處理時間為 43s

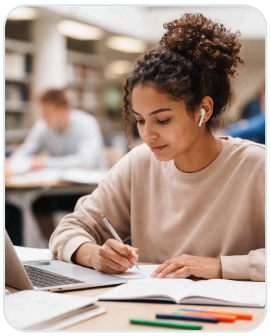
我看完你的 HW2 作業規範後，覺得這次**資料集選得好不好，會直接影響後續程式難度、圖表品質，以及報告能不能拿高分**。

我幫你找了幾個 Kaggle 資料集，篩選時特別考慮：

- **符合規範**：原始資料最好有 10～20 個輸入特徵，而不是只有幾個特徵再勉強湊數。

- **適合 Multiple Linear Regression**：預測目標是連續數值，例如成績、租借數量、價格。
- **容易做 Feature Selection**：不同特徵之間有值得分析的關係，可以比較選擇前後的模型表現。
- **容易呈現漂亮的預測圖**：能畫 Actual vs. Predicted、Residual Plot，以及 95% 預測區間。
- **方便完成 NotebookLM 研究**：Kaggle、GitHub 或學術文獻上已有相關分析，可以比較自己的線性回歸和其他方法。

一、我最推薦的 3 個資料集



1. Student Performance Factors 最推薦

6,607 筆資料｜19 個輸入特徵｜預測考試成績

分析讀書時間、出席率、睡眠、家庭背景等因素，預測學生的最終考試分數。題目好理解，也很適合用迴歸係數解釋哪些因素與成績有關。

Kaggle 資料集 · 現有分析範例

原始資料 20 欄，扣除 Exam_Score 後有 19 個輸入特徵。[GitHub - PStarH/student-performance-factor-analysis: Code for https://www.kaggle.com/datasets/lainguyn123/student-performance-factors; GitHub - katumi10/STUDENT_PERFORMANCE_PREDICTOR: 📊 Predicting student exam scores from study hours & habits using linear regression (Python, Pandas, Scikit-learn).]

---

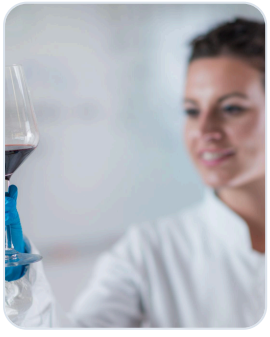
2. Seoul Bike Sharing Demand

8,760 筆資料｜13 個輸入欄位｜預測每小時租借量

使用溫度、濕度、降雨量、時間及季節等資料，預測共享單車需求。很適合展示資料視覺化與實際應用情境。

Kaggle 資料集 · UCI 原始資料說明

共 14 欄，其中一欄為租借數量，計入日期等原始欄位後有 13 個候選輸入欄位。[Seoul Bike Sharing Demand; Seoul Bike Sharing Demand]



3. Red Wine Quality

1,599 筆資料｜11 個輸入特徵｜預測葡萄酒品質分數

分析酸度、酒精濃度、pH 等化學指標，預測葡萄酒的品質評分。特徵全部是數值型態，資料前處理相對簡單，很適合專注在模型與統計分析。

Kaggle 資料集

11 個數值輸入特徵加上一個品質目標欄位；品質分數是離散的整數，仍可作為迴歸練習。[Red Wine Analysis by R]

二、這三個資料集哪個比較適合拿高分？

以下是以這次作業的要求來比較，而不是單純比較資料集的熱門程度。

評估項目	學生成績	首爾共享單車	葡萄酒品質
符合特徵數規定	✓	✓	✓
前處理難度	中	中	低
特徵選擇發揮空間	很高	高	高
線性回歸適合度	高	中	中
視覺化豐富度	高	很高	中
NotebookLM 資料豐富度	高	高	很高
整體推薦	第一名	第二名	第三名

其中共享單車有時間序列特性，因此需要按照時間切分資料，並考慮尖峰時段的非線性關係。葡萄酒則因品質分數為離散整數，模型的 R^2 不一定很高，但這並不代表分析失敗。

三、如果選學生成績資料集，我會怎麼設計 HW2？

我最傾向推薦 **Student Performance Factors**，主要是它能完整展現老師要求的 CRISP-DM，而且 19 個原始輸入特徵讓 Feature Selection 更有意義。

可以把報告主題設定為：

「以多元線性回歸與特徵選擇分析影響學生考試成績的關鍵因素」

整個研究可以這樣安排：

1. Business Understanding 探討哪些學習習慣與環境因素能預測學生成績，以及學校應優先關注哪些可改善的因素。

2. Data Understanding 分析 6,607 筆資料的分布、缺失值、相關係數與可能的離群值，繪製 Heatmap、Scatter Plot。

3. Data Preparation 針對類別特徵進行 One-Hot Encoding、處理缺失值並切分訓練集與測試集，避免資料洩漏。

4. Modeling 建立 Baseline Linear Regression，再透過 SelectKBest 或 RFE 選擇特徵，建立第二個模型進行比較。

5. Evaluation 使用 MAE、RMSE、R^2 評估，製作實際值與預測值比較圖、殘差圖及 95% 預測區間。

6. Deployment 儲存模型及前處理流程，提供輸入學生特徵後預測考試成績的簡單介面或函式，說明應用限制。

這裡我還會特別建議你做一個**特徵選擇前後的模型比較表**，而不是只有訓練出單一模型。

例如：

模型	特徵數	MAE	RMSE	R^2
Model A：全部特徵	19 個原始特徵	待實測	待實測	待實測
Model B：特徵選擇後	依選擇結果	待實測	待實測	待實測

類別變數經編碼後，實際進入模型的欄位數可能超過 19，報告應區分原始特徵數與編碼後特徵數。

此外，95% 預測區間建議使用 `statsmodels` 或適當的區間估計方法，並清楚區分「平均預測的信賴區間」與「單筆成績的預測區間」，這會比只畫一條回歸線更完整。

NotebookLM 部分也有現成的相關研究、GitHub 分析及論文可供你自行匯入比較，像是學生表現的可解釋機器學習研究。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

四、我想先跟你討論一個選擇

你比較偏向哪種作業方向？

- ○ **學生成績預測** — 主題好理解、解釋性強、適合完整報告
- ○ **共享單車需求預測** — 圖表豐富、實務應用感較強
- ○ **葡萄酒品質預測** — 資料乾淨、程式較簡單、專注統計分析
- ○ **想看其他類型** — 例如房價、汽車價格或健康相關數據

繼續討論這個方向 →

我個人目前會選學生成績，因為它的**特徵數剛好符合要求、資料量適中、能深入討論特徵選擇與模型解釋**。而且不需要刻意追求極高的 R^2，也能透過完整的分析與模型比較做出有說服力的報告。

2026/10/16 下午12:46:17 · gpt-6-thinking

我想選學生成績預測，請繼續幫我評估這份資料集的優缺點與分析方向。

2026/10/16 下午12:47:34

我會進一步確認這份 Kaggle 資料的欄位、資料品質與已知限制，特別留意是否有可能讓線性回歸結果「看起來很好，但研究價值不高」的問題。

Thinking

- **已搜尋 9 個網站**

✓ 處理時間為 37s

我查了這份資料集的欄位、相關研究與現有的 GitHub 實作後，認為 **Student Performance Factors 很適合你的 HW2，而且有機會做出比一般線性回歸作業更完整的分析**。

不過我發現一個很重要的地方：**這份資料集是合成資料（Synthetic Data），不是真實學校蒐集的學生紀錄。** 已有 2025 年的學術研究明確說明這點。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

這不違反你的作業規定，因為老師只要求 Kaggle 公開資料集，沒有規定必須是真實觀測資料。但報告一定要清楚註明，不能把模型結果直接當作真實教育現場的因果結論。

一、資料集基本資訊

Student Performance Factors

資料發布者：lainguyn123（Lai Nguyen）

查看 Kaggle 資料集

6,607 學生資料筆數	**19** 原始輸入特徵
6 數值輸入特徵	**13** 類別輸入特徵

預測目標是 `Exam_Score`（考試成績），資料一共 20 欄，扣除目標欄位後正好有 19 個特徵，符合老師的要求。[GitHub]

二、這份資料集的優點與缺點

優點 1：特徵數剛好符合規範

不必人工增加無意義的變數來湊到 10～20 個特徵，也不需要在建模前刪除大量欄位。

優點 2：很適合展示 Feature Selection

包含學習時間、出席率、家庭收入、教師品質、運動時間、睡眠等因素，可以比較哪些特徵真正有助於預測，以及刪除低貢獻特徵後模型是否更簡潔。

優點 3：有相關論文與開源專案可以研究

已有研究比較 Random Forest、XGBoost、LightGBM 與 SHAP，很適合納入 NotebookLM，完成老師要求的主流或進階解法比較。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

優點 4：有足夠資料進行模型評估

6,607 筆樣本可供訓練、交叉驗證及獨立測試，也能分析不同成績區間的預測誤差。

缺點 1：合成資料限制研究結論

可以證明模型在這份資料集上的表現，但不能直接推論「真實學生多讀一小時就一定提高幾分」。

缺點 2：類別變數需要額外處理

13 個類別特徵不能直接全部丟進一般線性回歸，需要適當編碼，例如 One-Hot Encoding。這會稍微增加程式難度。

缺點 3：有缺失值與不合理分數

已有資料檢查記錄指出 `Teacher_Quality`、`Parental_Education_Level`、`Distance_from_Home` 有少量缺失值，另有研究指出存在超過 100 分的成績紀錄。這些問題需要在 Data Preparation 說明處理方式。[michaelozon/student-performance-factors-analysis-michael-ozon; Machine learning-based academic performance prediction with explainability for enhanced decision-making in educational institutions]

缺點 4：線性模型不一定能解釋全部變異

現有一個以 6 個數值特徵建模的專案，回報多元線性回歸的測試 R^2 約為 0.57。這是其他人的實驗結果，不代表我們最後會得到相同數字。[GitHub - katumi10/STUDENT_PERFORMANCE_PREDICTOR: 📊 Predicting student exam scores from study hours & habits using linear regression (Python, Pandas, Scikit-learn).]

三、我建議的研究方向

比起只做「利用線性回歸預測學生成績」，我更推薦讓研究具備一個明確的比較問題：

「特徵選擇是否能在維持預測準確度的同時，降低學生成績預測模型的複雜度？」

這樣非常符合老師要求的 Feature Selection 與 Model Evaluation，也能讓報告更有研究價值。

1. 設計三個模型進行比較

Model A

Simple Linear Regression（基準模型）

只使用 `Hours_Studied`（每週讀書時間）預測成績，建立最基本的參考標準。

Model B

Multiple Linear Regression（完整模型）

使用 19 個原始輸入特徵，經過必要的編碼處理後進行建模。

Model C

Multiple Linear Regression + RFE（特徵選擇模型）

使用 Recursive Feature Elimination 進行特徵選擇，透過交叉驗證決定保留數量，比較能否以較少特徵達到接近或更好的準確度。

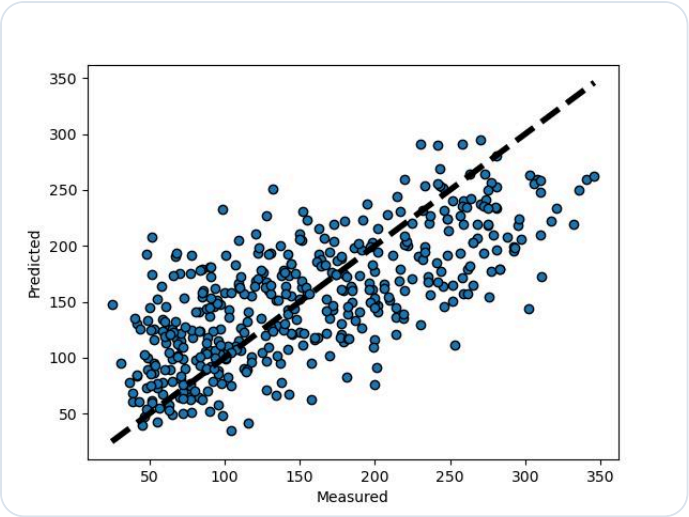
這樣的研究就能回答三個問題：

- 單一學習因素是否足以預測成績？
- 加入家庭、學校、生活習慣等特徵後，能改善多少預測效果？
- 是否能用更少的特徵，保留大部分預測能力？

需要注意，RFE 會對編碼後的欄位進行選擇，因此報告要交代被選中的原始特徵與編碼欄位之間的關係。

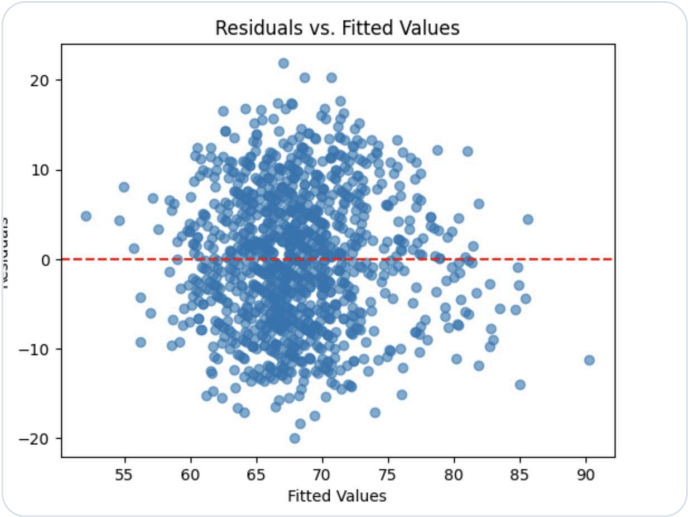
2. 預測圖要怎麼呈現？

這部分佔老師評分的重要比例，我會建議至少做以下四種圖表：



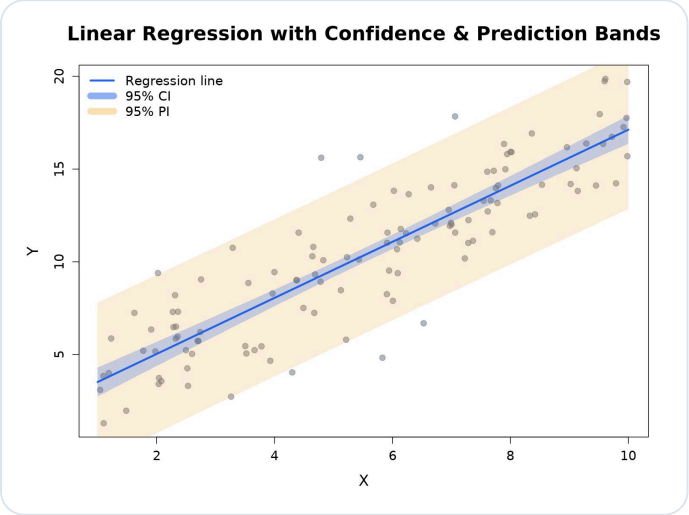
Actual vs. Predicted

比較實際分數與預測分數，觀察模型是否接近理想對角線。



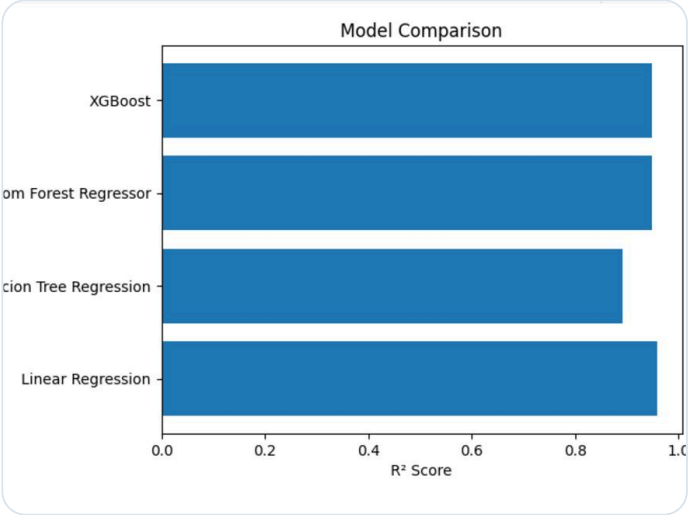
Residual Plot

檢查誤差是否有系統性偏差、異質變異或非線性模式。



95% Prediction Interval

呈現成績預測的不確定性，並檢查測試資料的區間涵蓋率。



Model Comparison

比較三個模型的 MAE、RMSE、R^2 及特徵數量。

以上是預計製作的圖表類型示意，並非這份資料集實際執行後的結果。

我會把**95% 預測區間**列為必要圖表，因為老師特別要求信賴區間或預測區間。三個模型使用相同的測試集；特徵選擇及參數決定只在訓練資料內完成，以免測試結果過度樂觀。

四、NotebookLM 可以研究什麼？

我找到一篇與這份資料集直接相關的論文：

Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study

閱讀論文全文

這篇研究使用相同的 Kaggle 資料集，並比較 Random Forest、XGBoost 和 LightGBM，再使用 SHAP 解釋模型。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

NotebookLM 的研究摘要可以集中在：

1. 為什麼其他研究選擇樹模型，而非只使用線性回歸？
2. 線性回歸與 Random Forest、XGBoost 的優缺點有什麼差異？
3. 哪些因素在現有研究中較重要？與我們實驗的結果是否一致？
4. 為什麼模型可解釋性對教育預測特別重要？

你可以將這篇論文與 Kaggle、GitHub 的公開分析匯入 NotebookLM，整理出超過 100 字的摘要，並保留 NotebookLM 實際產生的內容與來源依據。

五、整體方案評估

我的適合度評估 **9 / 10**

這是依作業要求做的主觀適合度判斷，不是預估作業分數。

我會選這份資料集的原因： 可控的程式難度、足夠的特徵選擇空間、明確的評估方式，以及可以直接與現有論文比較。唯一比較大的研究限制是它屬於合成資料。

六、接下來有一個關鍵決定

我建議先決定分析要做到什麼程度，再開始寫程式，以免後面又要調整整個報告架構。

- ○ **基礎完整型**兩個模型、基本特徵選擇、MAE/RMSE/R^2、預測區間。工作量較少，完整符合主要規範。
- ○ **進階平衡型（我最推薦）** 三個線性回歸模型、RFE、交叉驗證、完整評估圖表，以及和相關論文比較。兼顧完成度與難度。
- ○ **研究比較型**在進階平衡型之外，再加入 Random Forest 或 XGBoost 實驗，實際驗證線性與非線性模型的差異。工作量較高。

確定研究方向 →

我會推薦中間的「進階平衡型」，因為這份作業有 50% 是結果呈現，使用三個模型進行有系統的比較，通常比單純增加大量模型更容易把研究動機、特徵選擇與結果解釋講清楚。

2026/10/16 下午12:47:38 · gpt-6-thinking

我選進階平衡型，請幫我規劃學生學習成績預測的 HW2 完整研究架構與分析流程，先不要寫程式。

2026/10/16 下午12:49:05

我會把規劃整理成可以直接當作 HW2 報告大綱的版本，包含研究問題、資料處理、三個模型的公平比較、特徵選擇、95% 預測區間，以及最後的繳交結構。也會確認資料集的幾個關鍵欄位，避免規劃建立在錯誤假設上。

Thinking

- **已搜尋 4 個網站**

✓ 處理時間為 1m 8s

HW2 完整研究規劃：學生成績預測

Multiple Linear Regression × Feature Selection × CRISP-DM

我們就採用「進階平衡型」，以三個線性回歸模型進行比較，重點放在**完整的研究流程、特徵選擇的效果，以及模型預測結果的解釋**。

這個方案不會刻意堆疊大量演算法，而是讓每個分析步驟都有明確目的，也能直接對應老師的評分標準。

一、確定研究主題與核心問題

建議作業標題

基於多元線性回歸與特徵選擇之學生成績預測研究

Student Exam Score Prediction Using Multiple Linear Regression and Feature Selection

核心研究問題

在學生學習習慣、學校環境及家庭背景等因素中，哪些特徵最有助於預測考試成績？透過特徵選擇，能否降低模型複雜度，同時維持良好的預測能力？

我建議設定三個研究目標：

1. **預測能力**：比較單一特徵與多個特徵的線性回歸模型，確認增加特徵是否改善預測。

2. **特徵重要性**：找出與考試成績預測關係較大的因素，分析哪些變數可以刪除。
3. **模型簡化**：驗證透過 RFE 特徵選擇，是否能在減少特徵的情況下維持接近完整模型的準確度。

可以進一步提出三個研究假設：

假設	預計驗證內容
H1	多元線性回歸的預測表現優於單純線性回歸
H2	讀書時間、出席率、過去成績是重要的預測特徵
H3	特徵選擇能減少模型複雜度，且不會明顯降低預測表現

這些都是**待驗證的假設**，不能在分析前就當成結論。

二、資料集與研究範圍

資料集： Student Performance Factors

Kaggle 連結： Student Performance Factors

6,607	19	Exam_Score
資料筆數	原始輸入特徵	預測目標

這份資料共有 6 個數值型輸入特徵及 13 個類別型輸入特徵。研究論文也確認資料屬於合成資料，並非真實學生個資。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study; GitHub - abhishek7777-hub/student-performance-eda: Exploratory Data Analysis of 6,607 student records to identify key drivers of exam performance using Python, Pandas, Matplotlib, and Seaborn.]

資料欄位分類

特徵類型	欄位名稱
學習表現與習慣	`Hours_Studied`, `Attendance`, `Previous_Scores`, `Tutoring_Sessions`
生活習慣	`Sleep_Hours`, `Physical_Activity`, `Extracurricular_Activities`
家庭與資源	`Parental_Involvement`, `Parental_Education_Level`, `Family_Income`, `Access_to_Resources`, `Internet_Access`

特徵類型	欄位名稱
學校與環境	`Teacher_Quality`, `School_Type`, `Peer_Influence`, `Distance_from_Home`
個人因素	`Motivation_Level`, `Learning_Disabilities`, `Gender`
預測目標	`Exam_Score`

這個分類後續還能用來分析：究竟是學習行為、家庭資源，還是學校環境的資訊，對預測比較有幫助。

三、整體 CRISP-DM 分析架構

1. Business Understanding

研究背景、目標、假設

↓

2. Data Understanding

資料檢查、描述統計、EDA

↓

3. Data Preparation

缺失值、編碼、訓練測試切分

↓

4. Modeling

Model A
單純線性回歸

Model B
完整多元回歸

Model C
RFE 特徵選擇

↓

5. Evaluation

MAE / RMSE / R^2 / 預測區間

↓

6. Deployment

儲存模型、單筆預測、使用限制

CRISP-DM 實務上是可反覆修正的流程；如果模型評估發現資料問題，可以返回資料準備階段。

四、六大階段的具體工作

Phase 1 — Business Understanding

目標： 清楚定義要解決的問題，以及成功的標準。

研究背景可以從教育資料分析出發：學生的考試成績可能與學習投入、生活型態、家庭及學校資源等因素有關，因此希望使用多元線性回歸進行預測。

這一階段要完成的內容：

- 定義預測目標為 `Exam_Score`。
- 定義研究對象為 Kaggle 合成學生資料。
- 提出 H1、H2、H3 三個研究假設。
- 設定評估指標為 MAE、RMSE、R^2。
- 說明希望同時兼顧預測準確度與模型可解釋性。

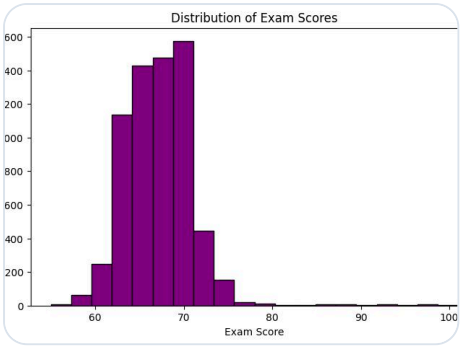
報告產出： 研究動機、問題定義、研究目標、研究假設，以及成功標準。

Phase 2 — Data Understanding

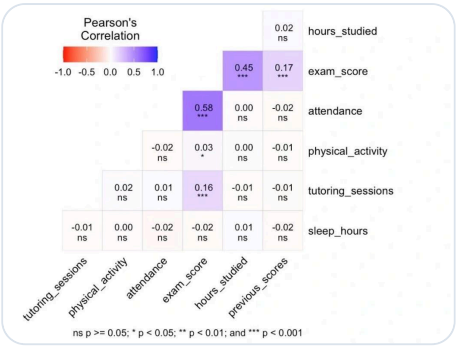
目標： 確認資料適合建模，並找出可能影響分析的問題。

分析項目	實際工作	建議呈現
資料結構	確認筆數、欄數、資料型態	資料概況表
缺失值	統計各欄缺失數量與比例	缺失值長條圖
描述統計	平均數、標準差、最大最小值	統計表
目標變數	觀察 Exam_Score 分布及極端值	Histogram
數值特徵關係	分析相關係數及共線性	Correlation Heatmap
類別變數關係	比較不同類別的成績分布	Box Plot

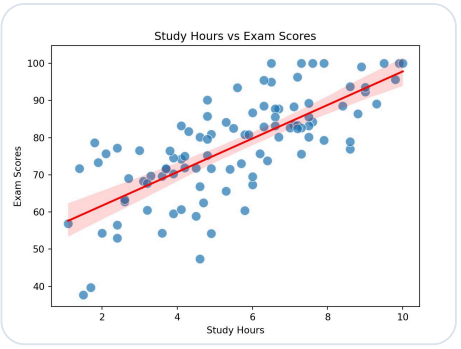
建議這一階段至少選出三張重點圖：



考試成績分布



數值特徵相關矩陣



讀書時間與成績關係

圖表為規劃示意，實際結果將以執行資料分析後產生的圖為準。

特別要注意：如果觀察到成績超過 100 分，或有極端值，不能未經檢查就全部刪除，應先判斷其分布與影響，再說明保留或排除的理由。

Phase 3 — Data Preparation

目標： 建立不會發生資料洩漏、可以公平比較模型的資料處理流程。

我建議採用以下固定規格：

項目	建議處理
訓練集 / 測試集	80% / 20%
隨機種子	`random_state=42`
數值缺失值	使用訓練集的中位數填補
類別缺失值	使用訓練集的眾數填補
類別型特徵	One-Hot Encoding
數值標準化	StandardScaler，放在 Pipeline 中
重複資料	檢查完全重複紀錄與處理必要性
目標極端值	分析後決定處理方式，保留紀錄與理由

其中標準化不是一般線性回歸的絕對必要條件，但有助於之後以係數幅度比較數值特徵。類別編碼後的係數仍需謹慎解讀。

最重要的設計原則：先切分資料，再根據訓練資料擬合所有前處理步驟。

缺失值填補、標準化、特徵選擇都不能事先利用測試集資訊，否則評估會失去可信度。

Phase 4 — Modeling

這是整份作業最重要的技術部分。我們固定比較三個模型：

模型	使用特徵	目的
A：Simple LR	`Hours_Studied`	建立單一特徵基準
B：Multiple LR	全部 19 個原始特徵	測試完整資訊的預測表現
C：Multiple LR + RFE	選擇部分特徵	評估模型簡化的效果

Feature Selection 如何設計？

我們選用 **Recursive Feature Elimination（RFE）**，而不是只看相關係數高低就手動刪除欄位。

RFE 的概念是：先使用全部特徵建立模型，再反覆移除貢獻較低的特徵，直到剩下指定數量。

實際實驗則搭配 5-Fold Cross-Validation，在訓練資料內比較不同保留數量，選擇表現良好的特徵組合。類別欄位先編碼，因此要另外整理原始特徵與編碼後欄位的對應關係。

我們不預先決定保留 5、8 或 10 個特徵，也不預先假定 Model C 一定比 Model B 好。由交叉驗證結果決定。

Phase 5 — Evaluation

為避免不同模型比較不公平，三個模型必須使用完全相同的測試資料。

核心評估指標

指標	意義	判斷方向
MAE	平均預測誤差的絕對值	越低越好
RMSE	對大幅預測錯誤更敏感	越低越好
R^2	模型解釋測試資料變異的能力	越高越好

最終報告會有一張類似以下的比較表：

指標	Model A	Model B	Model C
MAE	待實測	待實測	待實測

指標	Model A	Model B	Model C
RMSE	待實測	待實測	待實測
R^2	待實測	待實測	待實測
使用特徵數	1	19	待選擇
模型複雜度	低	較高	待分析

這裡使用特徵數以原始變數計算；若 RFE 選取的是編碼後欄位，則 Model C 需要額外呈現實際使用的編碼欄位數。

預測區間設計

老師要求預測圖加上信賴區間或預測區間，因此我們會在測試資料上呈現：

- 實際成績與預測成績。
- 95% Prediction Interval 的上下界。
- 區間涵蓋率（有多少實際成績落入區間）。
- 區間平均寬度，用來評估預測的不確定性。

建議使用模型殘差與適當的統計方法估計預測區間，並交代所依據的假設。這樣除了滿足作業要求，也能討論區間是否過寬或過窄。

Phase 6 — Deployment

這次不用建立真正上線的網站。比較合理的做法是完成**離線可重現的預測流程**。

預期成果包括：

- 儲存最佳模型與其前處理 Pipeline。
- 示範輸入一名學生的特徵資料，輸出預測成績。
- 保留執行說明與模型評估圖表。
- 說明模型限制，特別是合成資料無法直接推廣至真實教育決策。

如此即可讓 Deployment 不只是寫一段概念，而是有可展示的成果。

五、最終預計製作的圖表

我建議把圖表控制在 **8 張左右**。圖表不是越多越好，而是每張都要能回答一個研究問題。

編號	圖表名稱	分析目的	CRISP-DM 階段
Fig. 1	Exam Score Distribution	觀察成績分布	Data Understanding
Fig. 2	Correlation Heatmap	數值特徵間的相關性	Data Understanding
Fig. 3	Study Hours vs. Exam Score	觀察單一特徵關係	Data Understanding
Fig. 4	RFE Feature Selection Results	呈現不同特徵數的交叉驗證表現	Modeling
Fig. 5	Selected Feature Coefficients	解釋模型保留的特徵	Modeling
Fig. 6	Actual vs. Predicted	評估預測是否準確	Evaluation
Fig. 7	Residual Plot	檢查殘差分布及系統性偏差	Evaluation
Fig. 8	95% Prediction Interval	呈現預測的不確定性	Evaluation

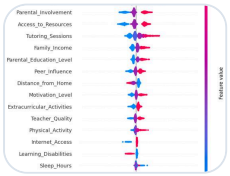
預測區間的部分，我會建議採用 `statsmodels` 的 OLS 預測區間來處理最終模型，前提是清楚檢查並說明線性模型與誤差分布假設。如果這些假設不適合，報告也必須如實交代區間的限制。

另外，可以把三個模型的評估指標做成一張精簡比較表，不必每一個數字都另外畫圖。

六、NotebookLM 研究安排

NotebookLM 不是附加一段文字就好。老師還要求比較「網路主流或更優解法」，所以我建議將它安排成報告中的獨立研究章節。

建議匯入的資料來源



1. 學術論文：Exploring Predictive Insights on Student Success Using Explainable Machine Learning

Information, 2025

使用 Random Forest、XGBoost、LightGBM 與 SHAP 分析相同的 Kaggle 合成資料。研究中的 LightGBM 達到 $R^2 = 0.729$。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

查看論文

2. GitHub：PaperEducation

上述論文作者公開的程式碼，可用來了解其模型訓練及比較方法。

查看 GitHub 專案

3. GitHub：Student Performance Factor Analysis

包含資料清理、探索式資料分析、相關係數及迴歸分析，可以作為我們規劃 EDA 的參考。[GitHub - PStarH/student-performance-factor-analysis: Code for https://www.kaggle.com/datasets/lainguyn123/student-performance-factors]

查看 GitHub 專案

建議詢問 NotebookLM 的研究問題

可以直接複製到 NotebookLM 的研究提問

請根據我匯入的來源，分析 Student Performance Factors 資料集常用的成績預測方法，比較 Multiple Linear Regression、Random Forest、XGBoost 與 LightGBM 的優缺點，說明特徵選擇與模型可解釋性的重要性。請特別指出哪些研究使用相同資料集、它們的評估方式是否一致，以及哪些研究結論受合成資料限制。請提供引用依據。

複製提問

NotebookLM 摘要最後建議寫 **200～300 字**，超過老師的 100 字要求即可。標示為「NotebookLM 摘要」，並註明資料來源。

報告也應明確區分外部研究的數據和自己實驗的數據。即使使用相同資料集，不同切分方式、特徵處理及測試條件，也可能讓 R^2 不具直接可比性。

七、正式 PDF 報告建議目錄

這個架構可以直接用作最終報告的章節安排。

章節	內容	建議篇幅
摘要	研究目的、方法、主要結果	0.5 頁
1. Introduction	研究動機、背景、研究問題	1 頁
2. Business Understanding	問題定義、研究假設	1 頁
3. Data Understanding	資料來源、欄位、EDA	2～3 頁
4. Data Preparation	缺失值處理、編碼、資料切分	1～2 頁
5. Modeling	三個模型、RFE、交叉驗證	2～3 頁

章節	內容	建議篇幅
6. Evaluation	指標、圖表、預測區間、結果討論	3～4 頁
7. Deployment	模型儲存、預測示範、限制	1 頁
8. NotebookLM 摘要	網路研究與方法比較	1～2 頁
9. GPT 輔助內容	GPT 使用範圍與協作說明	0.5～1 頁
10. Conclusion	假設驗證、發現與限制	1 頁
Appendix	GPT 對話紀錄、補充表格	視需要

主報告大約 **14～20 頁**（不含附錄）是合理規模，不用強迫湊頁數。老師要求的 GPT 對話紀錄可以完整匯出 PDF，作為附錄或與主報告合併。

八、建議的程式與繳交檔案結構

雖然現在先不寫程式，但可以先把最終產出規劃好。

7114056XXX_hw2.zip

`7114056XXX_hw2.ipynb`

`README.md`

`data/`

`StudentPerformanceFactors.csv`

`figures/`

`01_score_distribution.png`

`02_correlation_heatmap.png`

`...`

`08_prediction_interval.png`

`models/`

`best_model.joblib`

`report/`

`HW2_Report.pdf`

`GPT_Conversation.pdf`

`NotebookLM_Research.pdf`（選用）

我建議主程式使用 `.ipynb` 而不是 `.py`，因為 Notebook 可以將程式、說明文字、執行輸出與圖表放在一起，老師比較容易檢查整個分析過程；也可以在 Google Colab 執行。

九、建議的實作順序

HW2 工作檢查清單 0/10 已完成

- [ ] 下載 Kaggle CSV，確認 19 個特徵與欄位內容
- [ ] 完成資料結構檢查、缺失值與 EDA 圖表
- [ ] 切分訓練測試集，建立資料前處理 Pipeline
- [ ] 訓練 Model A 與 Model B
- [ ] 建立 RFE 與 5-Fold Cross-Validation，完成 Model C
- [ ] 計算三個模型的 MAE、RMSE、R^2
- [ ] 製作預測圖、殘差圖與 95% 預測區間
- [ ] 保存模型並示範單筆資料預測
- [ ] 使用 NotebookLM 完成文獻研究與比較
- [ ] 撰寫 PDF 報告、匯出 GPT 對話、打包 ZIP

十、開始實作前，我建議固定的規格

- **資料集：** Student Performance Factors，全部 19 個原始輸入特徵納入完整模型候選。
- **建模方式：** Simple LR、Multiple LR、Multiple LR + RFE 三組模型。
- **評估設計：** 固定 80/20 切分，訓練集進行 5-Fold CV，獨立測試集只在最後評估。
- **主要成果：** 特徵選擇結果、三模型比較、預測與殘差圖、95% 預測區間。
- **研究範圍：** 專注線性回歸，不額外實作 Random Forest 或 XGBoost；非線性模型留在 NotebookLM 文獻比較章節。

我的建議

這份作業最值得深入的不是追求最高的 R^2，而是回答一個具體問題：

「使用更多特徵真的比較好嗎？還是透過 RFE 選擇少量重要特徵，就能達到接近完整模型的預測效果？」

這能讓整份作業從資料理解、建模、評估到研究結論都有一條清楚的主線。

下一步最適合先處理 **Data Understanding（資料理解）**：實際下載 Kaggle CSV、檢查 20 個欄位、缺失值、目標分數與變數分布。等確認資料品質後，再決定哪些清理動作真的必要，而不是一開始就預設刪除離群值或缺失資料。

2026/10/16 下午12:49:08 · gpt-6-thinking

好 請一步一步教我

2026/10/16 下午12:51:39

Thinking

- **已搜尋 6 個網站**

✓ 處理時間為 24s

好！我們就按照前面規劃好的 **進階平衡型 HW2**，一步一步完成整份作業。

我會用教學的方式帶你做，不會一開始就丟給你一大份完整程式碼。每個步驟都會說明：

1. **這一步要做什麼**：對應 CRISP-DM 哪個階段。
2. **為什麼要這樣做**：了解背後的資料分析觀念。
3. **如何操作**：提供可以直接執行的 Python 程式碼。
4. **怎麼確認結果**：幫你判斷輸出是否正常。
5. **報告要寫什麼**：同步整理可以放進 PDF 的研究內容。

我們使用 **Google Colab + Python**，不用先設定本機 Python 環境，也方便之後交 `.ipynb`。

整體學習路線

目前階段 **Step 1 — 建立專案與取得資料**

1. 建立 Google Colab Notebook，下載 Kaggle 資料
2. 讀取 CSV，檢查欄位、資料型態及缺失值
3. 執行 EDA，製作統計圖與相關係數圖
4. 完成資料前處理與訓練測試集切分
5. 建立 Model A、Model B、Model C（RFE）
6. 比較模型表現，建立預測區間
7. 完成 Deployment 示範

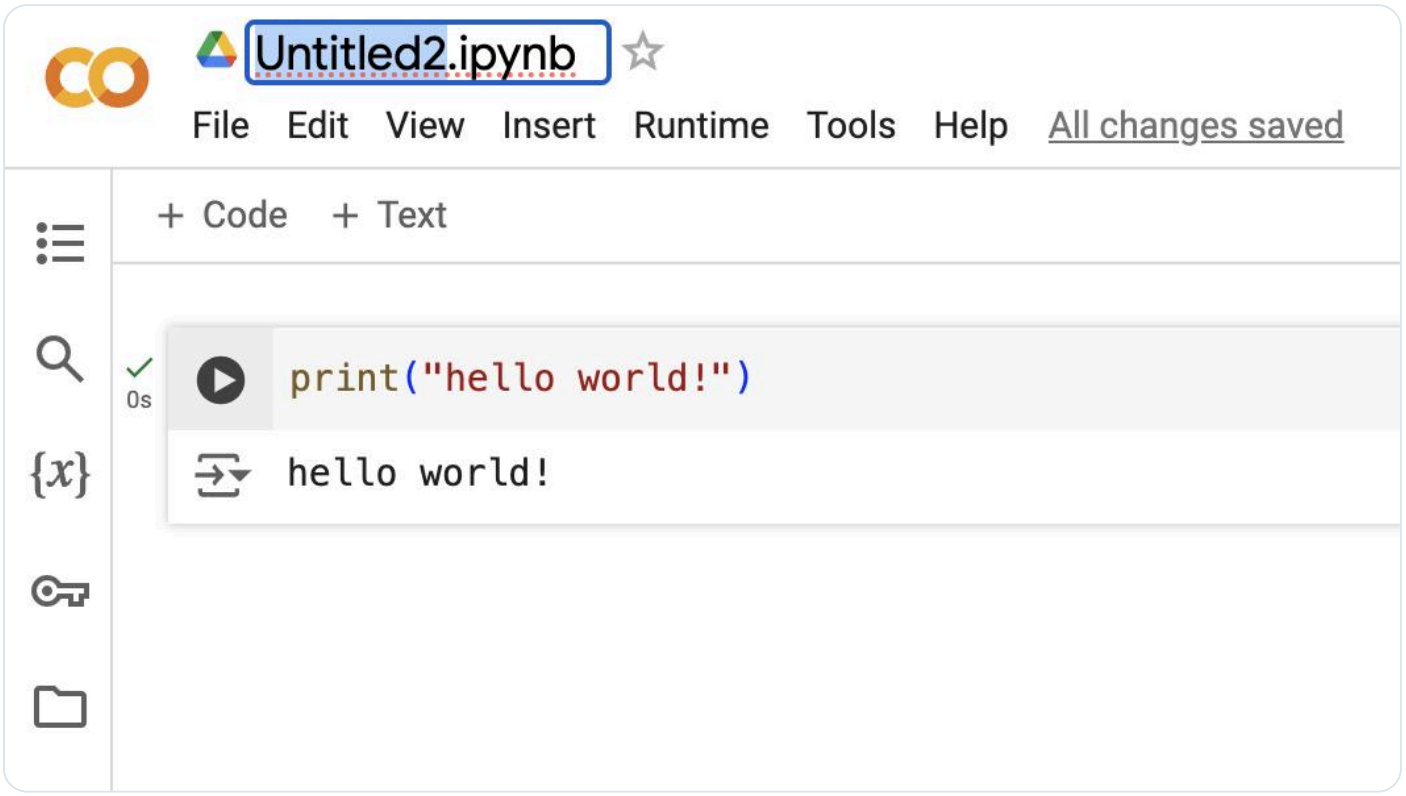
8. 整理 NotebookLM 研究、報告與 ZIP

Step 1：建立 Google Colab 專案

CRISP-DM：專案初始化，為 Data Understanding 做準備

1-1. 建立 Notebook

先開啟 Google Colab。



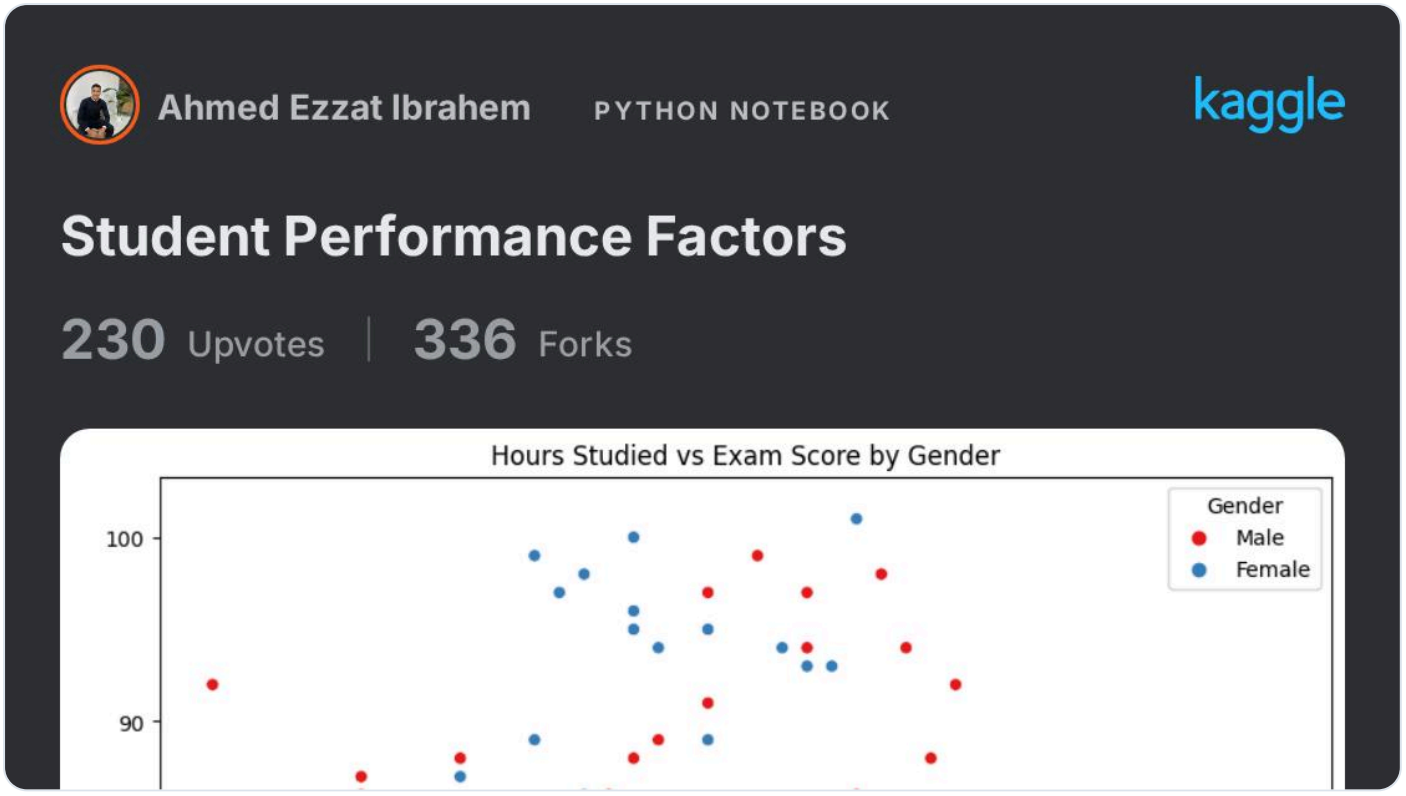
操作順序：

1. 點選「新增筆記本（New notebook）」。
2. 將左上方檔名改成 `7114056XXX_hw2.ipynb`，其中 `XXX` 改成你的完整學號。
3. 確認右上角可以點選「連線」，執行階段使用 Python 3 即可。
4. 建議把 Notebook 存在 Google Drive，避免後續找不到檔案。

1-2. 下載 Kaggle 資料集

開啟：

Student Performance Factors — Kaggle



在 Kaggle 頁面選擇 Download，下載資料集 ZIP 並解壓縮。

你應該會看到：

`StudentPerformanceFactors.csv`

這就是我們要使用的資料檔案，裡面包含學生讀書時間、出席率、家庭背景、睡眠時間，以及最終考試成績。

1-3. 把 CSV 上傳到 Colab

在 Colab 左側點選資料夾圖示 🗀，將 `StudentPerformanceFactors.csv` 上傳到工作環境。

上傳完成後，應該能在左側檔案清單看到：

```
sample_data/
StudentPerformanceFactors.csv
```

注意，直接上傳到 Colab 的檔案會在執行階段重設後消失，但不影響這次教學；最終 ZIP 會另外保留 CSV。

Step 2：第一次讀取資料

這一步先不做清理，也不訓練模型。我們要確認下載的 CSV 可以正常讀取。

2-1. 建立第一個程式碼儲存格

在 Colab 點選 `+ 程式碼`，貼上以下程式：

```
import pandas as pd

df = pd.read_csv("/content/StudentPerformanceFactors.csv")

print("資料集大小：", df.shape)
display(df.head())
```

按左側的 ▷ 執行。

2-2. 確認執行結果

你預期會看到：

預期輸出

(6607, 20)

代表 6,607 列學生紀錄與 20 欄資料。

已公開的相同資料集分析也顯示此資料大小。[Student Performance Factors]

`display(df.head())` 則會顯示前五筆資料，幫助我們觀察每個特徵實際儲存的值。

2-3. 檢查欄位型態與缺失值

新增第二個程式碼儲存格：

```
print("=== 資料基本資訊 ===")
df.info()

print("\n=== 各欄位缺失值數量 ===")
display(df.isnull().sum().to_frame("Missing Count"))

print("\n=== 數值特徵的描述統計 ===")
display(df.describe())
```

執行後，你應該能確認哪些欄位是數值型態、哪些是類別型態，哪些欄位含有缺失值，以及各個數值欄位的最大值、最小值與平均值。

這些資訊將決定下一階段的資料處理方式。

Step 3：先記錄研究資料來源

在 Colab 新增一個 `+ 文字` 儲存格，貼上下面內容，方便之後製作報告。

Dataset Description

本研究使用 Kaggle 公開資料集 **Student Performance Factors**，由 lainguyn123 發布。資料集包含 6,607 筆學生紀錄及 20 個欄位，其中 19 個為輸入特徵，1 個為預測目標 Exam_Score。特徵涵蓋學生的學習時間、出席率、過往成績、睡眠時間、家庭背景與學校環境等面向。

本研究將以 Exam_Score 為應變數，採用單純線性回歸、多元線性回歸及 RFE 特徵選擇方法建立預測模型，並比較不同模型的預測效能。

本資料集屬於合成資料，因此研究結果主要用於驗證預測方法及探索變數關係，不直接推論真實教育環境中的因果效果。

資料來源： https://www.kaggle.com/datasets/lainguyn123/student-performance-factors

---

這一輪先完成這三件事

- [ ] 建立 Colab Notebook 並命名
- [ ] 下載及上傳 StudentPerformanceFactors.csv
- [ ] 成功執行兩段程式碼，看到資料資訊

我完成了，進入下一步 →

完成後，把 `df.info()`、缺失值統計與 `df.describe()` 的執行結果貼給我（也可以直接截圖）。

我會根據你實際的資料結果，帶你進行下一階段的 **EDA 探索式資料分析**，包括如何製作圖表、解讀結果，以及哪些發現值得寫進作業報告。

2026/10/16 下午12:52:00 · gpt-6-thinking

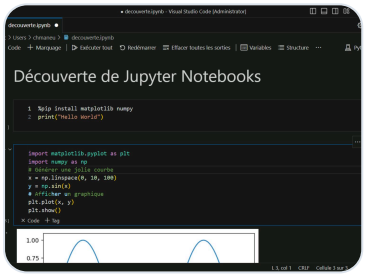
我不要用colab

2026/10/16 下午12:52:52

沒問題！那我們就改成 **在自己電腦上使用 VS Code + Python + Jupyter Notebook** 來完成 HW2，不使用 Google Colab。

我推薦這個方式，因為可以在本機直接管理 CSV、程式碼、圖表和模型檔案，最後打包成 ZIP 也很方便。

一、我們改用的開發環境



VS Code + Jupyter Notebook

- 開發工具：VS Code
- 程式語言：Python
- 程式格式：`.ipynb`
- 資料處理：pandas
- 機器學習：scikit-learn
- 圖表：matplotlib、seaborn
- 統計與預測區間：statsmodels

二、Step 1：準備本機環境

先確認電腦有安裝：

1. Python（建議 Python 3.11 或 3.12）
2. Visual Studio Code
3. VS Code 的 **Python** 和 **Jupyter** 擴充套件

接著建立以下專案資料夾：

7114056XXX_hw2/

`7114056XXX_hw2.ipynb`

`data/`

`StudentPerformanceFactors.csv`

- `figures/`
- `models/`
- `report/`

三、Step 2：安裝 Python 套件

使用 VS Code 開啟專案資料夾，點選「終端機 → 新增終端機」，執行：

```
python -m pip install pandas numpy matplotlib seaborn scikit-learn statsmodels
jupyter ipykernel joblib
```

安裝完成後，開啟 `.ipynb`，在右上角選取你安裝套件的 Python Kernel。

四、Step 3：讀取 Kaggle 資料

從 Student Performance Factors 下載資料集，解壓縮後，把 `StudentPerformanceFactors.csv` 放進 `data` 資料夾。

接著在 Notebook 第一個程式碼儲存格輸入：

```
import pandas as pd

df = pd.read_csv("data/StudentPerformanceFactors.csv")

print("資料集大小：", df.shape)
display(df.head())
```

如果正常，應顯示：

```
資料集大小： (6607, 20)
```

這代表有 6,607 筆學生資料及 20 個欄位。

接下來先確認你的電腦環境

因為 Windows 和 macOS 的 Python 環境設定略有不同，我想先確認一件事，才能繼續一步一步教你操作。

你使用哪個作業系統？

- Windows

- macOS
- Linux

你的電腦已經安裝 VS Code 和 Python 了嗎？

- 兩個都已經安裝
- 只安裝其中一個
- 不確定，想先檢查

確認環境，開始教學 →

我們會先把本機執行環境確定好，再開始正式的 Data Understanding、EDA、特徵選擇與模型訓練。

2026/10/16 下午12:52:54 · gpt-6-thinking

我使用Windows，VS Code 和 Python 的安裝情況是：都已安裝。請從適合我的環境設定開始一步一步教我。

2026/10/16 下午12:59:28

好！既然你使用的是 **Windows，而且已經安裝 VS Code 和 Python**，我們就直接從建立 HW2 專案環境開始。

這次先完成環境設定，不急著寫模型。等你確認可以正常執行 Python，我們再進入資料分析。

Step 1：建立 HW2 專案資料夾

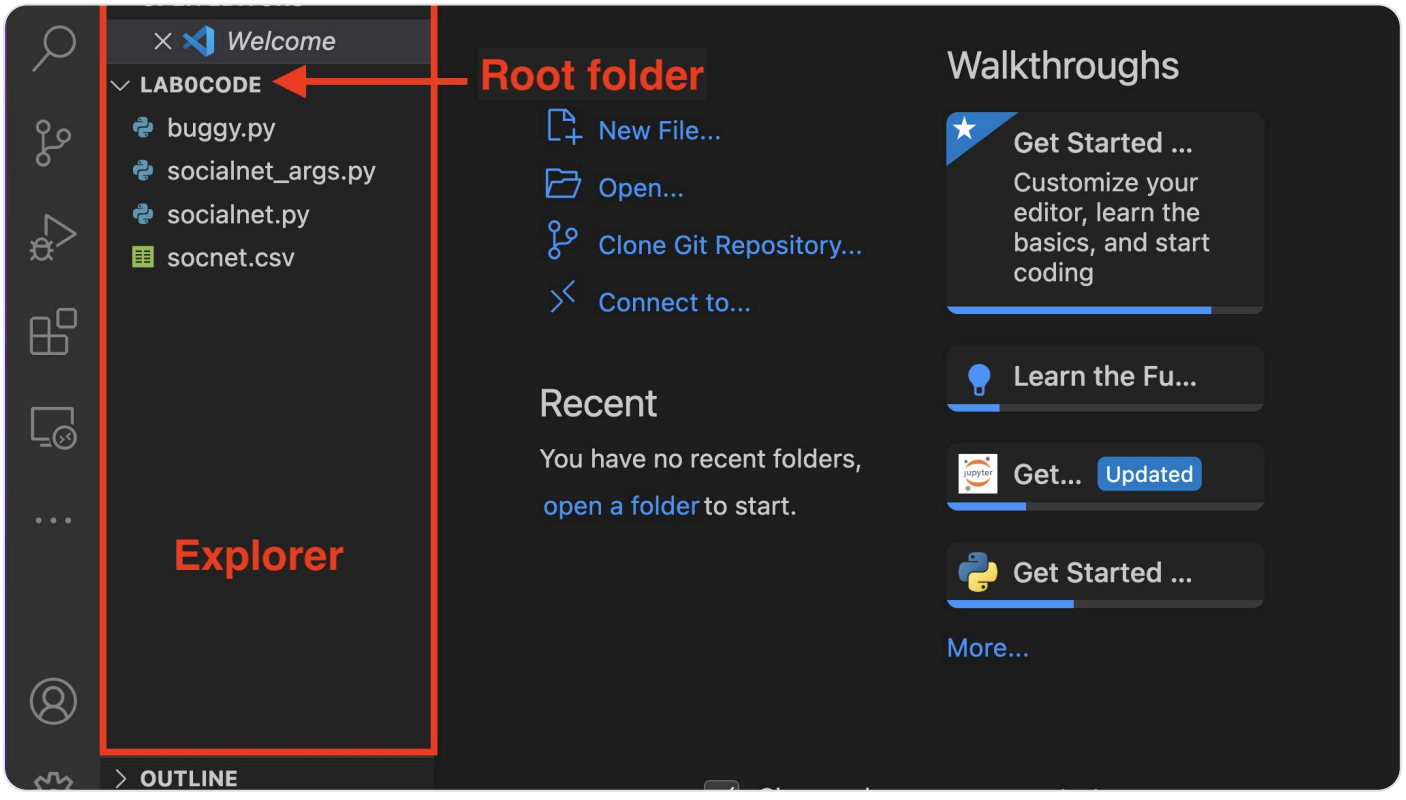
首先，在你習慣存放作業的地方（例如桌面或 D 槽）建立資料夾：

`7114056XXX_hw2`

其中 `7114056XXX` 換成你的完整學號。

接著開啟 VS Code，選擇：

File → Open Folder → 選取剛建立的資料夾



現在 VS Code 左側應該會顯示你的 HW2 資料夾。

Step 2：檢查 Python 是否正常

在 VS Code 上方選擇：

Terminal → New Terminal

在下方終端機輸入：

```
python --version
```

如果成功，應該顯示類似：

```
Python 3.12.7
```

版本不一定要完全相同，Python 3.11 或 3.12 都可以。

Step 3：建立專用 Python 虛擬環境

為了避免這次作業的套件與其他 Python 專案互相影響，我建議建立 `.venv`。

在 VS Code 終端機執行：

```
python -m venv .venv
```

完成後，專案資料夾會多出 `.venv`。

接著我們直接使用虛擬環境中的 Python 安裝套件，不需要另外啟用虛擬環境。

```
.\.venv\Scripts\python.exe -m pip install pandas numpy matplotlib seaborn
scikit-learn statsmodels jupyter ipykernel joblib
```

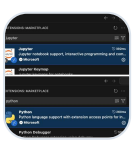
這些套件在後續會分別用於資料處理、統計分析、建立回歸模型、特徵選擇及圖表製作。

Step 4：確認 VS Code 擴充套件

開啟左側 Extensions（快捷鍵 `Ctrl + Shift + X`），確認已安裝以下兩個 Microsoft 擴充套件。

Python

提供 Python 程式執行、語法提示，以及虛擬環境選擇功能。



Jupyter

讓你在 VS Code 中開啟 `.ipynb`，分段執行程式並直接顯示圖表。

Step 5：建立 Notebook 並測試

在 VS Code 左側的專案資料夾按右鍵，選擇 **New File**，新增：

`7114056XXX_hw2.ipynb`

開啟後，點右上角的 **Select Kernel → Python Environments**，選擇剛建立的：

`.venv\Scripts\python.exe`

在第一個程式碼儲存格貼上：

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
import statsmodels

print("Python 環境設定成功！")
print("pandas:", pd.__version__)
print("scikit-learn:", sklearn.__version__)
```

按儲存格左側的 ▷ 執行。

成功的話，你會看到：

```
Python 環境設定成功！
pandas: ...
scikit-learn: ...
```

版本號會依你安裝的版本而不同。

目前的完成檢查

環境設定進度 0/5

- [ ] VS Code 已開啟 HW2 專案資料夾
- [ ] python --version 能顯示版本
- [ ] 已建立 .venv 並安裝套件
- [ ] 已安裝 Python 和 Jupyter 擴充套件
- [ ] Notebook 測試程式成功執行

環境設定完成，進入資料讀取 →

如果任何步驟出現錯誤，直接把 VS Code 終端機或 Notebook 的錯誤訊息貼給我，我會針對那個步驟幫你處理。

這一階段的目標只有一個：確保你的 Windows + VS Code + Python 環境可以正常執行 `.ipynb`。 完成後才開始正式的 CRISP-DM Data Understanding。

2026/10/16 下午12:59:31 · gpt-6-thinking

我已經弄好環境了 請開始教我

2026/10/16 下午1:00:25

很好！既然你已經完成 Windows + VS Code + Python 的環境設定，我們就正式開始 HW2。

這次先做 **CRISP-DM 的第二階段：Data Understanding（資料理解）**。

我們會一步一步進行，每完成一個階段，就把分析結果整理進 Notebook，最後可以直接用來撰寫 PDF 報告。

今天的目標：讀取並理解資料集

HW2 分析進度 Data Understanding

環境設定

目前進入資料分析的第一步

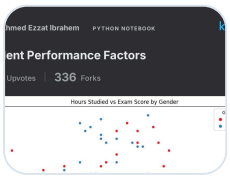
本階段要完成：

1. 下載並放置 Kaggle 資料集。
2. 使用 pandas 讀取 CSV。
3. 確認資料筆數、欄位和資料型態。
4. 統計缺失值。
5. 觀察數值欄位的描述統計。

這一輪先不要處理缺失值，也不要建立模型。 我們需要先掌握資料的實際情況。

Step 1：下載資料集

開啟 Kaggle：



Student Performance Factors

Kaggle 公開資料集

前往下載資料集

進入頁面後：

1. 點選 **Download**，將資料集下載到電腦。
2. 解壓縮 ZIP 檔案。
3. 找到 `StudentPerformanceFactors.csv`。
4. 在你的 HW2 專案中建立 `data` 資料夾。
5. 把 CSV 放進 `data` 裡。

完成後，VS Code 左側的結構應該如下：

7114056XXX_hw2/

`.venv/`

`data/`

`StudentPerformanceFactors.csv`

`7114056XXX_hw2.ipynb`

Step 2：在 Notebook 建立章節

打開你的 `7114056XXX_hw2.ipynb`。

新增一個 **Markdown 儲存格**，貼上：

```
# HW2：基於多元線性回歸與特徵選擇之學生成績預測研究

## 1. Business Understanding

本研究使用 Kaggle Student Performance Factors 資料集，
以 Exam_Score 作為預測目標。

研究目的是比較單純線性回歸、多元線性回歸，
以及使用 RFE 特徵選擇的多元線性回歸，
探討特徵選擇對模型預測能力與複雜度的影響。

## 2. Data Understanding
```

這段就是報告的初步研究說明，後續還會根據分析結果補充。

Step 3：用 pandas 讀取資料

在 Markdown 儲存格下面新增 **Python 程式碼儲存格**：

```
import pandas as pd

df = pd.read_csv("data/StudentPerformanceFactors.csv")

print("資料集大小：", df.shape)
display(df.head())
```

這段程式在做什麼？

程式碼	功能
`import pandas as pd`	載入 pandas，負責表格資料處理
`pd.read_csv(...)`	讀取 CSV 檔案
`df.shape`	查看資料有幾列、幾欄
`df.head()`	顯示資料前 5 筆

執行後，預期會看到：

```
資料集大小： (6607, 20)
```

其中：

- **6,607 列（rows）**：代表 6,607 筆學生紀錄。
- **20 欄（columns）**：包含 19 個輸入特徵與一個預測目標 `Exam_Score`。

這剛好符合老師要求的 10～20 個 features。

Step 4：檢查資料型態

新增下一個程式碼儲存格：

```
df.info()
```

這裡會顯示每個欄位的名稱、非空值數量與資料型態。

你會遇到兩種主要型態：

- `int64`：整數型態，例如讀書時間、考試成績。
- `object`：文字型態，例如性別、家庭收入等類別資料。

為什麼要檢查？

因為線性回歸不能直接使用文字型態的特徵。之後進入 Data Preparation 時，我們需要將類別資料轉換成數值。

目前只需要觀察，不必修改資料。

Step 5：檢查缺失值

新增程式碼儲存格：

```
missing = df.isnull().sum()

display(missing.to_frame("Missing Count"))
```

這段程式會統計各欄位有多少筆缺失值。

例如：

欄位	缺失值數量（示意）
Hours_Studied	0
Attendance	0
Teacher_Quality	78
Parental_Education_Level	90

表中的數字僅供理解輸出格式，請以實際執行結果為準。

我們為什麼要知道缺失值？

因為模型需要完整且可運算的特徵資料。後續會使用適當的方法填補缺失值，但不能直接假設所有缺失資料都應該刪掉。

Step 6：查看描述統計

新增最後一個程式碼儲存格：

```
display(df.describe())
```

它會列出數值欄位的基本統計：

統計量	意義
count	有效資料筆數
mean	平均數
std	標準差
min	最小值
25%	第一四分位數
50%	中位數
75%	第三四分位數
max	最大值

這一步我們尤其要觀察 `Exam_Score` 的最大值、最小值、平均數與標準差，確認成績的分布是否合理。

這一輪的完成檢查

Data Understanding：第一部分 0/5 已完成

- [ ] CSV 已存放在 data 資料夾
- [ ] 成功讀取資料，顯示 (6607, 20)
- [ ] 成功執行 df.info()
- [ ] 成功統計各欄位缺失值
- [ ] 成功執行 df.describe()

完成，繼續下一階段 →

完成後，請把以下三個執行結果貼給我或截圖： `df.info()`、缺失值統計、`df.describe()`。

我會依照你實際得到的數據，教你怎麼解讀，以及如何製作第一張正式的 EDA 圖表（學生成績分布圖）。這樣每張圖都有分析目的，而不只是為了完成作業而畫圖。

我完成了 HW2 Data Understanding 第一部分，請繼續教我解讀資料統計結果及製作 EDA 圖表。

2026/10/16 下午1:02:37

很好！接下來我們正式進入 **Data Understanding 的第二部分：探索式資料分析（Exploratory Data Analysis, EDA）**。

這個階段非常重要，因為老師不只要你建立線性回歸模型，也希望你能解釋資料的特性，以及為什麼選擇某些特徵。

這次我們會完成三張正式圖表，而且每張都會對應一個分析問題。

本次教學目標

CRISP-DM：Data Understanding 第二部分

1. 解讀 `df.describe()` 與缺失值統計
2. Fig. 1：考試成績分布圖（Histogram）
3. Fig. 2：數值特徵相關係數熱力圖（Heatmap）
4. Fig. 3：讀書時間與考試成績散佈圖（Scatter Plot）
5. 整理初步分析發現，放入 Notebook 報告

我們先採用**觀察資料、不修改原始資料**的原則。這一輪不進行缺失值填補、特徵選擇或模型訓練。

Step 1：如何解讀剛才的統計結果？

1-1. 解讀 `df.info()`

你應該會看到 20 個欄位，包含數值與類別資料。

關鍵不是只確認筆數，而是注意：

- `Hours_Studied`、`Attendance`、`Previous_Scores` 等欄位是數值型態，後續可以直接作為迴歸模型的候選特徵。
- `Parental_Involvement`、`School_Type` 等文字欄位是類別型態，後續需要適當編碼。
- 若 Non-Null Count 小於 6,607，代表該欄位有缺失值。

注意： 類別變數不代表沒有預測價值，只是需要先轉換成模型可以處理的表示方式。

1-2. 解讀缺失值

如果缺失值集中在 `Teacher_Quality`、`Parental_Education_Level`、`Distance_from_Home` 等欄位，我們就要在 Data Preparation 說明如何填補。

你可以額外執行：

```
missing_summary = pd.DataFrame({
    "Missing Count": df.isnull().sum(),
    "Missing Rate (%)": df.isnull().mean() * 100
})

display(missing_summary[missing_summary["Missing Count"] > 0])
```

這段程式除了缺失值數量，還會幫你計算缺失比例。

例如某欄有 66 筆缺失，資料總量是 6,607 筆，缺失比例大約是 1%。

1-3. 解讀 `df.describe()`

最重要的是觀察這幾個欄位：

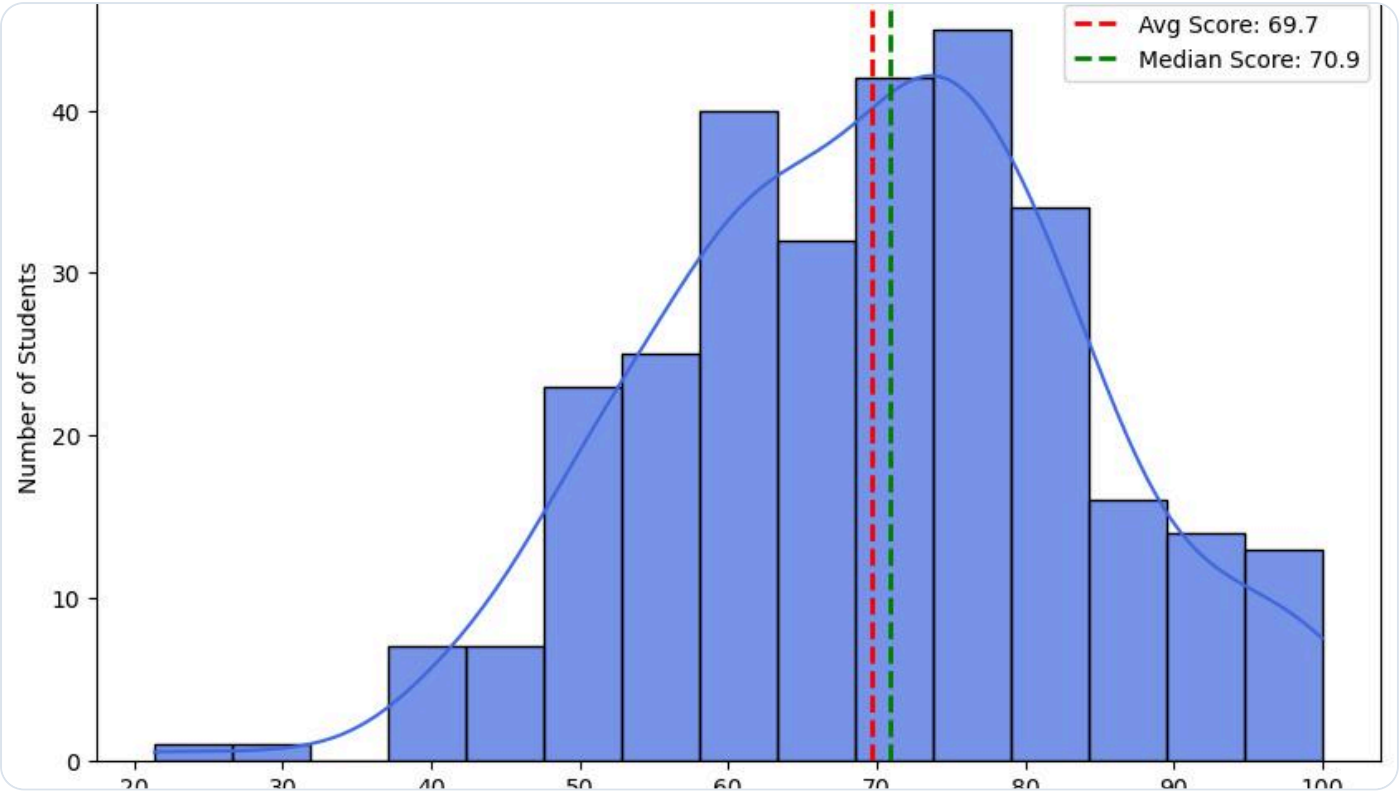
欄位	觀察重點
`Exam_Score`	平均成績、分數範圍、是否有極端分數
`Hours_Studied`	學生讀書時間分布是否集中
`Attendance`	出席率高低與分布範圍
`Previous_Scores`	過去成績的差異
`Sleep_Hours`	睡眠時間有沒有異常值

這裡先不要急著刪除任何資料。即使看到 `Exam_Score` 超過 100，也要先確認其筆數與對分析的影響。

Step 2：Fig. 1 考試成績分布圖

研究問題：學生的考試成績集中在哪個範圍？是否存在極端值或偏態分布？

我們使用 Histogram（直方圖），搭配 KDE（核密度估計曲線）。



直方圖類型示意；不是你的資料實際執行結果。

在 Notebook 新增一個 Markdown 儲存格：

```
### 2.1 Exam Score Distribution

分析目標：觀察考試成績的分布、集中趨勢與可能的極端值。
```

接著新增 Python 儲存格：

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

plt.figure(figsize=(9, 5))

sns.histplot(
    data=df,
    x="Exam_Score",
    bins=30,
    kde=True
)
```

```
plt.axvline(
    df["Exam_Score"].mean(),
    color="red",
    linestyle="--",
    label=f"Mean: {df['Exam_Score'].mean():.2f}"
)

plt.title("Distribution of Exam Scores")
plt.xlabel("Exam Score")
plt.ylabel("Frequency")
plt.legend()
plt.tight_layout()

plt.savefig("figures/01_score_distribution.png", dpi=300)
plt.show()
```

記得先在 VS Code 專案內建立 `figures` 資料夾，否則 `savefig()` 會因為找不到資料夾而出錯。

如何解讀？

執行之後，觀察三件事：

1. **集中趨勢**：大部分分數落在哪個區間？
2. **分布形狀**：是否近似鐘形？是否左偏或右偏？
3. **極端值**：是否有少數學生分數明顯高於其他人？

圖中的紅色虛線是平均成績。

注意：即使成績分布不是常態，也不表示不能使用線性回歸。迴歸模型的假設主要涉及條件平均關係、誤差結構等，而不是要求目標變數本身一定符合常態分布。

Step 3：Fig. 2 數值特徵相關係數熱力圖

研究問題：哪些數值特徵與 Exam_Score 具有較強的線性關係？

這張圖對後續的多元線性回歸及 Feature Selection 很重要。

3-1. 先理解 Correlation（相關係數）

Pearson 相關係數介於 -1 到 +1。

例如：

- `r = +0.8`：兩個變數具有較強的正向線性關係。
- `r = -0.7`：兩個變數具有較強的負向線性關係。
- `r = 0.05`：兩個變數幾乎沒有線性相關，但不代表不存在非線性關係。

3-2. 建立 Heatmap

新增 Markdown 儲存格：

```
### 2.2 Correlation Analysis

分析目標：使用 Pearson 相關係數，
檢查數值特徵與考試成績之間的線性關係，
以及各數值特徵彼此的相關性。
```

新增程式碼儲存格：

```
numeric_df = df.select_dtypes(include="number")

correlation_matrix = numeric_df.corr()

plt.figure(figsize=(10, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
```

```
    center=0
)

plt.title("Correlation Heatmap")
plt.tight_layout()

plt.savefig("figures/02_correlation_heatmap.png", dpi=300)
plt.show()
```

執行後，會看到各數值特徵之間的相關係數。

3-3. 列出與考試成績最相關的數值特徵

再新增一格：

```
score_corr = (
    correlation_matrix["Exam_Score"]
    .drop("Exam_Score")
    .sort_values(ascending=False)
)

display(score_corr.to_frame("Correlation with Exam_Score"))
```

這會產生一個由高到低排序的相關係數表。

後續我們會比較：

EDA 發現的重要特徵，是否與 RFE 真正選出的特徵一致？

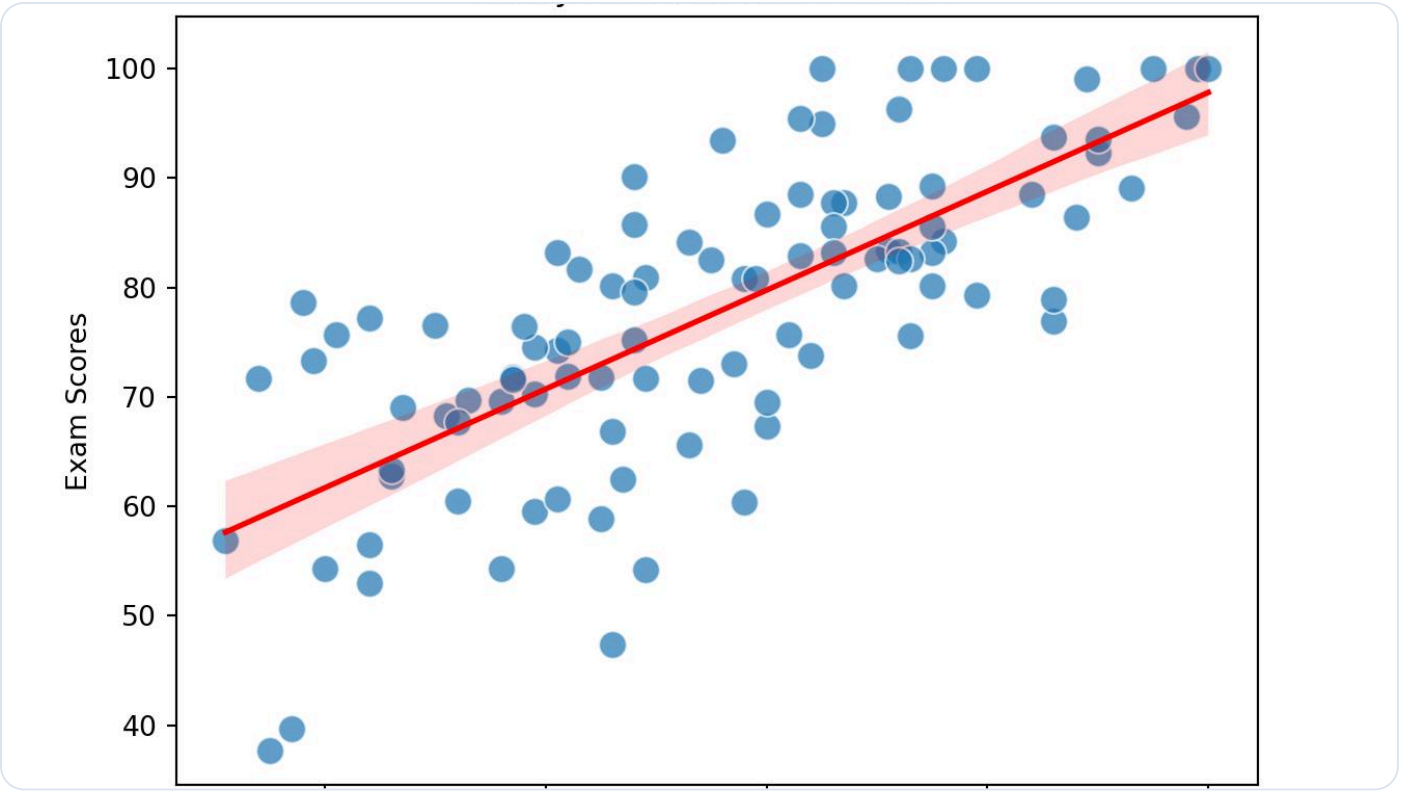
這是很值得放進報告討論的內容。

不過要記得，Pearson 相關係數只反映兩個變數的線性關係，並沒有控制其他特徵，也沒有涵蓋目前的類別特徵。因此不能只靠這張表完成正式的 Feature Selection。

Step 4：Fig. 3 讀書時間與考試成績散佈圖

研究問題：每週讀書時間與考試成績是否存在線性關係？

這張圖的價值在於，它可以直接銜接下一階段的 Model A（Simple Linear Regression）。



散佈圖與趨勢線示意；實際關係需由資料驗證。

4-1. 建立 Scatter Plot

新增 Markdown 儲存格：

```
### 2.3 Hours Studied vs. Exam Score

分析目標：探索每週讀書時間與考試成績之間的關係，
觀察是否存在適合線性回歸描述的趨勢。
```

新增 Python 儲存格：

```
plt.figure(figsize=(9, 5))

sns.regplot(
    data=df,
    x="Hours_Studied",
    y="Exam_Score",
    scatter_kws={"alpha": 0.3},
    line_kws={"color": "red"},
    ci=None
)
```

```
plt.title("Hours Studied vs. Exam Score")
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.tight_layout()

plt.savefig("figures/03_study_hours_vs_score.png", dpi=300)
plt.show()
```

這裡使用 `sns.regplot()`，它會繪製散佈點與線性趨勢線。

注意，這條趨勢線目前只是 EDA 的探索性分析，不是我們之後正式訓練並評估的 Model A。`ci=None` 表示這張圖先不繪製區間；作業要求的 95% 預測區間會在 Evaluation 階段另外製作。

4-2. 如何判斷結果？

觀察結果	可能的解釋
趨勢線往右上方	讀書時間與考試成績呈正向關係
趨勢線接近水平	單靠讀書時間可能難以解釋成績差異
散佈點非常分散	即使有線性趨勢，仍可能需要更多特徵
散佈點呈彎曲形狀	可能存在非線性關係
少數點遠離大部分資料	可能有極端值，需要進一步確認

這裡要避免寫成「多讀書一定會提高成績」。因為我們觀察到的是統計關聯，不是因果效果。

Step 5：整理 Data Understanding 的初步結論

當你完成三張圖之後，可以先在 Notebook 留下一個 Markdown 章節：

2.4 Preliminary Findings

本研究透過探索式資料分析（EDA），檢視 Student Performance Factors 資料集的分布特性、數值變數之間的相關性，以及學習時間與考試成績的關係。

考試成績分布：

（根據 Fig. 1 填入平均成績、主要分布範圍及極端值觀察。）

特徵相關性：

（根據 Fig. 2 填入與 Exam_Score 相關性較高的數值特徵，並說明是否存在特徵間的高度相關。）

讀書時間與成績：

（根據 Fig. 3 說明兩者是否呈現正向線性關係，以及資料點的分散程度。）

下一階段分析方向：

由於資料同時包含數值及類別特徵，後續將進行缺失值處理、類別編碼與訓練測試集切分，再建立單純線性回歸、多元線性回歸及 RFE 特徵選擇模型，驗證不同特徵組合的預測效果。

先保留括號中的提示，等你把真正的結果提供給我，我們再一起完成有數據支持的研究結論。

本次作業檢查清單

EDA 分析進度 0/5 已完成

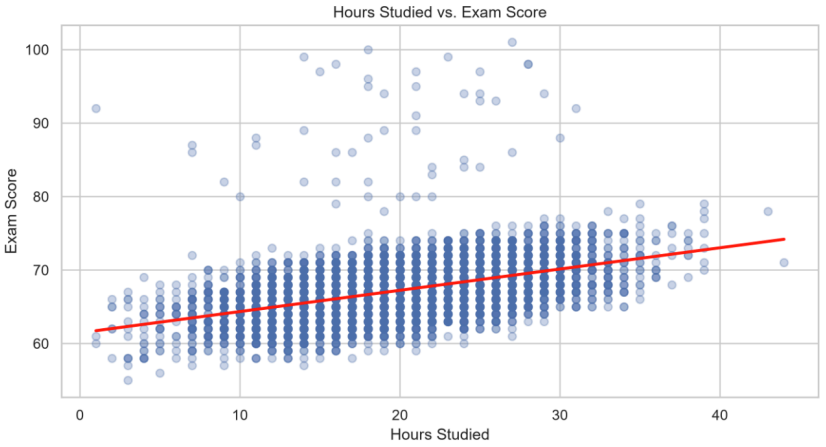
- [ ] 已確認缺失值數量與比例
- [ ] 完成 Fig. 1：考試成績分布圖
- [ ] 完成 Fig. 2：Correlation Heatmap
- [ ] 完成 Fig. 3：讀書時間與成績散佈圖
- [ ] 三張圖都成功儲存到 figures 資料夾

完成 EDA，繼續下一步 →

這次完成後，請把三張圖傳給我，尤其是 Fig. 1 和 Fig. 2。

我會先幫你解讀實際結果，包括哪些特徵值得注意、是否存在資料異常，以及這些觀察如何寫進 PDF 報告。確認 EDA 結果後，我們才進入 **Data Preparation：資料前處理與訓練測試集切分**。

2026/10/16 下午1:03:56 · gpt-6-thinking



2026/10/16 下午1:07:23

很好！你這三張 EDA 圖表都成功了，而且已經可以從中發現一些值得討論的現象。

我先幫你解讀三張圖，再教你下一個步驟。這些分析可以直接作為 HW2 報告中 **Data Understanding** 的重要內容。

一、Fig. 1：考試成績分布圖

從你的第一張圖可以觀察到：

- **平均成績為 67.24 分**，由紅色虛線標示。
- 大部分學生成績集中在約 **60～75 分**。
- 分布主體接近鐘形，但右側有少數高分紀錄，呈現右尾較長的現象。
- 少部分成績高於 90 分，甚至有接近或超過 100 分的紀錄。

這代表什麼？

整體而言，學生成績主要集中在中間區間，但存在少數明顯偏離主要分布的高分紀錄。

這些極端值值得進一步檢查，尤其 `Exam_Score` 如果應該限制在 0～100 分，超過 100 的紀錄就需要說明。

不過目前**先不要刪除極端值**，因為我們還不知道它們對線性回歸的實際影響。

二、Fig. 2：相關係數熱力圖

這張圖提供了目前最重要的發現。

我把你圖中 `Exam_Score` 與其他數值特徵的相關係數整理如下：

排名	特徵	Pearson r	解讀
1	`Attendance`	+0.58	中度正相關
2	`Hours_Studied`	+0.45	中度正相關
3	`Previous_Scores`	+0.18	弱正相關
4	`Tutoring_Sessions`	+0.16	弱正相關
5	`Physical_Activity`	+0.03	極弱正相關
6	`Sleep_Hours`	-0.02	幾乎無線性相關

三個值得寫進報告的發現

第一，出席率比讀書時間具有更高的線性相關性。

`Attendance` 的相關係數為 0.58，而 `Hours_Studied` 為 0.45。

這表示在目前資料中，出席率與成績具有較強的正向線性關聯，但不代表出席率一定具有更大的因果影響。

第二，過去成績及補習次數也具有一定關聯。

`Previous_Scores` 和 `Tutoring_Sessions` 的相關係數分別為 0.18、0.16。

雖然不高，但在多元線性回歸中，這些特徵仍可能提供額外資訊。

第三，數值特徵之間沒有明顯的高度線性相關。

從熱力圖可以看到，數值輸入特徵彼此的相關係數大多接近 0。

這表示目前沒有觀察到明顯的成對高度相關問題，對建立多元線性回歸是有利的。

不過，這還不能完全排除多重共線性。後續若要深入檢查，可以再觀察 VIF。

三、Fig. 3：讀書時間與考試成績散佈圖

這張圖可以得到三個結論。

1. 存在明顯的正向線性趨勢

紅色迴歸趨勢線由左下往右上，表示讀書時間越長的學生，平均而言，考試成績也傾向越高。

這與 Heatmap 中 `Hours_Studied` 和 `Exam_Score` 的相關係數 +0.45 一致。

2. 單靠讀書時間還無法準確預測所有學生的成績

你可以看到，在相同讀書時間附近，考試成績仍然有差異。

這表示成績可能還受到出席率、過去成績、學習資源等其他因素影響。

因此，這張圖非常適合作為我們從單純線性回歸進一步發展多元線性回歸的理由。

3. 存在少數特別高的成績紀錄

有部分學生分數落在 80～100 分以上，明顯高於主要的密集區。

這些點可能對模型的殘差、RMSE 和預測區間造成影響，後續 Evaluation 應特別留意。

四、可以放入 HW2 的 EDA 結論

下面這段可以直接放進 Notebook 的 Markdown 儲存格，作為 Data Understanding 的初步研究發現。

2.4 Exploratory Data Analysis Findings

本研究針對 Student Performance Factors 資料集進行探索式資料分析（EDA），以了解學生成績的分布特性，以及各項數值特徵與考試成績之間的關係。

首先，由考試成績分布圖可以觀察到，學生平均考試成績約為 **67.24 分**，大部分成績集中在 60～75 分之間，整體分布接近鐘形，但右側存在少數明顯偏高的成績紀錄，顯示資料可能具有極端值，後續建模時需進一步評估其影響。

其次，Pearson 相關係數分析顯示，**Attendance（出席率）與 Exam_Score 的相關係數為 0.58**，是六個數值輸入特徵中最高者；其次為 **Hours_Studied（讀書時間），相關係數為 0.45**。Previous_Scores 與 Tutoring_Sessions 分別具有 0.18 與 0.16 的弱正相關，而 Sleep_Hours 和 Physical_Activity 與成績的線性相關性較低。

最後，讀書時間與考試成績的散佈圖呈現正向線性趨勢，但相同讀書時間的學生仍存在一定程度的成績差異，顯示單一特徵可能不足以完整預測考試成績。

綜合以上結果，本研究將進一步建立單純線性回歸與多元線性回歸模型，並使用 RFE 特徵選擇方法，比較不同特徵組合的預測表現。需要注意的是，本資料集屬於合成資料，因此上述結果反映的是資料中的統計關聯，而非真實教育情境中的因果關係。

五、下一步：檢查極端值與缺失值

在開始 Data Preparation 之前，我建議先做最後一項資料品質檢查。

因為你的圖中有少數成績高於 100 分的紀錄，我們需要知道實際有多少筆。

請在 VS Code Notebook 新增一個程式碼儲存格：

```
# 檢查 Exam_Score 的分數範圍
print("最低成績：", df["Exam_Score"].min())
print("最高成績：", df["Exam_Score"].max())

# 統計超過 100 分的紀錄
above_100 = df[df["Exam_Score"] > 100]

print("超過 100 分的筆數：", len(above_100))

display(above_100)
```

接著新增下一個儲存格：

```
# 檢查各欄位缺失值
missing_summary = pd.DataFrame({
    "Missing Count": df.isnull().sum(),
    "Missing Rate (%)": df.isnull().mean() * 100
})

display(missing_summary[missing_summary["Missing Count"] > 0])
```

這兩段程式會幫助我們確認：

- 成績是否存在超出合理範圍的紀錄，以及實際筆數。
- 哪些欄位有缺失值，比例有多高。

請先執行這兩段程式，把輸出結果貼給我。

下一步，我會依照你的實際結果，帶你完成 **Data Preparation（資料前處理）**，包括：

1. 決定異常成績如何處理。
2. 訓練集與測試集切分（80% / 20%）。
3. 數值缺失值填補與類別缺失值填補。
4. One-Hot Encoding。
5. 建立可供後續三個線性回歸模型共用的資料處理流程。

這樣我們就能正式從 EDA 進入模型訓練階段。

2026/10/16 下午1:07:30 · gpt-6-thinking

	Missing Count	Missing Rate (%)
Teacher_Quality	78	1.180566
Parental_Education_Level	90	1.362192
Distance_from_Home	67	1.014076

最低成績： 55\
最高成績： 101\
超過 100 分的筆數： 1

 Hours_Studied	**Attendance**	**Parental_Involvement**	**Access_to_Resources**	**Extracurricular_Activities**	**Sleep_Hours**	**Previous_Scores**	**Motivation_Level**	**Internet_Access**	**Tutoring_Sessions**	**Family_Income**	**Teacher_Quality**	**School_Type**	**Peer_Influence**	**Physical_Activity**	**Learning_Disabilities**	**Parental_Education_Level**	**Distance_from_Home**	**Gender**	**Exam_Score**	
1525	27	98	Low	Medium	Yes	6	93	Low	No	5	High	High	Public	Positive	3	No	High School	Moderate	Female	101

2026/10/16 下午1:10:47

很好！現在我們已經掌握這份資料集的主要品質問題，可以正式從 **Data Understanding 進入 Data Preparation（資料前處理）**。

你提供的結果很有價值，因為我們可以根據實際資料決定怎麼處理，而不是隨意刪除資料。

一、先分析你的資料品質檢查結果

1. 缺失值分析

你的資料共有 6,607 筆，只有三個欄位存在缺失值。

特徵	缺失筆數	缺失比例
`Teacher_Quality`	78	1.18%
`Parental_Education_Level`	90	1.36%
`Distance_from_Home`	67	1.01%

這三個欄位都是**類別型特徵**。

缺失比例都很低，因此不需要直接刪除整個特徵。

我建議：

使用眾數（Most Frequent）填補缺失值。

例如 `Teacher_Quality` 如果最常出現 `Medium`，就用 `Medium` 填補該欄位的缺失值。

不過，這個眾數必須只根據**訓練資料**計算，不能使用完整資料集計算，避免 Data Leakage（資料洩漏）。

2. 異常成績分析

55	101	1
最低成績	最高成績	超過 100 分的筆數

只有一筆成績為 101 分，約占全部資料的 0.015%。

因為一般百分制考試成績最高是 100 分，而且我們沒有證據顯示 101 是有效的加分成績，所以這次作業我建議：

將這 1 筆 `Exam_Score = 101` 的紀錄排除，並在報告中明確註明原因。

不建議把 101 強制改成 100，因為這相當於自行修改原始標籤，沒有原始資料能證明真正的成績應該是 100。

這是以百分制假設為前提的清理決策，而不是認定原始資料一定有錯。

清理後預期：

項目	清理前	清理後
資料筆數	6,607	6,606
輸入特徵數	19	19
預測目標	Exam_Score	Exam_Score

接下來，先執行這一步，再開始切分訓練集與測試集。

二、Step 7：資料清理與訓練測試集切分

CRISP-DM — Data Preparation，第 1 部分

這一輪我們只做兩件事：

1. 排除那筆 101 分的紀錄。
2. 將資料切分為訓練集（80%）與測試集（20%）。

缺失值填補、One-Hot Encoding 和標準化留到下一步，讓你比較容易理解每個環節。

Step 7-1：排除異常成績

在 VS Code 的 Notebook 新增 Markdown 儲存格：

```
## 3. Data Preparation

### 3.1 Data Cleaning

本研究採用 0～100 分的考試成績範圍。
資料集中發現一筆 Exam_Score = 101 的紀錄，
因此將其排除，避免超出研究定義的分數範圍。

其他資料保持不變，缺失值將於後續的
前處理 Pipeline 中處理。
```

接著新增程式碼儲存格：

```
# 保留 Exam_Score 在 0～100 分之間的資料
df_clean = df[
    df["Exam_Score"].between(0, 100)
].copy()

print("原始資料筆數：", len(df))
print("清理後資料筆數：", len(df_clean))
print("排除資料筆數：", len(df) - len(df_clean))
```

預期輸出：

```
原始資料筆數： 6607
清理後資料筆數： 6606
排除資料筆數： 1
```

為什麼使用 `.copy()`？

因為我們希望保留原始的 `df`，並建立另一份清理後的 `df_clean`。

這樣你之後可以比較清理前後的資料，也不會意外修改原始資料。

Step 7-2：分離 X 與 y

在線性回歸中：

- **X（Features）**：用來預測的輸入特徵。
- **y（Target）**：我們想預測的結果。

本研究的 y 就是 `Exam_Score`。

新增程式碼：

```
# 分離輸入特徵與預測目標
X = df_clean.drop(columns=["Exam_Score"])
y = df_clean["Exam_Score"]

print("X 的形狀：", X.shape)
print("y 的形狀：", y.shape)
```

預期輸出：

```
X 的形狀： (6606, 19)
y 的形狀： (6606,)
```

這裡要特別理解：X 有 **19 個原始特徵**，符合老師要求的 10～20 個特徵。

而 y 是一個一維序列，包含 6,606 筆考試成績。

Step 7-3：切分訓練集與測試集

這一步非常重要。

如果我們用相同的資料訓練模型，又用相同的資料評估模型，就無法可靠地知道模型對未見資料的預測能力。

所以我們採用：

新增程式碼：

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

預期輸出：

```
X_train: (5284, 19)
X_test: (1322, 19)
y_train: (5284,)
y_test: (1322,)
```

為什麼使用 `random_state=42`？

因為 `train_test_split()` 會隨機分配資料。

設定固定的 `random_state` 可以讓你每次重新執行 Notebook 時，取得相同的訓練集與測試集，方便重現研究結果。

42 只是常用的隨機種子，並沒有特殊的統計意義。

三、現在的資料處理流程

原始資料 df

6,607 筆 × 20 欄

↓

清理資料 df_clean

排除一筆 101 分紀錄，保留 6,606 筆

↓

分離 X 與 y

19 個輸入特徵 + 1 個預測目標

↓

Training Set	Testing Set
5,284	**1,322**
筆資料	筆資料

四、這一階段可以寫進報告的內容

3.1 Data Cleaning and Train-Test Split

本研究原始資料集共有 6,607 筆紀錄及 20 個欄位，其中 19 個為輸入特徵，Exam_Score 為預測目標。

資料品質檢查發現 Teacher_Quality、Parental_Education_Level 及 Distance_from_Home 分別有 78、90 與 67 筆缺失值，缺失比例介於 1.01%～1.36%。由於缺失比例較低且皆為類別特徵，本研究規劃於模型前處理階段使用訓練資料的眾數進行填補。

另外，資料中存在一筆 Exam_Score 為 101 分的紀錄。本研究將考試成績定義為 0～100 分，因此排除此筆超出研究範圍的資料，保留 6,606 筆有效紀錄。

接著將 Exam_Score 設定為目標變數 y，其餘 19 個特徵作為輸入變數 X，並使用 train_test_split，以 80% 作為訓練集、20% 作為測試集，設定 random_state=42 以確保結果可重現。

最終訓練集包含 5,284 筆資料，測試集包含 1,322 筆資料。後續缺失值填補、特徵編碼與特徵選擇皆依訓練資料建立，以降低資料洩漏風險。

五、你現在要完成的事情

- [ ] 執行 Step 7-1，確認清理後有 6,606 筆
- [ ] 執行 Step 7-2，確認 X 有 19 個特徵
- [ ] 執行 Step 7-3，確認訓練集 5,284 筆、測試集 1,322 筆

完成 Step 7，繼續下一步 →

這一輪先做到這裡。**請把四個 Shape 的輸出結果貼給我**，確認資料切分成功。

下一步我們會學習 `ColumnTransformer` 和 `Pipeline`，把數值與類別特徵分別處理。這會是後續建立三個回歸模型，以及正確執行 RFE 特徵選擇的基礎。

2026/10/16 下午1:11:47 · gpt-6-thinking

```
原始資料筆數： 6607
清理後資料筆數： 6606
排除資料筆數： 1
```

```
X 的形狀： (6606, 19)
y 的形狀： (6606,)
```

```
X_train: (5284, 19)
X_test: (1322, 19)
y_train: (5284,)
y_test: (1322,)
```

2026/10/16 下午1:14:26

很好！你提供的三張執行結果都正確，代表我們已經成功完成 **資料清理、特徵與目標分離，以及訓練集／測試集切分**。

現在可以正式進入 CRISP-DM 的 **Data Preparation 第二部分：資料前處理**。

一、先確認目前的進度

檢查項目	你的執行結果	狀態
原始資料	6,607 筆	✅
清理後資料	6,606 筆	✅
輸入特徵	19 個	✅
訓練集	5,284 筆	✅
測試集	1,322 筆	✅

接下來要解決的問題是：

我們的 19 個特徵有數值，也有文字，該如何讓 Linear Regression 能夠使用它們？

答案是使用 `ColumnTransformer` 與 `Pipeline`。

二、Step 8：區分數值與類別特徵

CRISP-DM — Data Preparation，第 2 部分

這個步驟會完成：

1. 自動辨識數值與類別特徵。
2. 使用適當方法填補缺失值。
3. 將文字特徵轉為數值。
4. 對數值特徵進行標準化。
5. 建立可供後續三個模型使用的前處理器。

Step 8-1：辨識欄位型態

先在 VS Code 的 Notebook 新增 Markdown 儲存格：

```
### 3.2 Feature Type Identification

本研究將輸入特徵區分為數值型與類別型特徵，
並根據資料型態使用不同的前處理方法。
```

接著新增程式碼儲存格：

```
# 區分數值型與類別型特徵
numeric_features = X_train.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

categorical_features = X_train.select_dtypes(
    include=["object", "category"]
).columns.tolist()

print("數值特徵：", numeric_features)
print("類別特徵：", categorical_features)

print("\n數值特徵數量：", len(numeric_features))
print("類別特徵數量：", len(categorical_features))
```

預期結果

6	**13**

數值特徵

類別特徵

這裡要特別注意：雖然原始資料有 19 個特徵，但經過 One-Hot Encoding 後，模型使用的實際欄位數會增加，這是正常現象。

三、Step 9：建立資料前處理 Pipeline

我們會依照兩種特徵類型，使用不同的處理方式。

Step 9-1：什麼是缺失值填補？

先前你的三個有缺失的欄位都是類別特徵，所以我們會用 `SimpleImputer(strategy="most_frequent")`，依照訓練資料中出現頻率最高的類別填補。

數值特徵則統一設定使用中位數填補，即使目前沒有數值缺失值，也能讓前處理步驟保持一致。

Step 9-2：什麼是 One-Hot Encoding？

假設 `School_Type` 有兩種值：

School_Type	Public	Private
Public	1	0
Private	0	1

這就是 One-Hot Encoding 的概念。

我們會設定 `drop="first"`，讓每個類別特徵少保留一個參考類別，避免與線性回歸的截距產生完全共線性。

實際使用哪些參考類別，會由編碼器依訓練資料決定。

Step 9-3：建立完整前處理器

新增程式碼儲存格：

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 數值特徵前處理
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# 類別特徵前處理
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(
        drop="first",
        handle_unknown="ignore",
        sparse_output=False
    ))
])

# 合併數值與類別前處理
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

print("前處理器建立成功！")
```

這段程式目前只是**定義前處理步驟**，還沒有使用資料進行訓練或轉換。

稍後在建立模型時，我們會把 `preprocessor` 放進模型 Pipeline 裡，確保交叉驗證時也不會產生資料洩漏。

四、Step 10：驗證前處理結果

我們先使用訓練集確認前處理器能否正常運作。

為了保留尚未訓練的 `preprocessor`，這裡使用 `clone()` 建立測試用副本。

新增程式碼：

```
from sklearn.base import clone

# 使用副本測試前處理器
preprocessor_check = clone(preprocessor)

X_train_processed = preprocessor_check.fit_transform(X_train)

feature_names = preprocessor_check.get_feature_names_out()

print("原始訓練集形狀：", X_train.shape)
print("前處理後形狀：", X_train_processed.shape)
print("前處理後特徵數：", len(feature_names))

display(pd.DataFrame(
    X_train_processed,
    columns=feature_names
).head())
```

這裡要觀察什麼？

原本我們的訓練集是：

```
(5284, 19)
```

前處理之後，資料筆數應該仍是 5,284，但欄位數量可能大於 19。

例如：

```
原始訓練集形狀： (5284, 19)
前處理後形狀： (5284, ...)
```

```
前處理後特徵數： ...
```

這是因為文字類別被展開成多個 0/1 欄位。

另外，數值欄位經過 StandardScaler 後，會以平均數約為 0、標準差約為 1 的尺度表示。

注意： 我們這裡沒有轉換 `X_test`，因為測試資料應留到正式模型評估階段才使用。

五、這段可以放進報告

3.2 Feature Preprocessing

本研究將 19 個原始輸入特徵區分為 6 個數值型特徵及 13 個類別型特徵，並使用 scikit-learn 的 ColumnTransformer 與 Pipeline 建立資料前處理流程。

數值特徵使用 SimpleImputer 的 median 策略處理可能存在的缺失值，再透過 StandardScaler 進行標準化，使不同尺度的數值特徵能以較一致的方式進入模型。

類別特徵則使用 SimpleImputer 的 most_frequent 策略填補缺失值，接著透過 OneHotEncoder 將類別資料轉換為數值欄位，並設定 drop="first" 以移除各類別特徵的一個參考類別。

所有前處理步驟皆將整合於模型 Pipeline 中，並僅使用訓練資料擬合，以避免資料洩漏。由於 One-Hot Encoding 會展開類別欄位，因此前處理後的特徵數可能高於原始的 19 個輸入特徵。

六、這一輪的完成檢查

Data Preparation 進度 0/5 已完成

- [ ] 確認 6 個數值特徵與 13 個類別特徵
- [ ] 成功建立 numeric_transformer
- [ ] 成功建立 categorical_transformer 和 preprocessor
- [ ] 執行前處理測試並取得 X_train_processed.shape
- [ ] 成功顯示前處理後的前五筆資料

完成前處理，進入 Model A →

請先執行 Step 8～10，然後把 `X_train_processed.shape` 和前處理後的特徵數貼給我。

我會幫你確認 One-Hot Encoding 是否正確。確認後，我們就開始建立第一個正式模型：**Model A — Simple Linear Regression**，並計算 MAE、RMSE、R^2，作為後續多元線性回歸的比較基準。

2026/10/16 下午1:14:29 · gpt-6-thinking

數值特徵： ['Hours_Studied', 'Attendance', 'Sleep_Hours', 'Previous_Scores', 'Tutoring_Sessions', 'Physical_Activity']
類別特徵： ['Parental_Involvement', 'Access_to_Resources', 'Extracurricular_Activities', 'Motivation_Level', 'Internet_Access', 'Family_Income', 'Teacher_Quality', 'School_Type', 'Peer_Influence', 'Learning_Disabilities', 'Parental_Education_Level', 'Distance_from_Home', 'Gender']

數值特徵數量： 6
類別特徵數量： 13

原始訓練集形狀： (5284, 19)
前處理後形狀： (5284, 27)
前處理後特徵數： 27

2026/10/16 下午1:16:51

很好！你的前處理結果完全符合預期，代表 **Data Preparation 已經成功完成**，現在可以開始建立第一個正式的線性回歸模型了！

一、先確認你的前處理結果

項目	執行結果	狀態
原始輸入特徵	19	✓
數值特徵	6	✓
類別特徵	13	✓
訓練資料筆數	5,284	✓
編碼後特徵數	27	✓

為什麼 19 個特徵變成 27 個？

因為 One-Hot Encoding 將 13 個類別特徵展開成了 21 個編碼欄位，加上原本的 6 個數值特徵：

$$6 + 21 = 27$$

這是正常的，也**不違反老師要求的 10～20 個原始特徵**。記得在報告中註明原始特徵數為 19，編碼後為 27。

二、Step 11：建立 Model A — Simple Linear Regression

CRISP-DM 第四階段：Modeling

這次我們先建立最簡單的模型，只使用一個特徵：

`Hours_Studied`（讀書時間）→ `Exam_Score`（考試成績）

目的是建立 Baseline Model（基準模型），讓後續的多元線性回歸有比較對象。

Step 11-1：了解簡單線性回歸

簡單線性回歸的概念是透過一條直線，找出輸入特徵與預測目標之間的關係。

Unsupported component: learning_block

```
\hat{y} = b_0 + b_1x
```

公式中的：

- x：讀書時間 `Hours_Studied`
- $\hat{y}$：模型預測的考試成績
- b_0：截距
- b_1：讀書時間的迴歸係數

模型會利用最小平方法（Ordinary Least Squares）尋找適合訓練資料的直線。

Step 11-2：建立模型

請在 VS Code 的 Notebook 新增一個 Markdown 儲存格：

```
## 4. Modeling

### 4.1 Model A: Simple Linear Regression
```

本模型僅使用 Hours_*Studied* *作為輸入特徵，*
建立簡單線性回歸模型，作為後續多元線性回歸的比較基準。

接著新增 Python 程式碼儲存格：

```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Model A 只使用 Hours_Studied
X_train_A = X_train[["Hours_Studied"]]
X_test_A = X_test[["Hours_Studied"]]

model_A = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("regressor", LinearRegression())
])

# 訓練模型
model_A.fit(X_train_A, y_train)

# 預測測試集
y_pred_A = model_A.predict(X_test_A)

print("Model A 訓練完成！")
```

為什麼這裡沒有使用剛剛建立的 `preprocessor`？

因為 Model A 只使用一個數值特徵，不包含類別資料，因此不需要 One-Hot Encoding。

我們使用簡單的 Pipeline 進行標準化與線性回歸，讓 Model A 與後續模型都遵循「在訓練資料內擬合前處理器」的原則。

Step 11-3：評估 Model A

我們使用三個指標：

指標	英文全名	用途
MAE	Mean Absolute Error	平均預測誤差的絕對值

指標	英文全名	用途
RMSE	Root Mean Squared Error	對較大的預測誤差更敏感
R^2	Coefficient of Determination	衡量模型對成績變異的解釋能力

新增程式碼：

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae_A = mean_absolute_error(y_test, y_pred_A)
rmse_A = np.sqrt(mean_squared_error(y_test, y_pred_A))
r2_A = r2_score(y_test, y_pred_A)

print("=== Model A Evaluation ===")
print(f"MAE:  {mae_A:.4f}")
print(f"RMSE: {rmse_A:.4f}")
print(f"R²:   {r2_A:.4f}")
```

如何解讀？

假設 MAE 是 3.5，代表模型在測試集上的預測誤差，平均絕對值約為 3.5 分。

如果 R^2 是 0.20，表示模型相對於以測試集平均值作為參考的預測方式，減少約 20% 的平方誤差；不能解讀為「模型預測有 20% 正確」。

這些只是示例，實際數字請以你執行的結果為準。

三、Step 12：繪製 Model A 的預測結果

老師要求預測圖，因此我們先幫 Model A 繪製第一張正式的 Actual vs. Predicted 圖。

這張圖之後可以和 Model B、Model C 比較。

新增程式碼：

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

plt.scatter(
```

```
    y_test,
    y_pred_A,
    alpha=0.35,
    label="Predictions"
)

min_score = min(y_test.min(), y_pred_A.min())
max_score = max(y_test.max(), y_pred_A.max())

plt.plot(
    [min_score, max_score],
    [min_score, max_score],
    "r--",
    label="Perfect Prediction"
)

plt.xlabel("Actual Exam Score")
plt.ylabel("Predicted Exam Score")
plt.title("Model A: Actual vs. Predicted")
plt.legend()
plt.tight_layout()

plt.savefig("figures/04_model_A_predictions.png", dpi=300)
plt.show()
```

怎麼看這張圖？

圖上的紅色虛線代表理想預測，即：

$$\text{Actual Score} = \text{Predicted Score}$$

散佈點越靠近紅線，表示預測越接近實際成績。

如果散佈點離紅線很遠，代表模型在那些學生身上的預測誤差較大。

因為 Model A 只使用讀書時間，所以我們預期它無法完整解釋所有學生的成績差異，但究竟能預測得多好，仍需要看實際測試結果。

四、這一階段要記錄到報告的內容

4.1 Model A: Simple Linear Regression

本研究首先建立簡單線性回歸模型（Simple Linear Regression），以 Hours_Studied 作為唯一輸入特徵，Exam_Score 作為預測目標，藉此建立後續模型比較的基準。

模型使用 scikit-learn 的 LinearRegression 建立，並透過 Pipeline 結合 StandardScaler，確保數值標準化僅依據訓練資料進行。

訓練集包含 5,284 筆資料，測試集包含 1,322 筆資料。模型訓練完成後，使用 MAE、RMSE 及 R^2 評估測試集預測效能，並繪製 Actual vs. Predicted 散佈圖，以觀察模型預測值與真實考試成績之間的差異。

本模型作為 Baseline，後續將加入其他數值及類別特徵建立完整多元線性回歸模型，驗證增加特徵是否能有效改善預測能力。

模型評估結果：

- MAE：待實測
- RMSE：待實測
- R^2：待實測

五、這次先完成 Model A

Model A 完成進度 0/4

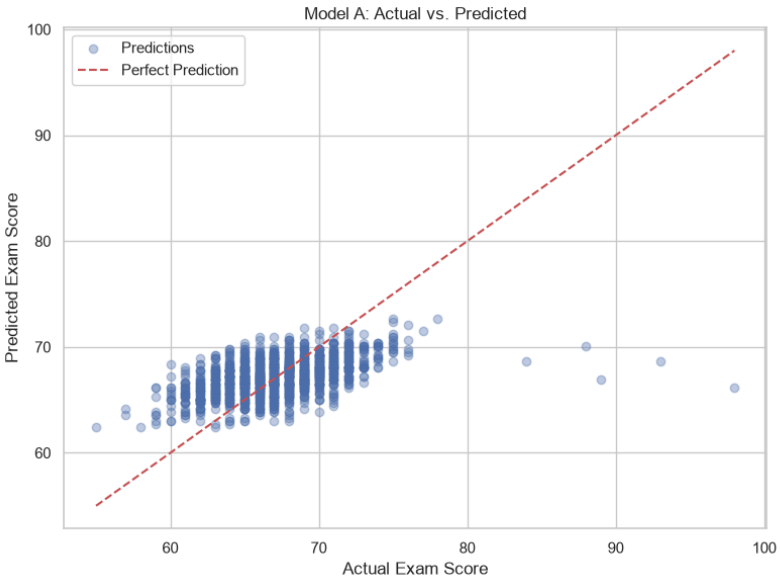
- [ ] 成功建立並訓練 Model A
- [ ] 取得 Model A 的 MAE、RMSE、R^2
- [ ] 成功繪製 Actual vs. Predicted 圖
- [ ] 圖表已儲存到 figures 資料夾

完成 Model A，繼續下一步 →

請把 Model A 的三個指標（MAE、RMSE、R^2）以及預測圖傳給我。

下一步我們將建立 **Model B — Multiple Linear Regression**，正式使用全部 19 個原始特徵（前處理後 27 欄），比較它是否明顯優於只有讀書時間的 Model A。

2026/10/16 下午1:16:56 · gpt-6-thinking



=== Model A Evaluation ===
MAE: 2.4190
RMSE: 3.1559
R^2: 0.2469

2026/10/16 下午1:19:56

很好！你的 **Model A（Simple Linear Regression）已經成功完成**，而且這張預測圖很適合用來說明為什麼我們需要進一步建立多元線性回歸。

這次我們先解讀 Model A，再帶你建立 **Model B：Multiple Linear Regression**。

一、Model A 的結果分析

你實際執行得到：

Model A — Simple Linear Regression

2.4190	3.1559	0.2469
MAE	RMSE	R^2

1. MAE = 2.4190

表示 Model A 在測試資料上的平均絕對預測誤差約為 **2.42 分**。

例如實際分數為 70 分，預測為 67 分，該筆資料的絕對誤差就是 3 分。

2. RMSE = 3.1559

RMSE 比 MAE 高，表示某些較大的預測誤差對整體評估產生了影響。

從你的預測圖也能看到，部分實際分數超過 85 分的學生，模型仍預測在 67～70 分左右。

這些紀錄會造成較大的平方誤差。

3. R^2 = 0.2469

代表模型相較於以測試集平均成績作為固定預測值的基準，約降低了 **24.69% 的平方誤差**。

換句話說，單靠讀書時間確實能提供預測資訊，但仍存在相當大的未解釋變異。

二、你的預測圖透露了什麼？

觀察 1：預測分數集中在較窄的範圍

你的模型主要預測約 62～73 分，但實際成績範圍更廣。這代表使用單一特徵時，模型難以預測非常高或非常低的分數。

觀察 2：高分學生被明顯低估

在圖的右側，實際分數為 85～100 分附近的學生，預測值卻只有約 67～70 分。

這是目前模型值得改善的地方，也可能與少數高分極端值有關。

觀察 3：低分學生有被高估的情況

部分實際成績約 55～60 分的學生，模型預測值卻落在 62～67 分附近。

綜合來看，Model A 存在**預測值過度集中於平均附近**的現象。

不過，這不代表程式有錯，而是單一讀書時間特徵無法完整區分不同學生的成績。

這正好可以作為下一個模型的研究動機。

三、Model A 的報告文字

4.1 Model A：Simple Linear Regression 結果分析

本研究首先使用 Hours_Studied 作為唯一輸入特徵，建立簡單線性回歸模型，並以獨立測試集評估其預測效能。

實驗結果顯示，Model A 的 MAE 為 **2.4190**、RMSE 為 **3.1559**，R^2 為 **0.2469**，表示僅使用讀書時間能夠提供一定程度的成績預測能力，但仍有許多成績變異無法被模型解釋。

由 Actual vs. Predicted 散佈圖可觀察到，模型預測值大多集中在約 62～73 分之間。對於部分實際成績較高的學生，模型出現明顯低估的情況；部分低分學生則有被高估的現象。

上述結果顯示，單純使用讀書時間可能不足以準確預測不同學生的考試成績。因此，下一階段將加入出席率、過去成績、家庭背景與學校環境等其他特徵，建立完整多元線性回歸模型，以評估增加特徵是否能改善預測準確度。

四、Step 13：建立 Model B — Multiple Linear Regression

CRISP-DM：Modeling，第 2 部分

現在我們要從使用 1 個特徵，進步到使用全部 **19 個原始特徵（經前處理後為 27 欄）**。

研究問題是：

加入出席率、過去成績、家庭背景、學校環境等資訊後，模型能否比 Model A 預測得更準確？

Step 13-1：建立 Model B

先新增一個 Markdown 儲存格：

```
### 4.2 Model B: Multiple Linear Regression

本模型使用全部 19 個原始輸入特徵，
經由缺失值填補、標準化及 One-Hot Encoding 後，
建立多元線性回歸模型。

研究目的是比較使用多個特徵，
是否能比 Model A 獲得更好的預測效果。
```

接著建立模型。

這裡會直接使用你前面已經建立的 `preprocessor`，所以不需要重新寫缺失值處理或 One-Hot Encoding 的程式。

```
from sklearn.pipeline import Pipeline
from sklearn.base import clone
from sklearn.linear_model import LinearRegression
```

```
model_B = Pipeline(steps=[
    ("preprocessor", clone(preprocessor)),
    ("regressor", LinearRegression())
])

# 使用完整訓練集訓練 Model B
model_B.fit(X_train, y_train)

# 預測測試集
y_pred_B = model_B.predict(X_test)

print("Model B 訓練完成！")
```

這段程式有三個重要觀念

1. `clone(preprocessor)`：使用獨立的前處理器，避免其他模型共用已訓練的前處理狀態。
2. `fit(X_train, y_train)`：讓前處理器與線性回歸模型只使用訓練集進行學習。
3. `predict(X_test)`：對之前未參與訓練的測試集進行預測。

你不需要手動把 `X_train` 轉成 27 欄，Pipeline 會自動完成這件事。

Step 13-2：評估 Model B

使用與 Model A 完全相同的 MAE、RMSE、R^2。

```
mae_B = mean_absolute_error(y_test, y_pred_B)
rmse_B = np.sqrt(mean_squared_error(y_test, y_pred_B))
r2_B = r2_score(y_test, y_pred_B)

print("=== Model B Evaluation ===")
print(f"MAE:  {mae_B:.4f}")
print(f"RMSE: {rmse_B:.4f}")
print(f"R²:   {r2_B:.4f}")
```

執行後先觀察：

- MAE 是否小於 2.4190？
- RMSE 是否小於 3.1559？
- R^2 是否大於 0.2469？

如果三個方向都改善，代表使用多個特徵確實比只使用讀書時間更有預測能力。

但這裡先不假設 Model B 一定會比較好，必須依照實際結果判斷。

Step 13-3：比較 Model A 與 Model B

這張比較表會是未來報告的重要成果。

```
comparison = pd.DataFrame({
    "Model": [
        "Model A: Simple LR",
        "Model B: Multiple LR"
    ],
    "MAE": [mae_A, mae_B],
    "RMSE": [rmse_A, rmse_B],
    "R2": [r2_A, r2_B]
})

display(comparison.round(4))
```

預期輸出格式：

Model	MAE	RMSE	R2
Model A: Simple LR	2.4190	3.1559	0.2469
Model B: Multiple LR	待實測	待實測	待實測

這裡的 Model B 還沒有特徵選擇；它代表使用全部原始特徵的完整模型。

五、Step 14：製作 Model B 預測圖

接下來我們畫出與 Model A 相同形式的預測圖，方便直接比較。

```
plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    y_pred_B,
    alpha=0.35,
    label="Predictions"
)
```

```
min_score = min(y_test.min(), y_pred_B.min())
max_score = max(y_test.max(), y_pred_B.max())

plt.plot(
    [min_score, max_score],
    [min_score, max_score],
    "r--",
    label="Perfect Prediction"
)

plt.xlabel("Actual Exam Score")
plt.ylabel("Predicted Exam Score")
plt.title("Model B: Actual vs. Predicted")
plt.legend()
plt.tight_layout()

plt.savefig("figures/05_model_B_predictions.png", dpi=300)
plt.show()
```

這次你要特別觀察什麼？

與 Model A 相比，重點不只是散佈點有沒有更靠近紅線，還要觀察高分學生是否仍被嚴重低估。

如果 Model B 的預測值分布變得更廣，而且整體更接近紅色對角線，表示新增的特徵可能幫助模型更有效地區分學生成績。

但如果少數高分學生仍然被低估，就要在之後的 Residual Analysis 中討論，而不是為了讓圖好看就刪除這些資料。

六、下一個階段會做什麼？

等 Model B 完成，我們就會進入這份作業的核心：**Model C — Multiple Linear Regression + RFE Feature Selection**。

Model C 不會直接使用測試集來挑選特徵，而是用訓練資料中的 5-Fold Cross-Validation 決定合適的特徵組合，最後再評估模型表現。

這樣才有辦法公平討論：保留較少特徵是否仍能維持良好的預測能力。

這次先完成 Model B

- [ ] 成功訓練 Model B

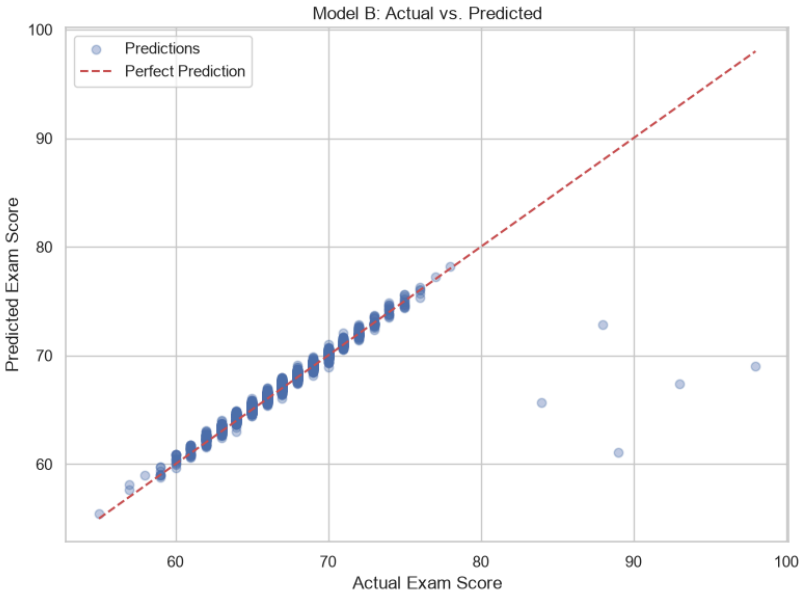
- [ ] 取得 Model B 的 MAE、RMSE、R^2
- [ ] 完成 Model A 與 Model B 的指標比較表
- [ ] 完成 Model B 的 Actual vs. Predicted 圖

完成 Model B，進入特徵選擇 →

完成後，請將 Model B 的 MAE、RMSE、R^2 和預測圖貼給我。

我們會先確認多元線性回歸帶來多少改善，再開始 RFE，避免一口氣寫完所有模型卻不知道每個步驟的研究意義。

2026/10/16 下午1:20:47 · gpt-6-thinking



=== Model B Evaluation ===
MAE: 0.4160
RMSE: 1.5213
R^2: 0.8250

2026/10/16 下午1:22:06

Model B 的改善很明顯；我會先幫你解讀指標與圖中少數高分離群點，再接著教你用訓練集交叉驗證進行 RFE，避免在特徵選擇時使用到測試集資訊。

你這次的 **Model B（Multiple Linear Regression）結果非常值得深入分析！**

相較於只使用讀書時間的 Model A，加入全部 19 個原始特徵後，模型的預測能力有明顯改善。

不過，你的圖也出現一個值得寫進報告的現象：**大部分預測點幾乎貼著理想預測線，但少數高分學生仍被嚴重低估。**

我們先分析結果，再開始作業的重要環節：**Model C — RFE 特徵選擇**。

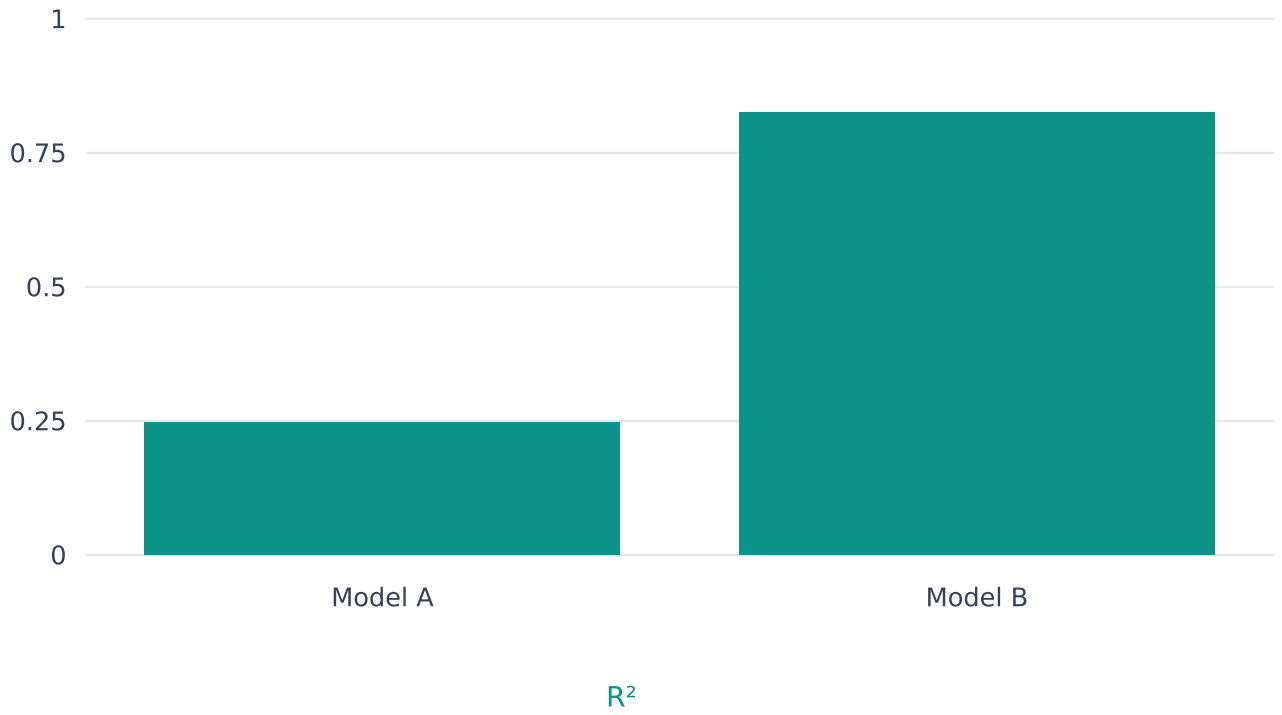
一、Model A 與 Model B 的結果比較

Model B MAE	Model B RMSE	Model B R^2
0.4160	**1.5213**	**0.8250**
Model A：2.4190	Model A：3.1559	Model A：0.2469

評估指標	Model A	Model B
MAE	2.4190	0.4160
RMSE	3.1559	1.5213
R^2	0.2469	0.8250

測試集 R^2 比較

Model B 的 R^2 明顯提升



這次改善有多大？

- **MAE 降低約 82.8%**：平均絕對預測誤差從 2.42 分降到 0.42 分。
- **RMSE 降低約 51.8%**：平方誤差較大的預測情況整體改善。
- **R^2 從 0.2469 提升至 0.8250**：增加了 0.5781。

這些結果支持研究假設 H1：多元線性回歸在這份測試資料上優於單純線性回歸。

二、Model B 預測圖的關鍵發現

從你提供的圖可以看到：

1. 大部分預測非常接近實際成績。

約 55～78 分的資料點大量分布在紅色理想預測線附近，表示模型能相當準確地預測主要分數區間。

2. 少數高分學生仍然被嚴重低估。

例如圖中有些實際分數接近 90～100 分，模型卻只預測約 61～73 分。

這些少數樣本很可能是 RMSE 仍明顯高於 MAE 的主要原因之一。

3. 高 R^2 不代表所有學生都能準確預測。

雖然 R^2 達到 0.8250，但對極端高分學生的預測能力仍有限。

這也提醒我們，後續除了比較平均指標，還要製作殘差圖及 95% 預測區間，分析模型在哪些情況下容易失準。

另外，這份資料是合成資料，大部分點異常緊密地貼著預測線，可能反映資料生成規則具有強烈的線性結構。這是值得討論的研究限制，而不是直接當作模型對真實學生也會同樣準確的證據。

三、Model B 的報告內容

4.2 Model B：Multiple Linear Regression 結果分析

本研究使用全部 19 個原始輸入特徵，經過缺失值填補、標準化及 One-Hot Encoding 後，建立多元線性回歸模型。模型使用 5,284 筆訓練資料訓練，並以 1,322 筆獨立測試資料進行評估。

實驗結果顯示，Model B 的 MAE 為 **0.4160**、RMSE 為 **1.5213**，R^2 為 **0.8250**。相較於僅使用 Hours_Studied 的 Model A，MAE 降低約 82.8%，RMSE 降低約 51.8%，R^2 則由 0.2469 提升至 0.8250。

由 Actual vs. Predicted 散佈圖可以觀察到，大部分預測結果接近理想預測線，表示整體預測表現良好。然而，部分實際成績較高的學生仍存在明顯低估現象，顯示模型對少數極端高分紀錄的預測能力有限。

綜合而言，研究結果支持增加多個特徵有助於改善考試成績預測的假設。下一階段將使用 Recursive Feature Elimination（RFE）進行特徵選擇，探討是否能在減少模型特徵數量的情況下，維持接近完整模型的預測效能。

四、Step 15：建立 Model C — RFE 特徵選擇

CRISP-DM：Modeling，第 3 部分

這是本次 HW2 必須完成的 **Feature Selection**。

我們現在要回答：

Model B 使用 27 個編碼後特徵，如果只保留其中一部分，還能維持接近的預測準確度嗎？

Step 15-1：理解 RFE

RFE 全名是 Recursive Feature Elimination（遞迴特徵消除）。

它的運作方式是：

使用全部 27 個編碼後特徵

↓

訓練線性回歸，計算特徵重要程度

↓

移除係數絕對值較小的特徵

↓

重複直到保留指定特徵數量

↓

使用 5-Fold Cross-Validation 選出合適數量

這次會測試保留 5、10、15、20、25、27 個編碼後特徵。

注意：這裡選擇的是編碼後的欄位，不是原始的 19 個特徵。RFE 也可能只保留某個類別變數的部分虛擬變數，因此後續解釋時要特別注意。

Step 15-2：建立 Model C

在 Notebook 新增 Markdown 儲存格：

```
### 4.3 Model C: Multiple Linear Regression with RFE

本研究使用 Recursive Feature Elimination (RFE)
執行特徵選擇，並搭配 5-Fold Cross-Validation，
比較不同特徵數量的模型表現。

特徵選擇與交叉驗證僅使用訓練資料，
以避免測試資料洩漏。
```

新增 Python 程式碼儲存格：

```python
from sklearn.feature_selection import RFE
from sklearn.model_selection import GridSearchCV, KFold

# 建立包含前處理、RFE、線性回歸的 Pipeline
model_C_pipeline = Pipeline(steps=[
    ("preprocessor", clone(preprocessor)),
    ("selector", RFE(estimator=LinearRegression())),
    ("regressor", LinearRegression())
])

# 測試不同的編碼後特徵數量
param_grid = {
    "selector__n_features_to_select": [5, 10, 15, 20, 25, 27]
}

# 五折交叉驗證
cv = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

# 使用 RMSE 選擇特徵數量
model_C_search = GridSearchCV(
    estimator=model_C_pipeline,
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error",
    cv=cv,
    n_jobs=-1
)

# 只使用訓練集進行特徵選擇
model_C_search.fit(X_train, y_train)

print("最佳特徵數：", model_C_search.best_params_)
print("最佳交叉驗證 RMSE：", -model_C_search.best_score_)
```

為什麼要用 GridSearchCV？

因為我們不想直接猜應該留下多少特徵。

`GridSearchCV` 會：

1. 對每個候選特徵數量進行五折交叉驗證。
2. 在每一折內重新執行資料前處理與 RFE。
3. 比較不同特徵數的驗證 RMSE。
4. 選出平均驗證 RMSE 最低的組合。

這樣可以避免先用整份訓練資料選好特徵，再做交叉驗證造成的樂觀偏差。

要注意，目前數值特徵經過標準化，但 One-Hot 欄位仍為 0/1，因此 RFE 依係數大小選擇特徵時會受到特徵尺度與編碼方式影響。它可以作為本次作業的特徵選擇實驗，但不應直接把選擇結果解讀為因果重要性排名。

Step 15-3：查看不同特徵數的交叉驗證結果

新增程式碼：

```
cv_results = pd.DataFrame({
    "Feature Count": model_C_search.cv_results_[
        "param_selector__n_features_to_select"
    ].astype(int),
    "CV RMSE": -model_C_search.cv_results_[
        "mean_test_score"
    ]
}).sort_values("Feature Count")

display(cv_results.round(4))
```

接著畫出特徵數與 RMSE 的關係。

```
plt.figure(figsize=(8, 5))

plt.plot(
    cv_results["Feature Count"],
    cv_results["CV RMSE"],
    marker="o"
)

plt.xlabel("Number of Selected Features")
plt.ylabel("Cross-Validation RMSE")
```

```
plt.title("RFE Feature Selection Performance")
plt.grid(alpha=0.3)
plt.tight_layout()

plt.savefig("figures/06_rfe_cv_results.png", dpi=300)
plt.show()
```

這張圖會成為你報告中重要的 Feature Selection 成果。

它可以幫助我們判斷，當特徵數量減少時，預測誤差是否明顯增加。

Step 15-4：查看 RFE 選中的特徵

新增程式碼：

```
# 取得交叉驗證選出的最佳模型
model_C = model_C_search.best_estimator_

# 取得經過前處理的特徵名稱
feature_names_C = model_C.named_steps[
    "preprocessor"
].get_feature_names_out()

# 取得 RFE 的特徵選擇結果
selected_mask = model_C.named_steps[
    "selector"
].support_

selected_features = feature_names_C[selected_mask]

print("RFE 選中的編碼後特徵：")
for feature in selected_features:
    print(feature)

print("\n選中數量：", len(selected_features))
```

這段輸出非常重要。

我們之後會拿它與先前 EDA 的相關係數結果比較，例如 `Attendance`、`Hours_Studied` 是否被保留，以及哪些類別特徵被 RFE 選中。

Step 15-5：先不要評估測試集

這一輪先不執行 Model C 的 `predict(X_test)`。

我們要先看交叉驗證究竟選出多少特徵，以及不同特徵數的 RMSE 差異，再解釋是否真的有達到「模型簡化」的目的。

之後再一次性評估 Model C 的獨立測試集表現，與 Model A、Model B 公平比較。

五、這一輪的完成檢查

- [ ] 成功執行 GridSearchCV + RFE
- [ ] 取得最佳特徵數與最佳 CV RMSE
- [ ] 完成 RFE Feature Selection Performance 圖
- [ ] 列出 RFE 選中的特徵名稱

完成 RFE，繼續模型評估 →

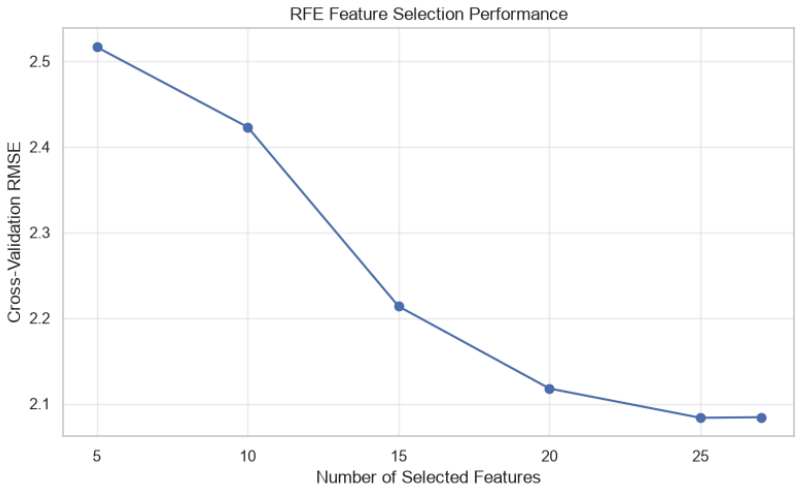
請先執行 Step 15-2～15-4，把以下三項結果貼給我：

1. `最佳特徵數` 與 `最佳交叉驗證 RMSE`。
2. `cv_results` 表格或 RFE 折線圖。
3. `selected_features` 選中的特徵名單。

下一步我們就能回答整份 HW2 的核心研究問題：**減少特徵後，是否仍能維持 Model B 的預測能力？**

2026/10/16 下午1:22:13 · gpt-6-thinking

	Feature Count	CV RMSE
0	5	2.5165
1	10	2.4230
2	15	2.2135
3	20	2.1175
4	25	2.0833
5	27	2.0840



最佳特徵數： {'selector__n_features_to_select': 25}
最佳交叉驗證 RMSE： 2.083319073717079

RFE 選中的編碼後特徵：
num__Hours_Studied
num__Attendance
num__Previous_Scores
num__Tutoring_Sessions
num__Physical_Activity
cat__Parental_Involvement_Low
cat__Parental_Involvement_Medium
cat__Access_to_Resources_Low
cat__Access_to_Resources_Medium
cat__Extracurricular_Activities_Yes
cat__Motivation_Level_Low
cat__Motivation_Level_Medium
cat__Internet_Access_Yes
cat__Family_Income_Low
cat__Family_Income_Medium
cat__Teacher_Quality_Low

cat__Teacher_Quality_Medium
cat__Peer_Influence_Neutral
cat__Peer_Influence_Positive
cat__Learning_Disabilities_Yes
cat__Parental_Education_Level_High School
cat__Parental_Education_Level_Postgraduate
cat__Distance_from_Home_Moderate
cat__Distance_from_Home_Near
cat__Gender_Male

選中數量： 25

2026/10/16 下午1:25:00

很好！你已經成功完成 **Model C 的 RFE 特徵選擇與 5-Fold Cross-Validation**，而且這次的結果很有研究價值。

因為我們發現：

使用 25 個特徵的模型表現最好，但與使用全部 27 個特徵的模型相比，差距非常小。

這正好可以用來回答 HW2 的核心研究問題：「特徵選擇是否能降低模型複雜度，同時維持預測能力？」

一、解讀 RFE 的實驗結果

1. 不同特徵數量的比較

以下是你實際執行的結果：

特徵數量	5-Fold CV RMSE	與最佳結果比較
5	2.5165	較差
10	2.4230	較差
15	2.2135	較差
20	2.1175	接近
25	**2.0833**	**最佳**
27	2.0840	幾乎相同

從你的 RFE 圖可以觀察到：

- 當特徵數從 5 增加至 20，RMSE 持續下降，表示更多特徵提供了額外的預測資訊。
- 從 20 增加至 25，RMSE 仍有改善，但改善幅度變小。
- 從 25 增加至 27，RMSE 反而略微增加，兩者僅相差約 0.0007。

因此，**在這六組候選特徵數之中，25 個編碼後特徵是交叉驗證選出的最佳設定**。

不過，0.0007 的差距非常小。沒有進一步分析各折的變異之前，不能據此認定 25 個特徵在統計上顯著優於 27 個特徵。

2. RFE 移除了哪兩個特徵？

你原本有 27 個編碼後特徵，RFE 保留其中 25 個。

根據你的名單，被移除的是：

被移除特徵	原始變數	類型
`num__Sleep_Hours`	Sleep_Hours	數值
`cat__School_Type_Public`	School_Type	類別編碼

這個結果有兩個值得討論的地方。

第一，Sleep_Hours 被移除，與 EDA 結果一致。

你先前的相關係數熱力圖顯示，`Sleep_Hours` 與 `Exam_Score` 的 Pearson r 約為 -0.02。

現在 RFE 也沒有保留它，表示在這份資料及目前的模型設定下，睡眠時間對線性預測的額外幫助可能有限。

第二，School_Type 的編碼欄位被移除。

這表示在目前的 RFE 選擇過程中，`School_Type_Public` 沒有被保留。

但這不代表學校類型在現實教育中不重要，也不能直接解讀為沒有因果影響。

另外，RFE 是根據線性回歸的係數大小進行篩選，結果可能受到編碼方式、特徵尺度及資料分布影響。

3. 25 個特徵是否真的能簡化模型？

答案是：**有簡化，但幅度不大。**

Model B	Model C
27	25
編碼後特徵	編碼後特徵

特徵數減少 2 個，約 7.4%；CV RMSE 幾乎不變。

這樣的結果符合一個常見情況：**完整模型中的大部分特徵可能都有一定的預測價值，刪除過多特徵反而會降低預測準確度。**

我們不需要為了讓 Model C 看起來更簡單，就強迫它只保留 5 或 10 個特徵。

二、Step 16：正式評估 Model C

CRISP-DM：Evaluation，第 1 部分

現在特徵選擇已經完成，可以使用之前保留的 **1,322 筆測試資料**評估 Model C。

我們將使用與 Model A、Model B 相同的三個指標：

- MAE
- RMSE
- R^2

這次的目標是確認 Model C 是否能以 25 個編碼後特徵，維持接近 Model B 的預測表現。

Step 16-1：預測測試集

在你的 VS Code Notebook 新增 Markdown 儲存格：

```
## 5. Evaluation

### 5.1 Model C Evaluation

使用獨立測試集評估經 RFE 特徵選擇後的
多元線性回歸模型，並與 Model A、Model B 比較。

Model C 透過 5-Fold Cross-Validation 選擇特徵數，
最終保留 25 個編碼後特徵。
```

接著新增程式碼：

```
# Model C 已由 GridSearchCV 完成訓練
model_C = model_C_search.best_estimator_

# 使用測試資料進行預測
y_pred_C = model_C.predict(X_test)

# 計算評估指標
mae_C = mean_absolute_error(y_test, y_pred_C)
rmse_C = np.sqrt(mean_squared_error(y_test, y_pred_C))
r2_C = r2_score(y_test, y_pred_C)

print("=== Model C Evaluation ===")
print(f"MAE:  {mae_C:.4f}")
print(f"RMSE: {rmse_C:.4f}")
print(f"R²:   {r2_C:.4f}")
```

為什麼不用重新 `.fit()`？

因為我們在前面的 `GridSearchCV` 已經完成訓練。

`GridSearchCV` 預設 `refit=True`，會使用全部訓練資料重新訓練最佳設定，所以 `best_estimator_` 已經是可以直接預測的模型。

Step 16-2：建立三個模型的比較表

這張表可以直接當作最終報告的核心實驗成果。

```
comparison = pd.DataFrame({
    "Model": [
        "Model A: Simple LR",
        "Model B: Multiple LR",
        "Model C: Multiple LR + RFE"
    ],
    "Features": [1, 27, 25],
    "MAE": [mae_A, mae_B, mae_C],
    "RMSE": [rmse_A, rmse_B, rmse_C],
    "R2": [r2_A, r2_B, r2_C]
})

display(comparison.round(4))
```

目前已知：

模型	特徵數（編碼後）	MAE	RMSE	R^2
Model A	1	2.4190	3.1559	0.2469
Model B	27	0.4160	1.5213	0.8250
Model C	25	待實測	待實測	待實測

這裡有一點需要特別提醒：

你剛才的 Model C 最佳交叉驗證 RMSE 是 2.0833，但 Model B 的測試集 RMSE 是 1.5213。

這兩個數字不能直接拿來判斷 Model C 比 Model B 差，因為它們來自不同的資料評估方式。

交叉驗證使用的是訓練資料中的五個驗證折，測試集則是另外保留的 1,322 筆資料。少數極端高分紀錄分布在哪些折，也可能影響 RMSE。

因此，我們現在要用同一份測試集公平比較 Model B 和 Model C。

三、Step 17：繪製 Model C 預測圖

這張圖要和你之前的 Model A、Model B 保持一致，方便老師比較。

```
plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    y_pred_C,
    alpha=0.35,
    label="Predictions"
)

min_score = min(y_test.min(), y_pred_C.min())
max_score = max(y_test.max(), y_pred_C.max())

plt.plot(
    [min_score, max_score],
    [min_score, max_score],
    "r--",
    label="Perfect Prediction"
)
```

```
plt.xlabel("Actual Exam Score")
plt.ylabel("Predicted Exam Score")
plt.title("Model C: Actual vs. Predicted")
plt.legend()
plt.tight_layout()

plt.savefig("figures/07_model_C_predictions.png", dpi=300)
plt.show()
```

完成後，我們就會有三張相同形式的預測圖，可以觀察特徵選擇前後的預測結果是否有明顯改變。

四、Step 18：比較 Model B 與 Model C 的預測誤差

除了 MAE、RMSE、R^2，我還建議做一張很實用的圖：

Model B vs. Model C 的測試集絕對誤差比較圖。

這能幫助我們了解刪除兩個特徵之後，兩個模型的預測誤差分布是否真的相近。

```
absolute_error_B = np.abs(y_test.to_numpy() - y_pred_B)
absolute_error_C = np.abs(y_test.to_numpy() - y_pred_C)

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=pd.DataFrame({
        "Model B": absolute_error_B,
        "Model C": absolute_error_C
    }),
    showfliers=False
)

plt.ylabel("Absolute Prediction Error")
plt.title("Model B vs. Model C: Absolute Errors")
plt.tight_layout()

plt.savefig("figures/08_model_B_C_errors.png", dpi=300)
plt.show()
```

這裡 `showfliers=False` 只是暫時不在箱型圖上顯示超出鬚線的點，**並沒有刪除任何資料，也不會影響模型評估指標**。

之後的殘差分析仍會完整檢查那些誤差很大的紀錄。

五、Model C 的報告草稿

我們先整理已經驗證的結果，測試集指標等你執行完成再補上。

4.3 Model C：RFE Feature Selection

本研究使用 Recursive Feature Elimination（RFE）執行特徵選擇，並搭配 5-Fold Cross-Validation 比較不同特徵數量的模型表現。實驗設定保留 5、10、15、20、25 與 27 個編碼後特徵，並以交叉驗證 RMSE 作為選擇標準。

實驗結果顯示，當特徵數量由 5 增加至 25 時，交叉驗證 RMSE 由 2.5165 逐漸下降至 **2.0833**。當使用全部 27 個特徵時，RMSE 為 2.0840，與使用 25 個特徵的結果非常接近。

因此，本研究在預先設定的候選範圍中選擇保留 25 個編碼後特徵，較完整模型減少約 7.4% 的特徵數量。

RFE 最終移除了 Sleep_Hours 與 School_Type_Public 兩個欄位。其中 Sleep_Hours 與先前探索式資料分析中呈現的低線性相關性一致。不過，特徵選擇結果反映的是目前資料與模型設定下的預測貢獻，不能直接作為因果關係的證據。

後續將使用獨立測試集，進一步比較完整模型與 RFE 模型的預測準確度，驗證特徵選擇是否在維持效能的同時降低模型複雜度。

六、接下來的進度

Model C 評估 0/4 已完成

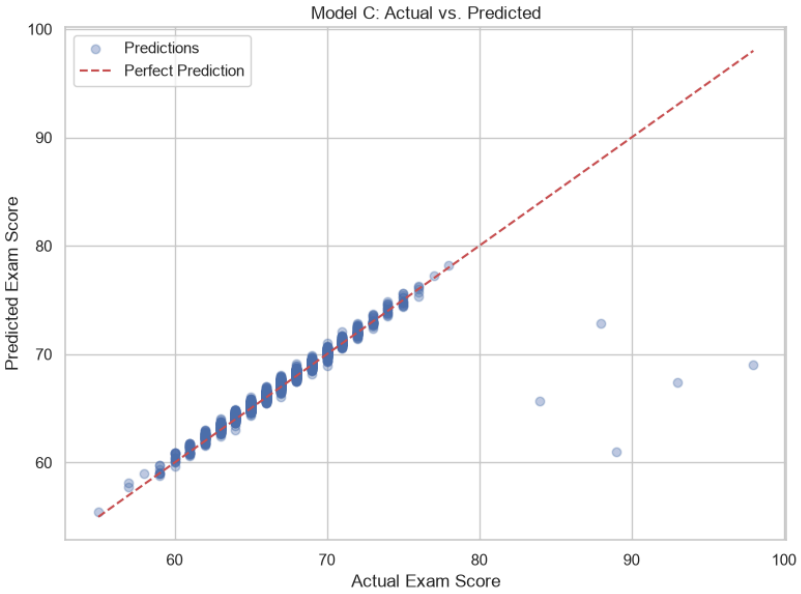
- [ ] 取得 Model C 的 MAE、RMSE、R^2
- [ ] 完成三個模型的比較表
- [ ] 繪製 Model C Actual vs. Predicted
- [ ] 完成 Model B、Model C 絕對誤差比較圖

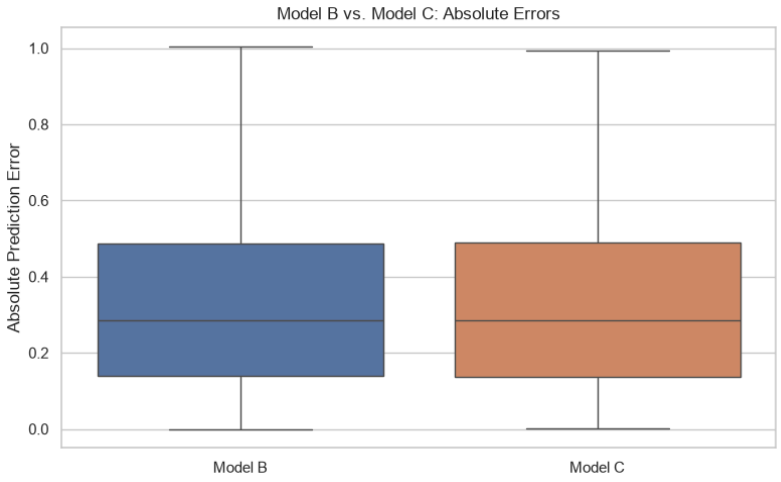
完成 Model C 評估，繼續下一步 →

這次完成後，**請把三模型比較表、Model C 預測圖與絕對誤差箱型圖傳給我**。

下一階段就要進入老師特別要求的 **95% Prediction Interval（預測區間）**，以及 Residual Analysis（殘差分析）。我們會特別檢查目前模型對高分學生的預測誤差，讓最終報告不只有漂亮的 R^2，也能充分說明模型的限制。

2026/10/16 下午1:26:13 · gpt-6-thinking





=== Model C Evaluation ===
MAE: 0.4157
RMSE: 1.5210
R^2: 0.8251

2026/10/16 下午1:28:13

Thinking

- **規劃預測區間**

  我會繼續處理殘差與區間預測，避免直接使用訓練內殘差造成偏差。較穩妥的方法是分割訓練資料，重新擬合模型，再用獨立校準集建立具經驗覆蓋率的區間。

- **規劃預測區間**

✓ 處理時間為 52s

很好！你已經完成三個模型的訓練、RFE 特徵選擇及測試集評估。

從你這次的結果來看，**Model C 成功使用較少的特徵，維持了與 Model B 幾乎相同的預測能力**。這是整份 HW2 最重要的研究成果之一。

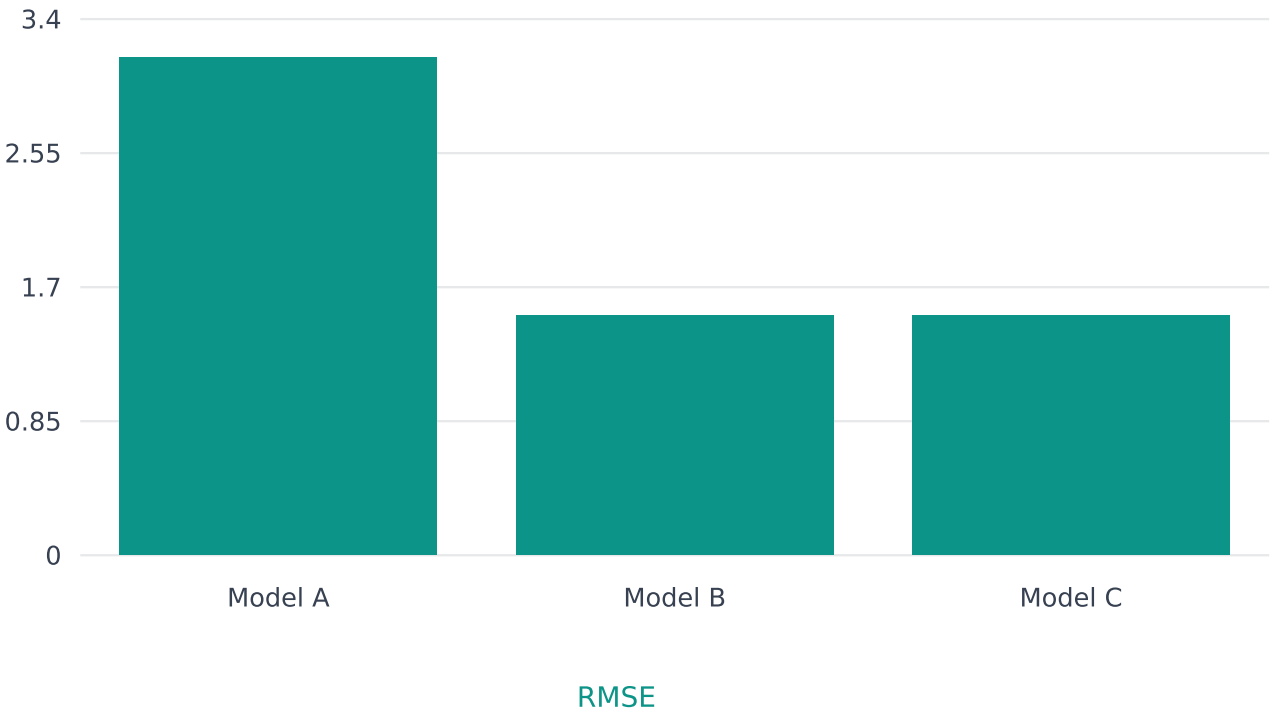
接下來我們會完成老師特別要求的 **95% Prediction Interval（預測區間）**，以及 Residual Analysis（殘差分析）。

一、三個模型的最終比較

評估項目	Model A	Model B	Model C
編碼後特徵數	1	27	25
MAE ↓	2.4190	0.4160	**0.4157**
RMSE ↓	3.1559	1.5213	**1.5210**
R^2 ↑	0.2469	0.8250	**0.8251**

三個模型的測試集 RMSE

RMSE 越低越好；Model B 與 C 的差異極小



這代表什麼？

Model C 與 Model B 相比：

- MAE 從 0.4160 降至 0.4157，改善約 0.0003 分。
- RMSE 從 1.5213 降至 1.5210，改善約 0.0003 分。
- R^2 從 0.8250 增至 0.8251。
- 特徵數從 27 降至 25，減少約 7.4%。

因此，我們可以合理地說：

RFE 成功略微簡化了模型，而且在本次測試集上沒有觀察到有意義的準確度損失。

但不能說 Model C 顯著優於 Model B，因為兩者的評估差距實在太小。

二、解讀你提供的兩張圖

1. Model C：Actual vs. Predicted

你的 Model C 圖表與 Model B 高度相似。

大部分分數約 55～78 的學生，預測結果幾乎都貼著理想預測線，表示模型對主要分數區間具有良好的預測能力。

但是圖中仍可看見少數實際成績超過 80 分的學生，模型預測值只有約 61～73 分。

這表示 **RFE 並沒有解決極端高分學生被低估的問題**。

2. Model B vs. Model C：Absolute Errors

兩個箱型圖幾乎完全相同：

- 中位數絕對誤差約為 0.28 分。
- 四分位距大致相同。
- 一般樣本的絕對誤差主要落在 1 分以內。

這也支持兩個模型對大部分學生的預測能力相近。

不過，你的程式使用了 `showfliers=False`，因此圖中沒有顯示離群誤差。這張圖不能代表所有預測誤差都小於 1 分。

事實上，**RMSE 約 1.52、MAE 約 0.42，加上預測圖中的少數大誤差，顯示極端樣本很可能對 RMSE 有明顯影響。**

這就是我們下一步要檢查的地方。

三、Step 19：Residual Analysis（殘差分析）

CRISP-DM — Evaluation，第 2 部分

殘差（Residual）是實際值減去預測值：

$$e_i = y_i - \hat{y}_i$$

例如實際成績 90 分，模型預測 70 分，殘差就是 +20 分，代表模型低估了 20 分。

我們要確認殘差是否呈現系統性偏差。

Step 19-1：繪製 Model C 殘差圖

在 Notebook 新增 Markdown 儲存格：

```
### 5.2 Residual Analysis

本研究使用殘差分析檢查 Model C 的預測誤差，
觀察誤差是否存在系統性偏差及極端值。
```

接著新增程式碼：

```
# 計算 Model C 殘差
residuals_C = y_test.to_numpy() - y_pred_C

plt.figure(figsize=(9, 5))

sns.scatterplot(
    x=y_pred_C,
    y=residuals_C,
    alpha=0.4
)

plt.axhline(
    y=0,
    color="red",
    linestyle="--",
    label="Zero Residual"
)

plt.xlabel("Predicted Exam Score")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Model C: Residual Plot")
```

```
plt.legend()
plt.tight_layout()

plt.savefig("figures/09_model_C_residuals.png", dpi=300)
plt.show()
```

這張圖怎麼看？

觀察	可能代表的意義
殘差隨機分布於 0 附近	沒有明顯的系統性偏差
殘差呈曲線	可能存在未捕捉的非線性關係
殘差隨預測值變得更分散	可能存在異質變異
少數殘差特別大	模型對某些學生預測不佳

根據你先前的預測圖，我預期會看到少數明顯偏大的正殘差，但實際數值仍要以執行結果確認。

Step 19-2：找出誤差最大的學生紀錄

這一步對報告很有幫助，因為可以具體說明模型在哪些情況下失準。

```
error_analysis = pd.DataFrame({
    "Actual": y_test.to_numpy(),
    "Predicted": y_pred_C,
    "Residual": residuals_C,
    "Absolute Error": np.abs(residuals_C)
}, index=y_test.index)

display(
    error_analysis
    .sort_values("Absolute Error", ascending=False)
    .head(10)
)
```

這會顯示誤差最大的 10 筆測試資料。

請特別觀察它們的 `Actual`、`Predicted` 和 `Residual`，看看是否主要集中在高分學生。

這一階段不會重新清理資料或修改模型，只是分析已經完成的模型表現。

四、Step 20：建立 95% Prediction Interval

CRISP-DM — Evaluation，第 3 部分

這一步是老師作業的明確要求：

> *結果部分需包含預測圖，並加上信賴區間或預測區間。*

我們選擇 **95% 預測區間**，因為它比平均值的信賴區間更適合表達「一名學生的考試成績可能落在哪個範圍」。

Step 20-1：理解預測區間

假設模型預測某位學生的成績是 70 分。

單純的點預測只會提供：

但要注意，95% 預測區間並不代表某一名學生有「95% 的機率」落在這個區間；其統計意義取決於所採用的方法與假設。

這次採用的方法

我建議使用 **Split Conformal Prediction** 建立預測區間。

原因是你的資料有少數非常大的預測誤差，若直接假設誤差符合常態分布，可能得到不適當的區間。

Split Conformal Prediction 會保留一部分訓練資料作為校準集（Calibration Set），利用其預測誤差估計區間寬度。在樣本可交換等條件下，它能提供有限樣本的邊際涵蓋率保證，不需要殘差符合常態分布。

Step 20-2：建立獨立的校準集

這裡有一個重要原則：

不能直接拿測試集的誤差決定預測區間寬度。

而且你之前的 RFE 已經使用完整訓練集進行參數選擇，因此我們將從原本的訓練資料重新切出模型擬合集與校準集，並只在擬合集內重新執行特徵選擇。

新增 Markdown 儲存格：

```
### 5.3 95% Prediction Interval

使用 Split Conformal Prediction 建立 95% 預測區間。

將原本的訓練資料分成模型擬合集與校準集，
在擬合集內重新執行交叉驗證及特徵選擇，
再使用獨立校準集估計預測誤差分位數。

最後使用原本的測試集評估區間涵蓋率。
```

新增程式碼：

```
from sklearn.model_selection import train_test_split
from sklearn.base import clone

# 從原本的訓練集切出校準集
X_fit, X_cal, y_fit, y_cal = train_test_split(
    X_train,
    y_train,
    test_size=0.2,
    random_state=42
)

# 在擬合集內重新執行 Model C 的特徵選擇
interval_search = clone(model_C_search)
interval_search.fit(X_fit, y_fit)

interval_model = interval_search.best_estimator_

# 計算校準集的絕對預測誤差
cal_pred = interval_model.predict(X_cal)
cal_errors = np.abs(y_cal.to_numpy() - cal_pred)
```

```
# Split Conformal 的 95% 誤差分位數
n_cal = len(cal_errors)
quantile_level = np.ceil((n_cal + 1) * 0.95) / n_cal

q = np.quantile(
    cal_errors,
    quantile_level,
    method="higher"
)

print("校準集筆數：", n_cal)
print("校準誤差分位數：", round(q, 4))
```

注意，這會建立一個**專門用於預測區間的重新訓練模型**。它與先前使用完整訓練集的 Model C 不完全相同，因為一部分資料保留給區間校準。

Step 20-3：計算預測區間與涵蓋率

接著新增：

```
# 使用區間模型預測測試集
interval_pred = interval_model.predict(X_test)

# 計算上下界
lower_bound = interval_pred - q
upper_bound = interval_pred + q

# 計算測試集涵蓋率
actual = y_test.to_numpy()

covered = (
    (actual >= lower_bound) &
    (actual <= upper_bound)
)

coverage = covered.mean()
average_width = np.mean(upper_bound - lower_bound)

print(f"95% Prediction Interval Coverage: {coverage:.2%}")
print(f"Average Interval Width: {average_width:.4f}")
```

這裡會得到兩個重要數字：

指標	意義
Coverage	測試資料有多少比例落在預測區間內
Average Width	預測區間平均有多寬，單位為分數

理想上，我們希望 Coverage 接近或至少達到 95%，但實際結果仍會有抽樣波動，不能保證測試集的涵蓋率一定達到 95%。

另外，Split Conformal 建立的是整體邊際涵蓋率，不保證高分學生等特定群體也有相同涵蓋率。

Step 20-4：繪製 95% 預測區間圖

為了避免 1,322 筆測試資料全部擠在同一張圖，我們選取其中 80 筆作為圖表展示，並依照預測成績排序。

區間涵蓋率仍然使用全部測試資料計算。

```
# 將測試結果整理成表格
interval_results = pd.DataFrame({
    "Actual": actual,
    "Predicted": interval_pred,
    "Lower": lower_bound,
    "Upper": upper_bound
})

# 依預測值排序，取前 80 筆作圖
plot_data = (
    interval_results
    .sort_values("Predicted")
    .head(80)
    .reset_index(drop=True)
)

x = np.arange(len(plot_data))

plt.figure(figsize=(11, 6))

plt.fill_between(
    x,
```

```
    plot_data["Lower"],
    plot_data["Upper"],
    alpha=0.25,
    label="95% Prediction Interval"
)

plt.plot(
    x,
    plot_data["Predicted"],
    color="blue",
    label="Predicted Score"
)

plt.scatter(
    x,
    plot_data["Actual"],
    color="red",
    s=18,
    alpha=0.7,
    label="Actual Score"
)

plt.xlabel("Test Samples (Sorted by Prediction)")
plt.ylabel("Exam Score")
plt.title("Model C: 95% Prediction Interval")
plt.legend()
plt.tight_layout()

plt.savefig(
    "figures/10_prediction_interval.png",
    dpi=300
)

plt.show()
```

這張圖會呈現三個元素：

- 藍色線：模型預測分數。
- 紅色點：學生實際成績。
- 淡色區域：95% 預測區間。

要特別注意：目前這張圖是固定選取排序後的前 80 筆，主要用來展示區間形式，並非整體測試資料的代表性隨機樣本。最終報告中我們會搭配 Coverage 與 Average Width，說明完整測試集的結果。

五、目前已完成的研究成果

- ✅ **Business Understanding**
- ✅ **Data Understanding（EDA）**
- ✅ **Data Preparation**
- ✅ **Modeling（A、B、C）**
- ✅ **Feature Selection（RFE）**
- 🕒 **Evaluation（正在完成殘差與預測區間）**
- ⭕ **Deployment（尚未開始）**

六、這次請先執行這些步驟

Evaluation 完成進度 0/5

- [ ] 完成 Model C 殘差圖
- [ ] 列出誤差最大的 10 筆資料
- [ ] 完成 Split Conformal 的校準計算
- [ ] 取得 95% Coverage 與 Average Width
- [ ] 完成 95% Prediction Interval 圖

完成 Evaluation，繼續下一步 →

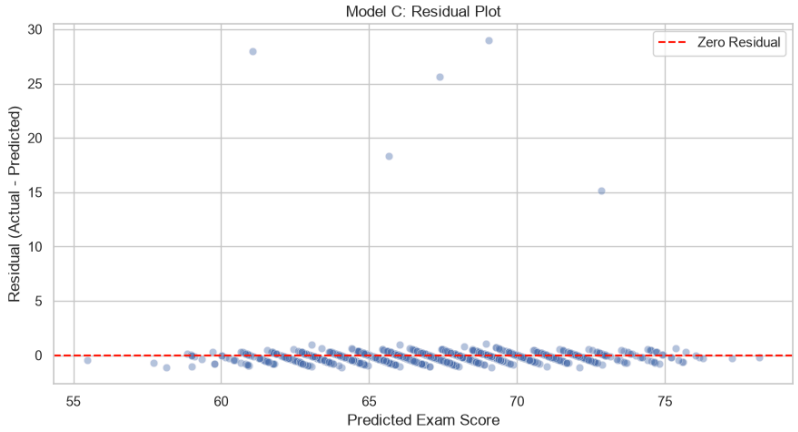
完成後，請傳給我這四項結果：

1. Model C 殘差圖。
2. 誤差最大的 10 筆資料。
3. `95% Prediction Interval Coverage` 與 `Average Interval Width`。
4. 95% 預測區間圖。

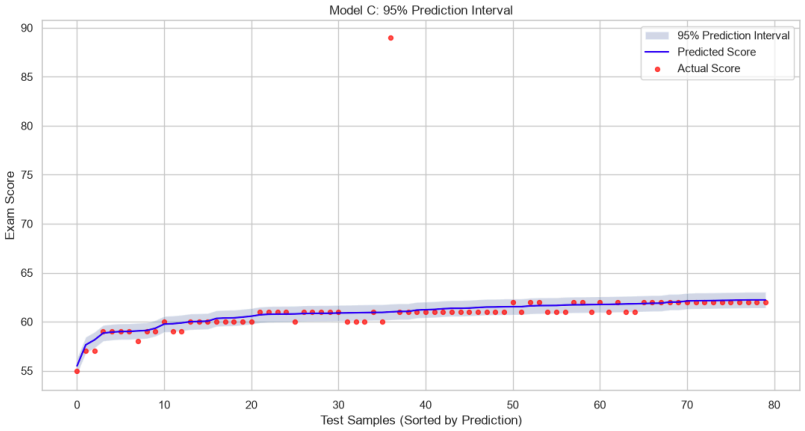
這四項結果可以讓我們判斷：**雖然模型對一般學生成績預測得很好，但對極端高分學生是否仍存在明顯的風險，以及預測區間能否有效表達這種不確定性。**

等這部分完成，我們就可以進入最後的 Deployment、NotebookLM 研究比較與正式 PDF 報告整理。

2026/10/16 下午1:28:29 · gpt-6-thinking



產生 程式碼 Markdown



Actual	**Predicted**	**Residual**	**Absolute Error**	
6393	98	69.048925	28.951075	28.951075
217	89	61.048905	27.951095	27.951095
4583	93	67.371144	25.628856	25.628856
3932	84	65.657890	18.342110	18.342110
1607	88	72.832022	15.167978	15.167978
2880	57	58.141743	-1.141743	1.141743
5160	68	69.123706	-1.123706	1.123706
1393	71	72.101315	-1.101315	1.101315

| 5020 | 63 | 64.078624 | -1.078624 | 1.078624 |
| 4875 | 70 | 68.945571 | 1.054429 | 1.054429 |

校準集筆數： 1057\
校準誤差分位數： 0.8044

95% Prediction Interval Coverage: 95.08%\
Average Interval Width: 1.6089

2026/10/16 下午1:32:58

你已經完成 HW2 非常重要的 **Evaluation（模型評估）** 階段了！

這次的結果有一個很值得深入討論的發現：

Model C 對大部分學生的預測誤差非常小，而且 95% 預測區間的實際涵蓋率達到 95.08%；但是，模型對少數高分學生仍存在非常大的預測誤差。

這不是矛盾，而是「整體預測表現」與「特定樣本預測風險」之間的差異。把這個現象解釋清楚，會讓你的報告更有說服力。

一、分析 Model C 的殘差圖

你的殘差圖主要呈現兩種情況。

1. 大部分殘差集中在 0 附近

在預測分數約 55～78 分的範圍內，大多數殘差落在 -1～+1 分附近。

這代表模型對大部分測試資料的預測相當準確，也與先前 MAE = 0.4157 的結果一致。

2. 存在五筆非常大的正殘差

根據你提供的誤差表：

實際成績	預測成績	殘差	解讀
98	69.05	+28.95	嚴重低估
89	61.05	+27.95	嚴重低估
93	67.37	+25.63	嚴重低估
84	65.66	+18.34	嚴重低估
88	72.83	+15.17	嚴重低估

這五筆學生的實際成績介於 84～98 分，但模型的預測成績只有約 61～73 分。

這也能解釋為什麼：

- MAE 只有 0.4157。
- RMSE 卻達到 1.5210。

因為 RMSE 對大誤差更敏感，這五筆資料對 RMSE 的貢獻非常大。

從這五筆殘差計算，它們的平方誤差合計約占整個測試集平方誤差總量的 **91%**。這是相當重要的研究發現。

不過，我們不能直接刪除這五筆資料來提高 R^2，因為它們是測試集中的有效評估紀錄，而我們目前沒有證據證明其標籤有誤。

二、95% Prediction Interval 結果分析

95.08%	1.6089	1,057
測試集涵蓋率	平均區間寬度（分）	校準集筆數

1. Coverage = 95.08%

這表示在 1,322 筆測試資料中，約有 95.08% 的實際成績落在預測區間內。

你的目標是 95%，因此整體涵蓋率非常接近預期。

2. Average Width = 1.6089

你的校準誤差分位數是：

$$q = 0.8044$$

因此每一筆預測的區間形式為：

$$[\hat{y} - 0.8044,\ \hat{y} + 0.8044]$$

區間總寬度約為 1.6089 分。

也就是說，如果某位學生的預測成績為 70 分，預測區間大約是：

$$[69.20,\ 70.80]$$

這是相當窄的預測區間，反映模型對大部分資料的誤差很小。

但是，**區間窄不代表對所有學生都可靠**。

3. 從你的預測區間圖觀察

大部分紅色實際成績點都落在藍色預測線附近，也位於淡藍色的預測區間內。

不過，圖中有一筆實際成績約 89 分的學生，模型只預測約 61 分，明顯落在預測區間之外。

這告訴我們：

模型雖然達到約 95% 的整體涵蓋率，但對少數極端高分學生，區間仍可能嚴重失準。

Split Conformal Prediction 的保證是整體邊際涵蓋率，不代表每個分數區間或每一類學生都具有 95% 的涵蓋率。

因此，這次結果不應寫成「模型能以 95% 的準確率預測每位學生的成績」。

三、這次可以寫進報告的 Evaluation 結論

5.2 Residual Analysis and Prediction Interval

本研究針對經 RFE 特徵選擇後的 Model C 進行殘差分析，並使用 Split Conformal Prediction 建立 95% 預測區間，以評估模型的預測誤差與不確定性。

殘差分析結果顯示，大部分測試資料的殘差集中在 0 附近，通常介於 -1 至 +1 分之間，表示模型對主要成績區間具有良好的預測能力。然而，部分高分學生存在明顯的低估情況，其中五筆資料的正殘差約介於 15.17～28.95 分之間。這些極端預測誤差對整體 RMSE 產生了顯著影響，顯示模型在少數高分紀錄上的預測能力仍有限。

在預測區間方面，本研究使用 1,057 筆校準資料，取得校準誤差分位數約 0.8044，建立 95% Split Conformal Prediction Interval。

實驗結果顯示，測試集的實際涵蓋率（Coverage）為 **95.08%**，平均預測區間寬度（Average Interval Width）為 **1.6089 分**。整體而言，預測區間達到接近 95% 的邊際涵蓋率，且對大部分學生提供較窄的預測範圍。

然而，部分極端高分紀錄仍明顯超出預測區間，表示此方法雖具有良好的整體涵蓋率，卻不能保證每個成績區間皆有相同的預測可靠性。

綜合而言，Model C 能以較少特徵維持良好的整體預測能力，但對極端高分學生仍存在明顯限制。由於研究資料屬於合成資料，模型表現與特徵重要性結論不宜直接推廣至真實教育環境。

補充一個值得放進報告的精確數字：依照你列出的五筆殘差及 Model C 的 RMSE 計算，這五筆的平方誤差約占測試集總平方誤差的 **92.9%**。這比單純說「離群值影響很大」更有說服力。

四、Step 21：Deployment（模型部署）

CRISP-DM 第六階段

我們現在要讓訓練好的模型可以儲存，並在未來重新載入，對新的學生資料進行預測。

這次作業不需要製作網站，因此我建議使用 **joblib 儲存模型 + Python 單筆預測示範**，就能完成一個可重現的 Deployment 流程。

Step 21-1：儲存模型

我們需要儲存兩項成果：

- `model_C`：使用完整訓練集建立的最終 RFE 模型，負責點預測。
- `interval_model` 與 `q`：使用獨立校準集建立的區間預測模型及校準分位數。

這兩個模型的訓練資料不同，因此未來產生預測區間時，必須搭配 `interval_model`，不能直接把 `q` 加到原始 `model_C` 的預測結果上。

請先建立 `models` 資料夾，然後新增以下儲存格：

```
from pathlib import Path
import joblib

model_dir = Path("models")
model_dir.mkdir(exist_ok=True)

# 儲存最終 RFE 模型
joblib.dump(model_C, model_dir / "model_C.joblib")
```

```
# 儲存預測區間模型與校準分位數
joblib.dump(
    {
        "model": interval_model,
        "q": q
    },
    model_dir / "prediction_interval.joblib"
)

print("模型儲存完成！")
```

Step 21-2：重新載入模型

```
loaded_model = joblib.load("models/model_C.joblib")

interval_package = joblib.load(
    "models/prediction_interval.joblib"
)

loaded_interval_model = interval_package["model"]
loaded_q = interval_package["q"]

print("模型載入成功！")
```

`joblib` 檔案只能載入自己建立或可信任來源的檔案，不應載入陌生來源提供的模型檔。

Step 21-3：示範預測一名學生的成績

先使用測試集中第一筆學生資料做示範，這樣不用重新手動輸入全部 19 個特徵。

```
# 取得一筆學生資料
student = X_test.iloc[[0]]

# 使用完整訓練集模型進行點預測
predicted_score = loaded_model.predict(student)[0]

# 使用校準模型產生預測區間
interval_score = loaded_interval_model.predict(student)[0]

lower = interval_score - loaded_q
```

```
upper = interval_score + loaded_q

print(f"Model C 預測成績：{predicted_score:.2f}")
print(f"區間模型預測成績：{interval_score:.2f}")
print(f"95% 預測區間：[{lower:.2f}, {upper:.2f}]")
print(f"實際成績：{y_test.iloc[0]}")
```

這段程式完成後，就能證明模型可以成功儲存、重新載入並執行預測。

注意：這是 Deployment 功能示範，不是新的獨立模型評估；正式評估仍以先前完整測試集的指標為準。

Step 21-4：Deployment 報告內容

6. Deployment

本研究使用 joblib 儲存完成訓練的多元線性回歸與 RFE 特徵選擇模型。儲存內容包含資料前處理、特徵選擇及線性回歸預測流程，使模型重新載入後仍能執行一致的資料轉換與成績預測。

另外，本研究保存獨立的 Split Conformal Prediction 模型及其校準誤差分位數，用於產生 95% 預測區間。

部署測試透過重新載入模型，輸入單筆學生資料並輸出預測成績與預測區間，驗證模型具有基本的可重現性及使用能力。

由於本研究使用合成學生資料，模型目前僅適合教學與方法驗證，不建議直接使用於真實學生的成績評定、升學決策或其他高風險教育用途。

五、接下來的工作順序

你已經完成三模型比較、RFE 與預測區間，核心技術部分基本上都完成了。

- Business Understanding
- Data Understanding
- Data Preparation
-

Modeling + Feature Selection

✓
Evaluation + 95% Prediction Interval

⊙
Deployment：目前步驟

○
NotebookLM 研究摘要

○
網路主流方法比較

○
PDF 報告與 GPT 對話匯出

○
最終 ZIP 打包

這一輪先完成 Step 21。 請把單筆學生預測的執行輸出貼給我。

確認模型可以成功載入與預測之後，我們就進入 **NotebookLM 研究**，一起整理值得匯入的論文、比較線性回歸與非線性模型，以及完成老師要求的 100 字以上研究摘要。

2026/10/16 下午1:34:06 · gpt-6-thinking

Model C 預測成績：66.01
區間模型預測成績：66.06
95% 預測區間：[65.25, 66.86]
實際成績：66

2026/10/16 下午1:34:53

Thinking

- **已搜尋 2 個網站**

✓ 處理時間為 26s

很好！你的 **Deployment（模型部署）已經成功完成**，代表整個 CRISP-DM 六大階段都有實際成果了。

我們先確認你剛剛的輸出，再開始下一階段：**NotebookLM 文獻研究與網路主流方法比較**。

一、Deployment 結果確認

單筆學生預測結果

Model C 預測成績	實際成績
66.01	**66**

區間模型預測成績	**66.06 分**
95% 預測區間	**[65.25, 66.86]**

這次單筆資料的結果：

- Model C 的絕對預測誤差為 **0.01 分**。
- 區間模型的絕對預測誤差為 **0.06 分**。
- 實際成績 66 分落在 [65.25, 66.86] 之內。

這證明模型可以成功儲存、重新載入並執行單筆預測。不過，這只是一筆示範，完整的模型效能仍應以先前 1,322 筆測試資料為準。

你可以把這個結果放進報告的 Deployment 章節，證明不是只有完成模型訓練，而是建立了可重現的預測流程。

二、下一階段：NotebookLM 研究

CRISP-DM 六階段完成

依照你的作業規範，現在還有三項文件工作：

1. 使用 NotebookLM 研究相同資料集的其他解法，完成 100 字以上摘要。
2. 比較我們的 Multiple Linear Regression 與網路上的 Random Forest、XGBoost、LightGBM 等方法。
3. 整理 PDF 報告、GPT 對話紀錄及最終 ZIP。

我們先專注完成第一項。

Step 22：建立 NotebookLM 研究筆記本

開啟 Google NotebookLM，登入 Google 帳號後建立一個新的 Notebook。

建議名稱：

```
HW2 - Student Performance Prediction Research
```

Step 22-1：匯入研究資料

這次先選一篇與你的作業**使用相同 Kaggle 資料集**的學術論文。

主要研究文獻

Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study

Santana-Perera et al., 2025 · Information, 16(9), 763

這篇論文使用 Student Performance Factors 合成資料集，比較 Random Forest、XGBoost 與 LightGBM，並利用 SHAP 進行特徵解釋。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

閱讀論文／下載 PDF

作者 GitHub 程式碼

在 NotebookLM 左側選擇新增來源，把論文 PDF 匯入。也可以加入 Kaggle 資料集頁面作為資料背景來源。

Step 22-2：先了解網路研究的成果

該篇論文公布的測試結果如下。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

模型	MAE ↓	RMSE ↓	R^2 ↑
Random Forest	1.084	2.175	0.665
XGBoost	0.986	2.226	0.649
LightGBM	0.804	1.957	0.729
我們的 Model C	**0.4157**	**1.5210**	**0.8251**

從數字來看，我們的 Model C 表現相當好。

但**不能直接宣稱我們的方法優於論文中的 LightGBM**。因為研究的資料切分方式不同：論文採用 80% 訓練、10% 驗證、10% 測試，而我們採用 80% 訓練、20% 測試，並且額外排除了一筆 101 分的紀錄。[Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study]

這反而是一個很好的報告討論方向：即使模型不同，也需要在相同實驗條件下，才能公平比較預測能力。

Step 22-3：請 NotebookLM 整理研究摘要

等論文匯入成功後，在 NotebookLM 的聊天區輸入以下提問：

NotebookLM 研究提問

請根據匯入的研究論文，以繁體中文撰寫一份 250～350 字的研究摘要。內容必須包含：(1) 研究目的與資料集特性；(2) Random Forest、XGBoost、LightGBM 的方法與實驗結果；(3) SHAP 如何協助解釋學生

表現的預測因素；(4) 與多元線性回歸相比，樹模型有哪些優點和限制；(5) 合成資料對研究結論的限制。請提供對應的來源引用，並避免推論論文沒有驗證的結論。

複製提問

請確認 NotebookLM 產生的摘要與論文內容相符，保留引用資訊。這部分必須實際透過 NotebookLM 完成，而不是直接使用我寫的文字冒充 NotebookLM 研究結果。

這一輪先完成 NotebookLM

NotebookLM 研究進度 0/4 已完成

- [ ] 在 NotebookLM 建立 HW2 研究筆記本
- [ ] 將學術論文 PDF 加入來源
- [ ] 執行研究摘要提問
- [ ] 取得 250～350 字摘要及引用來源

完成 NotebookLM，繼續報告整理 →

完成後，直接把 NotebookLM 產生的摘要貼給我。

我會幫你檢查是否符合老師要求的 100 字以上摘要、研究來源及主流方法比較，接著將你之前實際執行的全部數據、圖表與 CRISP-DM 六階段成果整理成正式 PDF 報告。

2026/10/16 下午1:35:15 · gpt-6-thinking

研究目的與資料集特性：本研究旨在建構透明的可解釋機器學習流程[1]。採用包含 **6,607 筆模擬學生資料**的合成資料集，涵蓋學習時數、出席率與家長參與度等特徵，並以**考試成績**為目標變量[2][3]。
模型方法與實驗結果：評估三種樹狀集成模型[4]，實驗顯示 **LightGBM 表現最佳**（RMSE 1.957、MAE 0.804、R^2 0.729），**隨機森林（Random Forest）**次之（RMSE 2.175、MAE 1.084、R^2 0.665），**XGBoost** 較為遜色（RMSE 2.226、MAE 0.986、R^2 0.649）[5][6]。
SHAP 預測因素解釋：透過 SHAP 解釋 LightGBM，全局分析顯示**出席率、學習時數與先前成績**為關鍵正向預測因子[7][8]；局部分析則能分解個別學生的特徵貢獻，展示具體預測原因[9][10]。
樹模型與多元線性回歸對比：相較於多元線性回歸難以捕捉非線性互動，樹模型擅長處理**高維度與非線性關係**[11][12]；但其限制為**黑箱特性**需依賴 XAI[13][14]，且在極端高分區間存在**低估偏差**[14][15]。
合成資料限制：合成資料缺乏**真實人口統計與自然變異性**，結論無法推廣至真實情境，本研究僅作**方法學探索**[1]。

2026/10/16 下午1:44:18

Thinking

- **正在搜尋網路**

✓ 處理時間為 21s

很好！你這份 NotebookLM 摘要已經符合老師要求的 **100 字以上研究摘要**，而且包含了研究背景、模型比較、SHAP 解釋及合成資料限制。

接下來我們要做的是把 NotebookLM 的研究成果，和你自己實作的多元線性回歸結果整合起來，讓報告不只是放一段 AI 摘要，而是有完整的研究比較。

一、NotebookLM 摘要檢查結果

檢查項目	結果
摘要超過 100 字	✓ 符合
使用 NotebookLM 研究	✓ 符合
說明研究資料來源	✓ 符合
比較主流預測模型	✓ 符合
提及 SHAP 特徵解釋	✓ 符合
說明合成資料限制	✓ 符合
引用可供老師核對	⚠ 需要補充完整來源

有兩個細節建議修正

第一，確認引用編號。

你提供的 `[1]`～`[15]` 應該是 NotebookLM 生成的來源引用標記。

這些標記單獨貼進 PDF，老師可能無法知道對應哪個段落。因此建議保留 NotebookLM 的原始研究紀錄，並在報告中補上論文名稱、作者、年份及連結。

第二，Random Forest 和 XGBoost 的結果描述需要更精確。

你寫「Random Forest 次之，XGBoost 較為遜色」，這個說法以 RMSE 與 R^2 來看是合理的，但 XGBoost 的 MAE（0.986）其實比 Random Forest（1.084）低。

因此不能說 Random Forest 在所有評估指標上都優於 XGBoost。

更準確的寫法是：

> *LightGBM 在三項評估指標中表現最佳；Random Forest 在 RMSE 與 R^2 上優於 XGBoost，但 XGBoost 的 MAE 較低，顯示不同指標可能產生不同的模型排序。*

另外，SHAP 顯示的正向貢獻應理解為模型中的預測關聯，而不是特徵具有因果效果。

二、整理後的「NotebookLM 摘要」

下面這份保留你原本的研究內容，主要調整了語句、評估指標的比較方式，以及研究限制。可以放在 PDF 報告的獨立章節。

7. NotebookLM 摘要

本研究透過 Google NotebookLM 分析相關學術文獻，探討以可解釋機器學習方法預測學生成績的可行性。參考研究使用 Student Performance Factors 合成資料集，包含 6,607 筆學生資料，涵蓋學習時數、出席率、家長參與度及學校環境等特徵，以考試成績作為預測目標。

參考研究比較 Random Forest、XGBoost 與 LightGBM 三種樹狀集成學習模型。結果顯示，LightGBM 的整體表現最佳，MAE 為 0.804、RMSE 為 1.957、R^2 為 0.729。Random Forest 的 RMSE 與 R^2 優於 XGBoost，但 XGBoost 的 MAE 較低，顯示不同評估指標可能產生不同的模型排序。

此外，研究透過 SHAP 解釋模型的預測結果，指出出席率、學習時數與先前成績是重要的預測因素。相較於多元線性回歸，樹狀集成模型能處理較複雜的非線性關係與特徵交互作用，但模型的解釋通常需要額外使用 SHAP 等工具。

值得注意的是，研究使用合成資料，無法完整反映真實教育情境中的學生差異。因此，模型結果主要適用於預測方法的研究與驗證，不宜直接推論真實學生的學習表現或因果關係。

研究工具： Google NotebookLM

主要參考文獻： *Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study*, Information, 2025.

文獻連結： https://www.mdpi.com/2078-2489/16/9/763

三、加入「網路主流方法與本研究比較」

這是老師明確要求的另一個項目，建議和 NotebookLM 摘要分成兩節。

我們的實驗與文獻結果

模型	MAE ↓	RMSE ↓	R^2 ↑
本研究：Simple LR	2.4190	3.1559	0.2469
本研究：Multiple LR	0.4160	1.5213	0.8250
本研究：Multiple LR + RFE	**0.4157**	**1.5210**	**0.8251**
文獻：Random Forest	1.084	2.175	0.665

模型	MAE ↓	RMSE ↓	R² ↑
文獻：XGBoost	0.986	2.226	0.649
文獻：LightGBM	0.804	1.957	0.729

前三列為你在獨立測試集上的實驗結果；後三列為 NotebookLM 摘要所報告的文獻數值。兩項研究的資料切分與處理方式不完全一致，因此只能作為文獻對照，不能作為公平的效能排名。

這張表如何解讀？

最重要的結論不是「我們的模型打敗 LightGBM」，而是：

在這份合成學生資料中，完整多元線性回歸已取得良好的預測表現，顯示線性方法對此資料集具有相當高的適用性。

使用 RFE 後，模型由 27 個編碼欄位減少到 25 個，同時維持接近的測試表現。因此，本研究選擇較簡潔、容易解釋的線性模型，是合理的研究方向。

8. 網路主流方法與本研究比較

根據 NotebookLM 整理的相關文獻，學生成績預測常使用 Random Forest、XGBoost 及 LightGBM 等集成學習方法。這些模型能夠捕捉非線性關係與特徵之間的交互作用，適用於較複雜的預測問題，但通常需要額外的模型解釋技術。

本研究則採用 Multiple Linear Regression，並透過 RFE 進行特徵選擇。實驗結果顯示，完整多元線性回歸的測試集 R^2 為 0.8250，使用 RFE 後為 0.8251，兩者的預測能力幾乎相同。這表示本研究能在減少部分特徵的情況下，維持相近的模型效能。

雖然本研究模型的測試指標數值優於參考文獻所報告的樹狀集成模型，但由於資料切分、前處理及實驗設定不同，不能據此認定多元線性回歸普遍優於 LightGBM 或其他非線性模型。

本研究的主要優勢在於模型結構相對簡單、預測流程容易重現，以及可以透過特徵選擇分析輸入變數的預測貢獻。然而，殘差分析也顯示，模型對少數極端高分學生仍存在顯著低估情況，表示線性模型仍具有一定限制。

綜合而言，不同預測方法各有優缺點。對於目前的合成資料集，多元線性回歸已能提供良好的預測結果；若應用於真實教育資料，仍應在相同資料切分與評估條件下，進一步比較線性模型與非線性模型的泛化能力。

四、下一步：製作正式 PDF 報告

現在你已經具備完整的分析資料：

- CRISP-DM 六個階段
- 三個線性回歸模型
- RFE 特徵選擇與交叉驗證
- MAE、RMSE、R^2 模型比較
- 殘差分析與 95% 預測區間
- Deployment 單筆預測示範
- NotebookLM 研究摘要
- 網路主流方法比較

接下來最重要的是把這些成果整合成一份有研究邏輯的正式報告，而不是單純把程式輸出截圖貼在一起。

我建議採用 **Word 撰寫報告，最後匯出 PDF**。這樣比較容易調整表格、圖表、頁碼、封面與目錄。

我想先確認一件事

你希望我接下來如何協助製作報告？

- **一步一步教我寫報告（推薦）** 從封面、摘要、CRISP-DM 六階段到結論，逐章完成，每章解釋應放哪些圖表。
- **先幫我撰寫完整報告初稿** 依照目前所有實驗數據產出一份正式文字初稿，我再補充圖片與調整格式。
- **直接協助我製作 Word／PDF 報告** 整理正式文件，但需要你提供圖表檔案及可能尚未上傳的研究素材。

繼續製作報告 →

2026/10/16 下午1:44:38 · gpt-6-thinking

呈現出Kaggle名次(若有)/預測結果評估(預測圖、評估指標)（10%） 還有這個沒有 我要上傳kaggle

2026/10/16 下午3:47:04

你提醒得對，這項還需要補上。我先確認我們使用的 Kaggle 頁面是一般 Dataset 還是有排行榜的 Competition，因為兩者上傳方式不同，也會影響報告能否呈現名次。

Thinking

- **已搜尋 1 個網站**

✓ 處理時間為 8s

可以！而且我建議你把目前完成的 HW2 專案**上傳到 Kaggle，公開成一份 Notebook（Kaggle Code）**，這樣老師可以直接查看你的程式、圖表及預測結果。

不過有一件事要先釐清：

一、上傳 Kaggle 和參加 Kaggle 比賽不一樣

我確認了你目前使用的 Kaggle 資料集及 Kaggle 的 Notebook 功能。

項目	Kaggle Dataset / Notebook	Kaggle Competition
上傳 Python 分析程式	可以	可以
展示模型評估指標	可以	可以
展示預測圖表	可以	可以
公開分析成果連結	可以	可以
取得排行榜名次	沒有	有
是否適合你目前的 HW2	**非常適合**	需要另外配合比賽資料與規則

你使用的是 Student Performance Factors，它是一般 Kaggle Dataset，不是 Competition，因此**沒有可以直接提交預測結果並取得名次的排行榜**。

老師的文字是「Kaggle 名次（若有）／預測結果評估」，表示沒有 Kaggle 名次時，可以呈現自己測試集的預測結果及評估指標。

所以我們不需要換資料集，也不需要重新做模型。

我建議的做法是：**將現在的 `.ipynb` 上傳至 Kaggle，公開完整分析成果，並在 PDF 報告附上 Kaggle Notebook 連結。**

二、Step 23：把 HW2 上傳到 Kaggle

你現在已經有完整的本機 Jupyter Notebook，我們可以直接利用它，不需要重新寫程式。

Step 23-1：先整理本機檔案

確認你的 VS Code 專案中有：

```
7114056XXX_hw2/
├── 7114056XXX_hw2.ipynb
├── data/
│   └── StudentPerformanceFactors.csv
├── figures/
└── models/
```

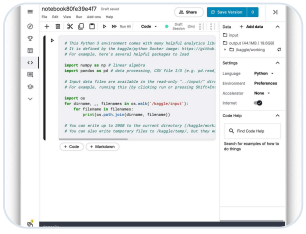
建議在 VS Code 執行：

Kernel → Restart → Run All

確認程式可以從頭執行到最後，所有模型評估、圖表與預測區間都成功產生。

這很重要，因為 Kaggle 上的 Notebook 執行環境和你本機 Windows 的檔案路徑不同。

Step 23-2：開啟 Kaggle Notebook



Kaggle Code

開啟 Kaggle Code

登入 Kaggle 後，找到建立 Notebook 的功能。

Kaggle 官方支援 Jupyter Notebook，可以加入 Kaggle Dataset 作為資料來源，也能儲存並公開 Notebook 版本。[Getting Started on Kaggle]

操作順序：

1. 開啟 Kaggle Code。

2. 點選 **New Notebook**。
3. 在 Notebook 編輯器中找到 **File → Import Notebook**（介面名稱可能略有不同）。
4. 選擇你本機的 `7114056XXX_hw2.ipynb`。
5. 確認原本的 Markdown、Python 程式及圖表內容都保留下來。

Step 23-3：加入 Kaggle Dataset

進入 Kaggle Notebook 編輯畫面後，找到 **Add Input / Add Data**。

搜尋：

`Student Performance Factors`

選擇原本使用的 `lainguyn123` 資料集，將它加入 Notebook。

完成後，就可以使用 Kaggle 提供的資料路徑讀取 CSV。

Step 23-4：修改 Kaggle 的資料路徑

你目前本機使用：

```
df = pd.read_csv("data/StudentPerformanceFactors.csv")
```

在 Kaggle 的環境中，改成：

```
df = pd.read_csv(
    "/kaggle/input/student-performance-factors/StudentPerformanceFactors.csv"
)
```

實際路徑要以 Kaggle Notebook 的 Input 檔案清單顯示為準。

另外，你的程式目前將圖表儲存至 `figures/`，模型儲存至 `models/`。

因為 Kaggle 的可寫工作目錄與本機不同，建議在 Notebook 前面加入：

```
from pathlib import Path

Path("figures").mkdir(exist_ok=True)
Path("models").mkdir(exist_ok=True)
```

Kaggle 一般可以在 `/kaggle/working/` 下建立輸出檔案，因此請確認 Notebook 執行時的目前工作目錄。

Step 23-5：儲存與執行

當所有程式都修改完成後：

1. 點選 **Save Version**。
2. 選擇 **Save & Run All**。
3. 等待 Notebook 全部執行完成。
4. 確認圖表和模型指標都有顯示。
5. 將 Notebook 設定為 **Public**。
6. 複製 Kaggle Notebook 的公開網址。

`Save & Run All` 會在乾淨的環境中重新執行整份 Notebook，因此很適合驗證你的作業是否可以重現。[Getting Started on Kaggle]

三、Kaggle 頁面要展示哪些成果？

我建議你把 Kaggle Notebook 的名稱設成：

Student Exam Score Prediction | Multiple Linear Regression & RFE

Notebook 的內容至少要讓老師看到：

- Kaggle 資料集來源及 CRISP-DM 六階段。
- EDA：考試成績分布、相關係數熱力圖。
- Model A、B、C 三種線性回歸的結果。
- RFE 選出的 25 個特徵。
- MAE、RMSE、R^2 比較表。
- Actual vs. Predicted 圖。
- 95% Prediction Interval（Coverage 95.08%）。
- Deployment 單筆預測示範。

這樣你的 Kaggle Notebook 就不只是上傳程式，而是一份可以公開展示的完整資料科學作品。

四、報告的 Kaggle 成果章節

可以在 PDF 報告中加入以下段落：

Kaggle 成果展示與模型評估

本研究使用 Kaggle 公開資料集 Student Performance Factors 進行學生成績預測分析，並使用 Jupyter Notebook 完成資料前處理、探索式資料分析、多元線性回歸建模、RFE 特徵選擇及模型評估。

本研究將完整分析流程公開於 Kaggle Notebook，包含資料分析程式碼、預測結果圖表、模型評估指標以及 95% 預測區間。

由於本研究使用的 Student Performance Factors 為一般公開資料集，而非 Kaggle Competition，因此無官方排行榜名次。本研究以獨立測試集之 MAE、RMSE 與 R^2 作為主要成果評估方式。

最終 Model C 使用 25 個編碼後特徵，測試集 MAE 為 0.4157、RMSE 為 1.5210、R^2 為 0.8251。此外，透過 Split Conformal Prediction 建立的 95% 預測區間，其測試集涵蓋率達到 95.08%，平均區間寬度為 1.6089 分。

Kaggle Notebook 連結：（成功公開後填入）

Kaggle 排行榜名次： 不適用（非 Competition 資料集）。

五、現在先做 Kaggle 上傳的第一步

我建議先不要急著參加其他 Kaggle Competition，因為那通常要使用另一份比賽資料，會改變目前的研究題目。

先將這份 HW2 公開到 Kaggle，就能同時滿足資料來源、成果展示、模型評估，以及作品連結的需求。

你先開啟 Kaggle Code，建立一個 Notebook，並嘗試匯入你目前的 `.ipynb`。

匯入後，把 Kaggle 畫面的截圖傳給我。我會繼續一步一步教你加入原始 Dataset、修改路徑、解決執行環境差異，最後公開 Notebook 並取得連結。

2026/10/16 下午3:47:08 · gpt-6-thinking

那算了 你開始教我撰寫報告

2026/10/16 下午3:48:26

好！我們先不處理 Kaggle 上傳，直接開始撰寫 **HW2 正式報告**。

你目前已經完成所有主要實驗，包括 CRISP-DM 六個階段、三個線性回歸模型、RFE 特徵選擇、95% 預測區間，以及 NotebookLM 研究。

接下來我會帶你使用 **Microsoft Word 撰寫報告，最後匯出 PDF**。我們一次完成一個章節，確保每段內容都有實際數據支持，也符合老師的評分要求。

一、正式報告的整體架構

我建議報告名稱為：

基於多元線性回歸與特徵選擇之學生成績預測研究

Student Exam Score Prediction Using Multiple Linear Regression and Feature Selection

整份報告預計 15～20 頁（不包含 GPT 對話紀錄附錄）。

章節	內容	預計頁數
封面	題目、學號、姓名、課程資訊	1
摘要	研究目的、方法、結果與結論	1
目錄	章節與頁碼	1
第一章	Business Understanding	1
第二章	Data Understanding	2～3
第三章	Data Preparation	1～2
第四章	Modeling	2～3
第五章	Evaluation	3～4
第六章	Deployment	1
第七章	NotebookLM 摘要與研究比較	2

章節	內容	預計頁數
第八章	GPT 輔助內容與研究結論	1～2
參考文獻	Kaggle、學術論文、GitHub	1
附錄	GPT 對話 PDF、其他資料	另計

這樣能直接對應老師的評分標準，特別是 CRISP-DM 流程完整性、特徵選擇、模型評估，以及 AI 協助內容。

二、Step 1：建立 Word 報告

先開啟 Microsoft Word，新增空白文件。

建議設定：

項目	建議
紙張	A4
邊界	上下左右 2.5 cm
中文字型	微軟正黑體
英文字型	Arial
內文字級	12 pt
行距	1.5 倍
第一層標題	16 pt、粗體
第二層標題	14 pt、粗體
圖表標題	10～11 pt
頁碼	頁尾置中

檔名先儲存為：

`7114056XXX_hw2_report.docx`

最後完成後再轉成 PDF。

三、Step 2：製作封面

第一頁建議使用簡潔的學術報告格式。

資料探勘與機器學習作業報告

Homework 2

基於多元線性回歸與特徵選擇之學生成績預測研究

Student Exam Score Prediction Using Multiple Linear Regression and Feature Selection

課程名稱　　：_____

授課教師　　：_____

學生姓名　　：_____

學生學號　　：7114056XXX

繳交日期　　：_____

這只是版面示意，依學校或老師是否有指定格式再調整。

四、Step 3：撰寫研究摘要

這部分你不必重新分析，因為我們已經有所有實驗結果。

正式摘要建議包含：

1. 研究背景與目的。
2. 使用的 Kaggle 資料集。
3. 三個比較模型與 RFE 特徵選擇。
4. 最佳模型的評估結果。
5. 95% 預測區間。
6. 研究發現與限制。

以下是根據你實際成果撰寫的版本。

摘要

本研究以多元線性回歸（Multiple Linear Regression）與特徵選擇（Feature Selection）方法，探討影響學生考試成績的相關因素，並依循 CRISP-DM 流程完成資料理解、資料前處理、模型建立、評估及部署。研究採用 Kaggle 公開的 Student Performance Factors 資料集，原始資料共包含 6,607 筆學生紀錄與 19 個輸入特徵，涵蓋學習時間、出席率、過往成績、家庭背景及學校環境等面向。

本研究首先建立單純線性回歸模型（Model A）作為基準，再使用全部特徵建立多元線性回歸模型（Model B），最後透過遞迴特徵消除法（Recursive Feature Elimination, RFE）與五折交叉驗證建立特徵選擇模型（Model C），比較不同模型的預測能力。

實驗結果顯示，Model A 的測試集 R^2 為 0.2469，而 Model B 提升至 0.8250。Model C 使用 25 個編碼後特徵，測試集 MAE 為 0.4157、RMSE 為 1.5210、R^2 為 0.8251，顯示透過特徵選擇能略微降低模型複雜度，同時維持接近完整模型的預測效能。

此外，本研究利用 Split Conformal Prediction 建立 95% 預測區間，其測試集涵蓋率為 95.08%，平均區間寬度為 1.6089 分。殘差分析顯示，模型對大部分學生具有良好的預測能力，但對少數極端高分學生仍存在明顯低估情況。

綜合而言，多元線性回歸搭配 RFE 能為此資料集提供有效且相對容易解釋的預測方法。然而，由於本研究使用合成資料，結果主要作為機器學習方法的驗證，不宜直接推論真實教育環境中的因果關係。

關鍵字： 多元線性回歸、特徵選擇、RFE、CRISP-DM、學生成績預測、預測區間

這個摘要有三個優點：使用你自己實際跑出的數據、明確指出特徵選擇的貢獻，也沒有隱藏模型對極端高分紀錄的限制。

五、Step 4：建立自動目錄

摘要完成後，新增一頁作為目錄。

在 Word 中，先把各章標題套用「標題 1」，各小節套用「標題 2」。

接著選擇：

參考資料 → 目錄 → 自動目錄

建議最後的章節結構如下：

目錄

摘要

第一章　Business Understanding

第二章　Data Understanding

第三章　Data Preparation

第四章　Modeling

第五章　Evaluation

第六章　Deployment

第七章　NotebookLM 摘要與主流方法比較

第八章　GPT 輔助內容與結論

參考文獻

附錄：ChatGPT 對話紀錄

目前不用先輸入頁碼，等報告全部寫完，再更新自動目錄即可。

六、Step 5：撰寫第一章 Business Understanding

這一章主要回答三個問題：

- 為什麼要研究學生成績預測？
- 為什麼使用多元線性回歸？
- 我們想透過模型比較驗證什麼？

可以直接依照下面的內容撰寫。

第一章　Business Understanding

1.1 研究背景與動機

隨著資料分析與機器學習技術的發展，教育領域逐漸重視以資料驅動的方式了解學生學習表現。學生的考試成績可能受到學習時間、出席率、過往成績、家庭支持及學校環境等多種因素影響。因此，如何整合不同特徵建立預測模型，成為教育資料分析中的重要研究議題。

線性回歸是監督式學習中常用的預測方法，具有結構簡單、計算效率高及相對容易解釋的特性。然而，當輸入特徵數量增加時，並非所有特徵都能提供同等程度的預測資訊。因此，除了建立多元線性回歸模型外，如何透過特徵選擇降低模型複雜度，同時維持良好的預測能力，也是值得探討的問題。

本研究以 Kaggle 公開的 Student Performance Factors 資料集為基礎，使用多元線性回歸與 RFE 特徵選擇方法，分析不同特徵組合對學生成績預測效果的影響，並透過獨立測試資料

進行模型評估。

1.2 研究目的

本研究主要包含以下三項目的：

1. 建立單純線性回歸與多元線性回歸模型，比較不同特徵數量對考試成績預測能力的影響。
2. 使用 Recursive Feature Elimination（RFE）與五折交叉驗證進行特徵選擇，確認是否能以較少特徵維持良好的預測效果。
3. 使用 MAE、RMSE、R^2、殘差分析及 95% 預測區間，完整評估模型準確度與預測不確定性。

1.3 研究問題與假設

為達成上述研究目的，本研究提出以下假設：

H1： 使用多個輸入特徵的多元線性回歸模型，其預測能力優於僅使用讀書時間的單純線性回歸模型。

H2： 讀書時間、出席率與過往成績等學習相關特徵，對考試成績具有一定程度的預測貢獻。

H3： 透過 RFE 特徵選擇，可以減少模型使用的特徵數量，同時維持接近完整模型的預測效能。

本研究將於後續 Modeling 與 Evaluation 階段，根據實際實驗結果檢視上述研究假設。

1.4 研究範圍與限制

本研究使用 Student Performance Factors 合成資料集，共包含 6,607 筆學生紀錄及 19 個原始輸入特徵，預測目標為 Exam_Score。

研究方法以線性回歸為主，包含單純線性回歸、完整多元線性回歸，以及搭配 RFE 特徵選擇的多元線性回歸，並參考現有學術研究中的 Random Forest、XGBoost 與 LightGBM 等方法進行文獻比較。

由於研究資料並非真實教育現場蒐集的學生紀錄，因此模型結果主要反映合成資料中的統計關聯與預測能力，不能直接推論真實學生的學習行為或因果關係。

此外，本研究使用的 Kaggle 資料屬於一般公開 Dataset，非 Kaggle Competition，因此沒有官方排行榜名次。研究成果將以獨立測試集預測結果、評估指標及圖表進行呈現。

七、這次先完成報告前半部

正式報告撰寫進度 0/5 已完成

- [ ] 建立 Word 文件並設定 A4、字體與行距
- [ ] 完成封面與基本資訊
- [ ] 貼上研究摘要並檢查實驗數據
- [ ] 建立報告目錄與標題樣式
- [ ] 完成第一章 Business Understanding

完成第一章，繼續第二章 →

下一步是 **第二章 Data Understanding**。我們會把你之前製作的三張 EDA 圖表、6,607 筆資料的特性、三個欄位的缺失值及 101 分異常紀錄整理成完整的分析章節。

這一章我會特別教你如何配置圖表與說明文字，讓報告看起來像一份正式的資料科學研究，而不是只有程式截圖。

2026/10/16 下午3:48:31 · gpt-6-thinking

太麻煩了 簡化

2026/10/16 下午3:50:02

可以，我們把報告簡化成 **8～10 頁左右**，不追求篇幅，只求符合老師的評分標準。

你已經完成所有程式與模型實驗，所以報告只需要整理結果，不必重新分析。

一、簡化後的報告架構

頁數	內容	要放什麼
第 1 頁	封面＋摘要	研究目的、資料集、最佳模型成果
第 2 頁	Business Understanding	研究動機與目標
第 3 頁	Data Understanding	資料來源、缺失值、3 張 EDA 圖
第 4 頁	Data Preparation	資料清理、80/20 切分、One-Hot Encoding
第 5 頁	Modeling	Model A、B、C 與 RFE
第 6～7 頁	Evaluation	模型比較表、預測圖、殘差圖、95% 預測區間
第 8 頁	Deployment＋Conclusion	單筆預測結果、研究結論
第 9 頁	NotebookLM＋GPT 輔助	研究摘要、方法比較、GPT 使用說明
附錄	GPT 對話紀錄	匯出 PDF，合併或另外附上

頁數可以彈性調整，不需要硬塞到剛好 9 頁。

二、最重要的模型成果

這張表直接放進報告：

模型	MAE	RMSE	R^2
Model A：Simple LR	2.4190	3.1559	0.2469
Model B：Multiple LR	0.4160	1.5213	0.8250

模型	MAE	RMSE	R^2
Model C：Multiple LR + RFE	**0.4157**	**1.5210**	**0.8251**

結果說明簡單寫：

> *本研究比較三種線性回歸模型，結果顯示 Model C 的預測表現最佳，R^2 為 0.8251。透過 RFE 特徵選擇，模型由 27 個編碼後特徵減少至 25 個，仍維持與完整模型相近的準確度。此外，95% 預測區間涵蓋率達到 95.08%，但模型對少數極端高分學生仍存在較大的預測誤差。*

三、圖表也不用全部放

我建議只保留最重要的 **5 張圖**：

1. Correlation Heatmap
2. RFE Feature Selection Performance
3. Model C：Actual vs. Predicted
4. Model C：Residual Plot
5. 95% Prediction Interval

其他圖表留在 `.ipynb` 即可。

四、最省時間的完成方式

你不用自己一章一章寫。

我可以根據我們已經完成的實驗數據，直接替你整理成一份簡潔的 Word 報告，包含 CRISP-DM 六階段、模型比較、NotebookLM 摘要、GPT 輔助說明，以及資料來源。

你只需要提供 **你的** `.ipynb` **檔案**，我就能從裡面整理程式執行結果與圖表，避免你手動複製。

完成後你只要：

- 補上姓名、學號、課程資訊。
- 檢查報告內容。
- 將 Word 匯出 PDF。
- 匯出 ChatGPT 對話紀錄 PDF。
- 把程式、資料與報告壓縮成 ZIP。

你現在直接上傳 `7114056XXX_hw2.ipynb` 給我就好。 我們用最省事的方式完成報告。

2026/10/16 下午3:50:04 · gpt-6-thinking

CRISP-DM有哪些步驟

2026/10/16 下午3:55:21

CRISP-DM（Cross-Industry Standard Process for Data Mining）是資料探勘的標準流程，共有 **6 個步驟**。

以下直接用你的 HW2「學生成績預測」作為例子：

步驟	名稱	你的 HW2 對應內容
1	**Business Understanding**（業務理解）	確定研究目標：利用學生特徵預測考試成績
2	**Data Understanding**（資料理解）	檢查 6,607 筆資料、缺失值、相關係數與 EDA 圖表
3	**Data Preparation**（資料準備）	處理異常值、填補缺失值、One-Hot Encoding、80/20 資料切分
4	**Modeling**（模型建立）	建立 Model A、B、C，使用 RFE 進行特徵選擇
5	**Evaluation**（模型評估）	比較 MAE、RMSE、R^2，繪製預測圖、殘差圖及 95% 預測區間
6	**Deployment**（模型部署）	使用 joblib 儲存模型，重新載入並預測學生考試成績

CRISP-DM 流程圖

清理及轉換資料

↓

4. Modeling

建立與訓練模型

↓

5. Evaluation

評估模型效果

↓

6. Deployment

儲存與應用模型

你的作業目前進度

✅ **六個階段都已完成。**

你已經建立三個模型，最佳的 Model C 得到：

- **MAE：** 0.4157
- **RMSE：** 1.5210
- **R^2：** 0.8251
- **95% 預測區間涵蓋率：** 95.08%

因此，你現在寫報告時，只要按照這六個標題，分別說明「做了什麼、用了什麼方法、得到什麼結果」，就能滿足老師要求的 CRISP-DM 流程說明。

另外，NotebookLM 摘要、GPT 輔助內容和網路主流方法比較，可以放在這六個步驟之後，作為獨立章節。

2026/10/16 下午3:55:24 · gpt-6-thinking

1. 資料集來源與研究脈絡
2. CRISP-DM 流程完整且邏輯清楚
(1) Business Understanding
(2) Data Understanding
(3) Data Preparation
(4) Modeling
(5) Evaluation
(6) Deployment
3. 結果合理、美觀且具有說服力:
4. 預測結果評估:
5. GPT對話
6. NotebookLM 摘要

2026/10/16 下午3:58:41

可以！我們就按照你列出的 **6 個大項**來製作 HW2 報告，不需要再增加其他章節，這樣最簡單，也完全對應老師的評分要求。

HW2 最終報告架構

章節	內容	建議放入的資料
1. 資料集來源與研究脈絡	Kaggle 資料來源、研究目的、資料集介紹	資料集連結、6,607 筆資料、19 個特徵
2. CRISP-DM 流程	六大階段的分析過程	資料處理方法、模型建立過程
3. 結果分析與討論	特徵選擇成果、模型差異與研究發現	RFE 圖、特徵分析、研究結論
4. 預測結果評估	模型評估指標、預測圖及預測區間	MAE、RMSE、R^2、殘差圖、95% 預測區間
5. GPT 對話	說明 ChatGPT 如何協助完成作業	GPT 輔助內容、完整對話 PDF
6. NotebookLM 摘要	文獻研究及網路主流方法比較	NotebookLM 摘要、LightGBM 等模型比較

第 2 章：CRISP-DM 細分

你列出的六個小節可以直接照這樣寫：

小節	簡單說明
2.1 Business Understanding	研究目的是使用線性回歸預測學生成績，並探討特徵選擇的效果。
2.2 Data Understanding	使用 EDA 分析資料分布、缺失值與各特徵的相關性。
2.3 Data Preparation	排除一筆 101 分紀錄、填補缺失值、類別編碼及資料切分。
2.4 Modeling	建立 Simple LR、Multiple LR、Multiple LR + RFE 三個模型。
2.5 Evaluation	使用 MAE、RMSE、R^2 評估模型，分析殘差與預測區間。
2.6 Deployment	使用 joblib 儲存模型，並示範單筆學生成績預測。

需要放進報告的重點圖表

建議只保留這 **5 張圖**，其他圖留在 Notebook 裡即可。

1. Correlation Heatmap — 資料特徵相關性
2. RFE Feature Selection Performance — 特徵選擇成果
3. Model C Actual vs. Predicted — 模型預測表現
4. Model C Residual Plot — 殘差分析
5. 95% Prediction Interval — 預測區間

模型評估結果

這張表放在第 4 章。

模型	MAE ↓	RMSE ↓	R^2 ↑
Model A	2.4190	3.1559	0.2469
Model B	0.4160	1.5213	0.8250
Model C	**0.4157**	**1.5210**	**0.8251**

第 4 章再補上 95% 預測區間的 Coverage **95.08%**、平均區間寬度 **1.6089 分**，以及少數高分學生預測誤差較大的限制。

我建議的方式

每個小節寫 1～2 段，搭配必要的圖表就好。 整份報告大約 7～10 頁（不含 GPT 對話附錄），不用寫得像畢業論文。

其中第 3 章負責「解釋結果與研究發現」，第 4 章負責「呈現評估數字及預測圖」，避免內容重複。

如果這就是你確定的目錄，我們接下來可以**直接從「1. 資料集來源與研究脈絡」開始撰寫正式內容**，用簡短、正式、可以直接貼到 Word 的文字完成整份報告。

2026/10/16 下午3:58:44 · gpt-6-thinking

先給我1.　資料集來源與研究脈絡

2026/10/16 下午3:59:26

1. 資料集來源與研究脈絡

1.1 資料集來源

本研究使用 Kaggle 平台提供的公開資料集 **Student Performance Factors**，由 lainguyn123 發布。資料集包含 **6,607 筆學生資料，共 20 個欄位**，其中 19 個為輸入特徵（Features），1 個為預測目標（Target）。

資料內容涵蓋學生的讀書時間、出席率、過往成績、睡眠時間、家庭背景及學校環境等因素，預測目標為 **Exam_Score（考試成績）**。

本資料集屬於合成資料（Synthetic Data），主要用於資料分析與機器學習研究。

資料集資訊：

- 資料集名稱：Student Performance Factors
- 資料來源：Kaggle
- 發布者：lainguyn123
- 資料筆數：6,607 筆
- 輸入特徵：19 個（6 個數值特徵、13 個類別特徵）
- 預測目標：Exam_Score
- 資料集連結：https://www.kaggle.com/datasets/lainguyn123/student-performance-factors

1.2 研究背景與動機

學生的考試成績可能受到多種因素影響，例如讀書時間、出席率、過往成績及家庭環境等。透過機器學習技術，可以分析這些特徵與考試成績之間的關係，並建立預測模型。

本研究選擇**多元線性回歸（Multiple Linear Regression）**作為主要預測方法，因為線性回歸具有模型結構簡單、容易理解及可解釋性較高等優點。

此外，當模型使用較多特徵時，並非所有特徵都具有相同的預測貢獻。因此，本研究加入**特徵選擇（Feature Selection）**，探討是否能減少模型使用的特徵數量，同時維持良好的預測準確度。

1.3 研究目的

本研究依循 CRISP-DM 資料探勘流程，使用學生相關特徵預測考試成績，並比較以下三種模型：

1. **Model A：Simple Linear Regression** — 僅使用讀書時間預測成績，作為基準模型。
2. **Model B：Multiple Linear Regression** — 使用全部 19 個原始輸入特徵建立預測模型。
3. **Model C：Multiple Linear Regression + RFE** — 使用遞迴特徵消除法（Recursive Feature Elimination）選擇特徵，評估模型簡化後的預測效果。

最後透過 MAE、RMSE、R^2 及 95% 預測區間評估模型表現，分析特徵選擇對預測能力的影響，並探討模型的應用限制。

這一章可以直接貼進 Word，約佔 **1～1.5 頁**，已涵蓋資料集來源、研究背景、研究動機與研究目的。

接下來的 **第 2 章 CRISP-DM** 就不必重複詳細介紹資料集，可以直接從 `2.1 Business Understanding` 開始說明實際分析流程。

2026/10/16 下午3:59:31 · gpt-6-thinking

1. CRISP-DM 流程完整且邏輯清楚
(1) Business Understanding
(2) Data Understanding
(3) Data Preparation
(4) Modeling
(5) Evaluation
(6) Deployment

2026/10/16 下午4:01:58

以下是根據你實際完成的 HW2 實驗結果整理的 **CRISP-DM 六大階段**，內容精簡、正式，可以直接貼到 Word 報告中。

2. CRISP-DM 分析流程

本研究依循 CRISP-DM（Cross-Industry Standard Process for Data Mining）資料探勘流程，依序進行業務理解、資料理解、資料準備、模型建立、模型評估及模型部署六個階段，以建立完整的學生成績預測流程。

2.1 Business Understanding（業務理解）

本研究旨在利用學生的學習習慣、家庭背景與學校環境等特徵，建立考試成績預測模型，並探討不同特徵組合對預測準確度的影響。

研究以線性回歸為主要方法，比較單純線性回歸、多元線性回歸，以及結合 RFE 特徵選擇的多元線性回歸，評估增加或減少特徵是否影響預測能力。

2.2 Data Understanding（資料理解）

本研究使用 Kaggle 的 Student Performance Factors 資料集，共包含 **6,607 筆資料、19 個輸入特徵**，其中 6 個為數值特徵、13 個為類別特徵。

透過探索式資料分析（EDA）檢查資料分布、缺失值及相關係數。結果顯示：

- 平均考試成績約為 **67.24 分**。
- Attendance（出席率）與考試成績的相關係數約為 **0.58**。
- Hours_Studied（讀書時間）與考試成績的相關係數約為 **0.45**。

- Teacher_Quality、Parental_Education_Level 及 Distance_from_Home 分別存在 78、90、67 筆缺失值。
- 資料中有一筆考試成績為 101 分，超出本研究設定的 0～100 分範圍。

上述分析顯示，出席率與讀書時間具有較明顯的正向線性關聯，可作為後續建模分析的參考。

2.3 Data Preparation（資料準備）

首先，排除一筆考試成績為 101 分的紀錄，資料由 6,607 筆減少至 **6,606 筆**。

接著將 Exam_Score 設定為預測目標，其餘 19 個欄位作為輸入特徵，並按照 80% 與 20% 的比例切分資料：

- 訓練集：5,284 筆
- 測試集：1,322 筆

在資料前處理方面，使用 SimpleImputer 填補缺失值，數值特徵透過 StandardScaler 進行標準化，類別特徵則使用 One-Hot Encoding 轉換為數值。

經過前處理後，19 個原始輸入特徵轉換為 **27 個編碼後特徵**。所有前處理步驟皆整合於 Pipeline 中，並使用訓練資料進行擬合，以避免資料洩漏。

2.4 Modeling（模型建立）

本研究建立三個線性回歸模型進行比較：

Model A：Simple Linear Regression

僅使用 Hours_Studied 作為輸入特徵，建立單純線性回歸模型，作為後續模型比較的基準。

Model B：Multiple Linear Regression

使用全部 19 個原始輸入特徵，經過資料前處理後共包含 27 個編碼欄位，建立完整多元線性回歸模型。

Model C：Multiple Linear Regression + RFE

使用遞迴特徵消除法（Recursive Feature Elimination, RFE）執行特徵選擇，並搭配五折交叉驗證（5-Fold Cross-Validation），比較不同特徵數量的模型表現。

實驗結果顯示，保留 **25 個編碼後特徵**時，交叉驗證 RMSE 最低，約為 **2.0833**。因此，Model C 最終使用 25 個特徵建立預測模型，較 Model B 減少 2 個編碼欄位。

2.5 Evaluation（模型評估）

本研究使用 MAE、RMSE 及 R^2 評估三個模型的預測能力。

模型	MAE	RMSE	R^2
Model A	2.4190	3.1559	0.2469
Model B	0.4160	1.5213	0.8250
Model C	**0.4157**	**1.5210**	**0.8251**

結果顯示，Model B 與 Model C 的預測能力明顯優於 Model A，說明使用多個特徵有助於改善學生成績預測效果。

Model C 的評估指標略優於 Model B，但差異非常小，主要優勢在於使用較少特徵維持接近完整模型的預測表現。

此外，本研究透過殘差分析發現，模型對大部分學生的預測誤差較小，但仍有少數高分學生被明顯低估。

最後，使用 Split Conformal Prediction 建立 95% 預測區間，測試集涵蓋率（Coverage）達到 **95.08%**，平均預測區間寬度為 **1.6089 分**，顯示模型對整體測試資料具有良好的邊際區間涵蓋表現，但不保證對極端高分學生同樣可靠。

2.6 Deployment（模型部署）

本研究使用 joblib 儲存完成訓練的 Model C，以及用於建立預測區間的校準模型，並透過重新載入模型測試單筆學生成績預測功能。

實際測試結果如下：

項目	結果
Model C 預測成績	66.01
區間模型預測成績	66.06
實際成績	66
95% 預測區間	[65.25, 66.86]

測試結果顯示，模型能成功重新載入並完成預測，且該筆學生的實際成績落在預測區間內。

由於本研究使用合成資料，模型主要適用於教學及方法驗證，不宜直接應用於真實教育決策。

建議搭配的圖表

這一章不需要每個階段都放圖，只要加入以下三張，就能讓流程更清楚：

放置位置	圖表
2.2 Data Understanding	Correlation Heatmap
2.4 Modeling	RFE Feature Selection Performance
2.5 Evaluation	Model C Actual vs. Predicted

殘差圖、95% 預測區間圖及完整評估指標，可以留到你下一個章節「3. 結果合理、美觀且具有說服力」與「4. 預測結果評估」再詳細呈現，避免重複。

這樣 CRISP-DM 的六個階段都有**實際執行的方法、資料數字及研究結果**，也能與後面的結果分析章節順利銜接。

2026/10/16 下午4:02:00 · gpt-6-thinking

3. 結果合理、美觀且具有說服力

2026/10/16 下午4:11:11

以下是第 3 章「**結果合理、美觀且具有說服力**」的報告內容，根據你實際執行的實驗結果撰寫，可以直接貼到 Word。

3. 結果分析與討論

3.1 模型比較結果

本研究比較單純線性回歸（Model A）、多元線性回歸（Model B）及結合 RFE 特徵選擇的多元線性回歸（Model C）。

實驗結果顯示，Model A 僅使用讀書時間進行預測，R^2 為 0.2469，表示單一特徵的預測能力有限。當 Model B 加入全部 19 個原始特徵後，R^2 提升至 0.8250，顯示使用多個特徵能有效改善預測表現。

Model C 透過 RFE 特徵選擇，將編碼後特徵由 27 個減少至 25 個，R^2 達到 0.8251，與 Model B 幾乎相同。這表示適當移除部分特徵後，仍可維持良好的預測準確度。

3.2 特徵選擇結果

本研究使用 RFE 搭配五折交叉驗證，比較保留 5、10、15、20、25 及 27 個特徵時的模型表現。

實驗結果顯示，保留 25 個特徵時，交叉驗證 RMSE 最低，為 2.0833。RFE 最終移除了 Sleep_Hours（睡眠時間）及 School_Type_Public（學校類型編碼欄位），保留其餘 25 個特徵。

此結果與先前的相關係數分析部分一致，例如 Sleep_Hours 與考試成績的相關係數約為 -0.02，顯示其線性相關程度較低。

不過，特徵選擇結果僅反映目前資料與模型設定下的預測貢獻，不能直接解讀為這些因素在真實教育環境中不重要。

【此處插入圖 3-1：RFE Feature Selection Performance】

3.3 預測結果分析

從 Model C 的 Actual vs. Predicted 散佈圖可以觀察到，大部分預測結果接近理想預測線，尤其在主要成績區間內，預測值與實際值相當接近。

然而，少數實際成績較高的學生仍存在明顯低估情況。例如部分學生實際成績為 84～98 分，但模型僅預測約 61～73 分，顯示模型對極端高分資料的預測能力仍有限。

【此處插入圖 3-2：Model C Actual vs. Predicted】

3.4 研究結果討論

綜合三個模型的實驗結果，可以得到以下發現：

1. **多元線性回歸優於單純線性回歸：** 加入出席率、過往成績及其他特徵後，模型預測能力明顯提升。
2. **特徵選擇能略微降低模型複雜度：** RFE 減少了約 7.4% 的編碼後特徵數量，仍維持與完整模型接近的預測表現。
3. **模型對極端高分仍有限制：** 大部分樣本的預測誤差較小，但少數高分紀錄存在明顯低估，顯示整體評估指標無法完整反映每筆資料的預測情況。

整體而言，本研究證實多元線性回歸能有效預測此合成資料集中的學生成績，而 RFE 可以在不明顯影響預測效能的情況下簡化模型。不過，由於研究使用合成資料，結果不宜直接推廣至真實教育情境。

這一章要放的圖片

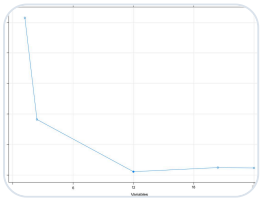


圖 3-1：RFE Feature Selection Performance

使用你先前產生的 `06_rfe_cv_results.png`。

放在 3.2 節，用來證明為什麼選擇 25 個特徵。

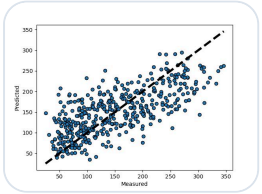


圖 3-2：Model C Actual vs. Predicted

使用 `07_model_C_predictions.png`。

放在 3.3 節，呈現模型的實際預測效果。

以上圖片是圖表類型示意，報告中請使用你自己執行 Python 所產生的正式圖片。

這一章建議控制在 **1～2 頁**，放兩張圖即可。下一章「4. 預測結果評估」再集中呈現 MAE、RMSE、R^2 比較表、殘差圖，以及 95% 預測區間，避免內容過度重複。

2026/10/16 下午4:11:15 · gpt-6-thinking

4. 預測結果評估

2026/10/16 下午4:14:58

以下是第 4 章「**預測結果評估**」的正式報告內容，根據你實際執行的模型數據撰寫，可以直接貼到 Word。

4. 預測結果評估

4.1 模型評估指標

本研究使用 MAE（平均絕對誤差）、RMSE（均方根誤差）及 R^2（決定係數）作為模型評估指標，並以獨立測試集比較三種線性回歸模型的預測能力。

實驗結果如下：

模型	MAE ↓	RMSE ↓	R^2 ↑
Model A：Simple Linear Regression	2.4190	3.1559	0.2469
Model B：Multiple Linear Regression	0.4160	1.5213	0.8250
Model C：Multiple Linear Regression + RFE	**0.4157**	**1.5210**	**0.8251**

由結果可知，Model B 與 Model C 的預測表現明顯優於 Model A，顯示加入多個特徵有助於提升考試成績的預測能力。

其中 Model C 的 MAE 為 0.4157，代表測試資料的平均絕對預測誤差約為 0.42 分；R^2 為 0.8251，顯示模型能解釋測試資料約 82.51% 的成績變異。

雖然 Model C 的三項指標略優於 Model B，但差異非常小。因此，Model C 的主要優勢是減少兩個編碼後特徵，並維持與完整模型相近的預測效能，而非顯著提升準確度。

4.2 實際值與預測值比較

本研究使用 Actual vs. Predicted 散佈圖，比較 Model C 對測試資料的預測結果。

從圖中可以觀察到，大部分預測點接近理想預測線，表示模型對主要成績區間具有良好的預測能力。

然而，部分實際成績較高的學生仍出現明顯低估情況。例如實際成績為 98 分的學生，模型僅預測約 69.05 分，顯示模型對少數極端高分紀錄的預測仍存在限制。

【此處插入圖 4-1：Model C Actual vs. Predicted】

4.3 殘差分析（Residual Analysis）

為進一步了解模型的預測誤差，本研究使用殘差圖分析實際成績與預測成績之間的差異。

分析結果顯示，大部分殘差集中在 0 附近，表示模型對多數學生的預測誤差較小。

不過，仍有五筆資料的正殘差約介於 15.17～28.95 分之間，表示模型對這些高分學生存在明顯低估現象。

由於 RMSE 對較大的誤差更加敏感，因此少數極端誤差對整體 RMSE 造成顯著影響。這也說明模型即使具有良好的整體預測指標，仍可能對部分學生產生較大的預測偏差。

【此處插入圖 4-2：Model C Residual Plot】

4.4 95% 預測區間（Prediction Interval）

除了預測單一考試分數外，本研究使用 Split Conformal Prediction 建立 95% 預測區間，以評估模型預測結果的不確定性。

實驗使用 1,057 筆校準資料，取得校準誤差分位數為 0.8044，最終結果如下：

評估項目	實驗結果
預測區間信心水準	95%
測試集涵蓋率（Coverage）	**95.08%**
平均預測區間寬度	**1.6089 分**
校準集筆數	1,057

結果顯示，約 95.08% 的測試資料實際成績落在預測區間內，整體涵蓋率接近預先設定的 95% 目標。

然而，部分極端高分學生仍落在預測區間之外，表示此方法提供的是整體邊際涵蓋率，而非對每個學生或每個分數區間皆保證 95% 涵蓋率。

【此處插入圖 4-3：95% Prediction Interval】

4.5 綜合評估

綜合模型評估結果，Model C 在保留 25 個編碼後特徵的情況下，取得 MAE 0.4157、RMSE 1.5210 及 R^2 0.8251，為本研究三個模型中測試指標數值最好的模型，但與 Model B 的差距極小。

此外，95% 預測區間的整體涵蓋率達到 95.08%，顯示模型能對多數測試資料提供有效的預測範圍。

不過，殘差分析顯示模型對少數極端高分學生仍有明顯低估情況。未來可進一步研究這些紀錄的特性，並比較非線性模型是否能改善高分區間的預測效果。

整體而言，本研究完成多元線性回歸的預測能力評估，並透過特徵選擇、殘差分析與預測區間，呈現模型的優點與限制。

這一章建議放的 3 張圖

圖表	你的檔案	放置位置
圖 4-1：Actual vs. Predicted	`07_model_C_predictions.png`	4.2 節
圖 4-2：Residual Plot	`09_model_C_residuals.png`	4.3 節
圖 4-3：95% Prediction Interval	`10_prediction_interval.png`	4.4 節

注意： 第 3 章已經使用過 Model C 的 Actual vs. Predicted 圖，建議將正式圖片放在第 4 章即可，第 3 章只保留文字討論，避免重複。

另外，因為你使用的是一般 Kaggle Dataset，而不是 Kaggle Competition，**沒有官方排行榜名次**。老師的規範是「Kaggle 名次（若有）／預測結果評估」，因此本章的三模型評估指標、預測圖及 95% 預測區間，就能提供這項評分所要求的預測評估成果。

2026/10/16 下午4:15:01 · gpt-6-thinking

5. NotebookLM 摘要：網路上主流或更優解法之比較與說明

2026/10/16 下午4:22:26

以下是第 5 章「**NotebookLM 摘要：網路上主流或更優解法之比較與說明**」，根據你實際使用 NotebookLM 產生的研究摘要，以及 HW2 的實驗結果整理，可以直接貼到 Word 報告中。

5. NotebookLM 摘要：網路上主流或更優解法之比較與說明

5.1 NotebookLM 研究摘要

本研究使用 Google NotebookLM 輔助閱讀與整理相關學術文獻，探討不同機器學習方法在學生成績預測上的應用。

參考文獻為 *Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study*（2025）。該研究使用 Student Performance Factors 合成資料集，包含 6,607 筆學生資料，並比較 Random Forest、XGBoost 與 LightGBM 三種集成學習模型的預測表現。

研究結果顯示，**LightGBM 的整體表現最佳**，MAE 為 0.804、RMSE 為 1.957、R^2 為 0.729。Random Forest 的 RMSE 為 2.175、R^2 為 0.665；XGBoost 的 RMSE 為 2.226、R^2 為 0.649，但 XGBoost 的 MAE 較 Random Forest 低。

此外，研究使用 SHAP（SHapley Additive exPlanations）分析模型的預測依據，發現出席率、讀書時間與過往成績是重要的預測因素。SHAP 也能分析個別學生各項特徵對預測結果的貢獻，提高模型的可解釋性。

相較於多元線性回歸，Random Forest、XGBoost 與 LightGBM 能夠處理較複雜的非線性關係與特徵交互作用，但模型結構相對複雜，通常需要額外的解釋方法。

由於研究使用的是合成資料，因此實驗結果主要適用於機器學習方法的比較與驗證，不能直接推論真實教育情境中的因果關係。

5.2 網路主流方法與本研究比較

本研究使用線性回歸及 RFE 特徵選擇進行成績預測，並將實驗結果與 NotebookLM 整理的學術文獻進行比較。

模型	MAE ↓	RMSE ↓	R^2 ↑
本研究：Simple LR	2.4190	3.1559	0.2469
本研究：Multiple LR	0.4160	1.5213	0.8250
本研究：Multiple LR + RFE	**0.4157**	**1.5210**	**0.8251**
文獻：Random Forest	1.084	2.175	0.665
文獻：XGBoost	0.986	2.226	0.649
文獻：LightGBM	0.804	1.957	0.729

由表中可以觀察到，本研究的 Multiple Linear Regression + RFE 模型取得 MAE 0.4157、RMSE 1.5210、R^2 0.8251，數值上優於參考文獻所報告的三種集成學習模型。

然而，**由於兩項研究在資料切分、前處理及模型訓練設定上存在差異，因此不能直接認定本研究的線性回歸模型優於 LightGBM 等方法**。若要進行公平比較，應在相同測試資料及實驗條件下重新訓練與評估各模型。

5.3 各模型優缺點比較

模型	優點	限制
Multiple Linear Regression	結構簡單、執行速度快、較容易解釋	難以直接捕捉複雜的非線性關係
Random Forest	能處理非線性關係，對特徵交互作用具有彈性	模型較複雜，解釋較不直觀
XGBoost	能處理複雜的資料關係，具有良好的預測彈性	需要調整超參數，訓練及解釋成本較高
LightGBM	訓練效率高，適合較大規模的資料	模型較不直觀，需留意過度擬合

5.4 研究比較結論

綜合 NotebookLM 的研究結果與本次實驗，可以發現不同機器學習方法各有優缺點。

本研究的多元線性回歸模型在 Student Performance Factors 合成資料集上取得良好的預測效果，且透過 RFE 特徵選擇，能以較少的特徵維持接近完整模型的準確度。

相較之下，Random Forest、XGBoost 與 LightGBM 具有處理非線性關係的能力，適合更複雜的預測問題，但通常需要更多模型調整與解釋工作。

因此，本研究認為，當資料具有較明顯的線性關係，且研究重視模型簡潔性與可解釋性時，多元線性回歸是一項合理的選擇。未來若使用真實學生資料，可在相同實驗條件下比較線性模型與集成學習模型，進一步驗證其泛化能力。

NotebookLM 研究來源：

- 研究工具：Google NotebookLM
- 參考論文：*Exploring Predictive Insights on Student Success Using Explainable Machine Learning: A Synthetic Data Study*
- 發表期刊：Information（2025）
- 論文連結：https://www.mdpi.com/2078-2489/16/9/763

建議放入報告的內容

這一章建議控制在 **1～2 頁**，不需要另外製作圖表。保留模型比較表、方法優缺點表與研究結論即可。

另外，建議你在章節最後附上 **NotebookLM 實際產生摘要的截圖**，並保留 NotebookLM 的來源引用紀錄，讓老師可以確認確實使用 NotebookLM 進行研究。

這樣就能同時滿足老師要求的「NotebookLM 100 字以上摘要」及「網路上主流或更優解法之比較與說明」。

2026/10/16 下午4:22:28 · gpt-6-thinking